

7. 存储系统

2026 春

杨莉萍
信息学院大数据系

|| 本章主要内容

- 存储器概述
- SRAM存储器
- DRAM存储器
- 并行存储器
- Cache存储器
- 虚拟存储器。

4.1 存储器概述

目录：

1. 存储器的分类
2. 存储器的分级。

4.1.1 存储器的分类

按存储介质分类：

- 半导体存储器
 - 含 双极型存储器、MOS存储器
 - 特点：速度快、功耗低
- 磁存储器
 - 磁芯存储器、磁带、磁盘
 - 特点：容量大，速度慢、体积大
- 激光存储器
 - CD-ROM DVD-ROM CD-RW ...
 - 便于携带，廉价，易于保存。

■ 随机存储器

- 特点：存取速度与存储单元的物理位置无关
- 磁芯存储器、半导体存储器 属于之。

■ 顺序存储器

- 特点: 存取速度与存储单元的物理位置有关, 只能依地址顺序访问
- 磁带存储器、激光存储器。

■ 只读存储器 (ROM)

- 存储器内容是预置的，固定的，无法改写
- 例如 半导体只读存储器、CD-ROM等。

■ 读写存储器 (RAM)

- 既能读出也能写入的存储器。

按信息的可保存性分类

- 易失性存储器

- 断电后信息消失
- 例如 SRAM、DRAM

- 非易失性存储器

- 是断电后 所保存的信息不丢失的存储器
- 磁盘存储器、光存储器CD-ROM等 。

按功能和存取速度分类

- 主存储器
- 辅存储器
- 高速缓冲存储器Cache
- 寄存器存储器。

有关存储器的重要术语 (重)

- **存储容量**：存储器可以存储的二进制信息总量称为存储容量。它代表了一个存储器可容纳的存储单元总数。存储容量有两种表示方法：
 - **1) 位表示法**：以存储器中的存储单元总数与存储字位数的乘积表示。如 $1K \times 4$ 位，表示该芯片有1K个单元 ($1K=1024$)，每个存储单元长度为4bit。
 - **2) 字节表示法**：以存储器中的单元总数表示(一个存储单元一般由8bit组成，称为1字节B)。如128B, 表示该芯片有128个存储单元。它直接表示了存储器能存储二进制信息的总容量。

- **存取时间 (存储器访问时间):** 存储器接收到一个读/写命令到从存储器中读出/写入信息 所经历的时间。通常写操作时间等于读操作时间，故都称为存储器存取时间。
- **存取(储)周期：**是连续两次独立的存储器操作(如连续两次读操作)所需的最小时间间隔。
- **存储器带宽：**
 - 指单位时间内 从存储器存取的信息量。
 - 计量单位: Byte/s。

- 80年代开始，几乎所有机器都用字节编址。
- 计算机系统设计者要考虑的两个问题：
 - 如何根据一个字节地址取到一个32位的字？
 - ◆ 考虑：字的存放问题
 - 一个字能否存放在任何字节边界？
 - ◆ 字的边界对齐问题。

数据在内存中的存放顺序 (※)

- 例: 将数据 **0x12 34 56 78** 写入到 单元地址0x0000开始的内存中.

	大端方式	小端方式
0x0000	0x <u>12</u>	<u>0x78</u>
0x0001	0x34	0x56
0x0002	0x56	0x34
0x0003	0x78	0x12

- **大端方式:** 存储器的低字节地址单元中存放的是数据的最高位字节.
 - IBM 360/370, Motorola 68k, **MIPS**, Sparc, HP PA
- **小端方式:** 存储器的低字节地址单元中存放的是数据的最低位字节.
 - **Intel 80x86**, DEC VAX
- 有些机器两种方式都支持, 由特定控制位来设定.



存储器按字节编址；

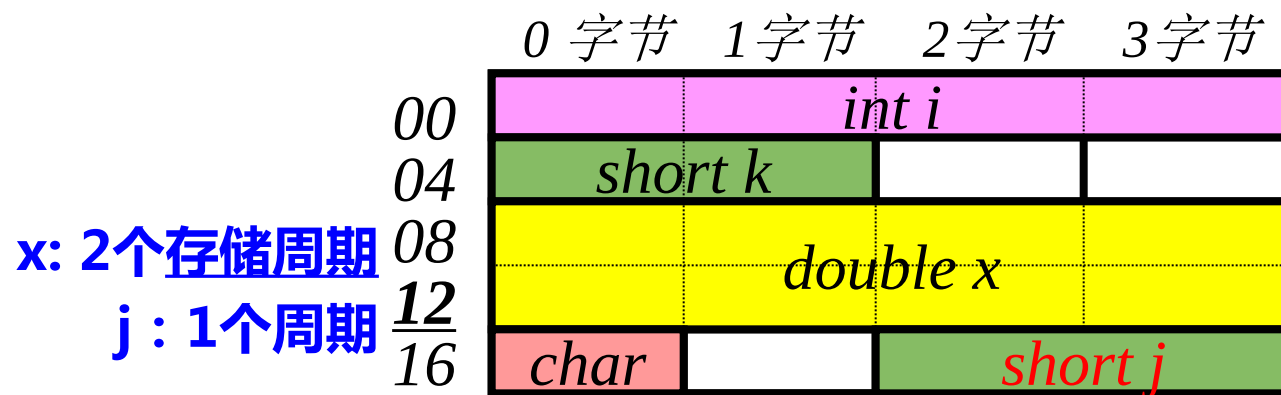
对不同类型变量的主存空间分配：

方案1(对齐模式, 右图):
每次只能读写某个字地址(如字长4B)开始的4个单元(字节)中 **连续的1个(1/4字长)、2个(半字长)或 4个字节**.

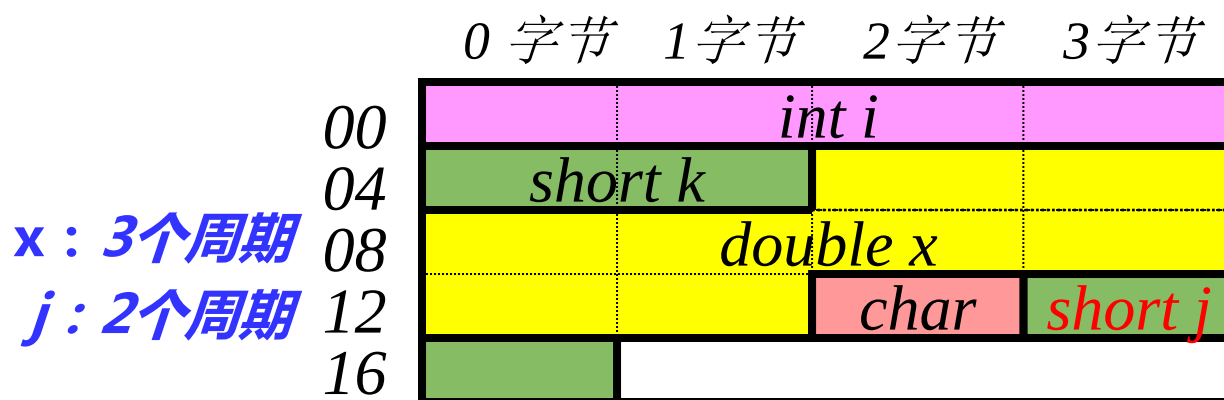
方案2(未对齐模式): 虽节省了空间，但增加了访存次数！

总结: 需要权衡; 目前情况看, 浪费一点存储空间(用对齐模式) 没有关系！

例: `int i; short k; double x; char c; short j;`



则: `&i=0; &k=4; &x=8; &c=16; &j=18` 半字对齐



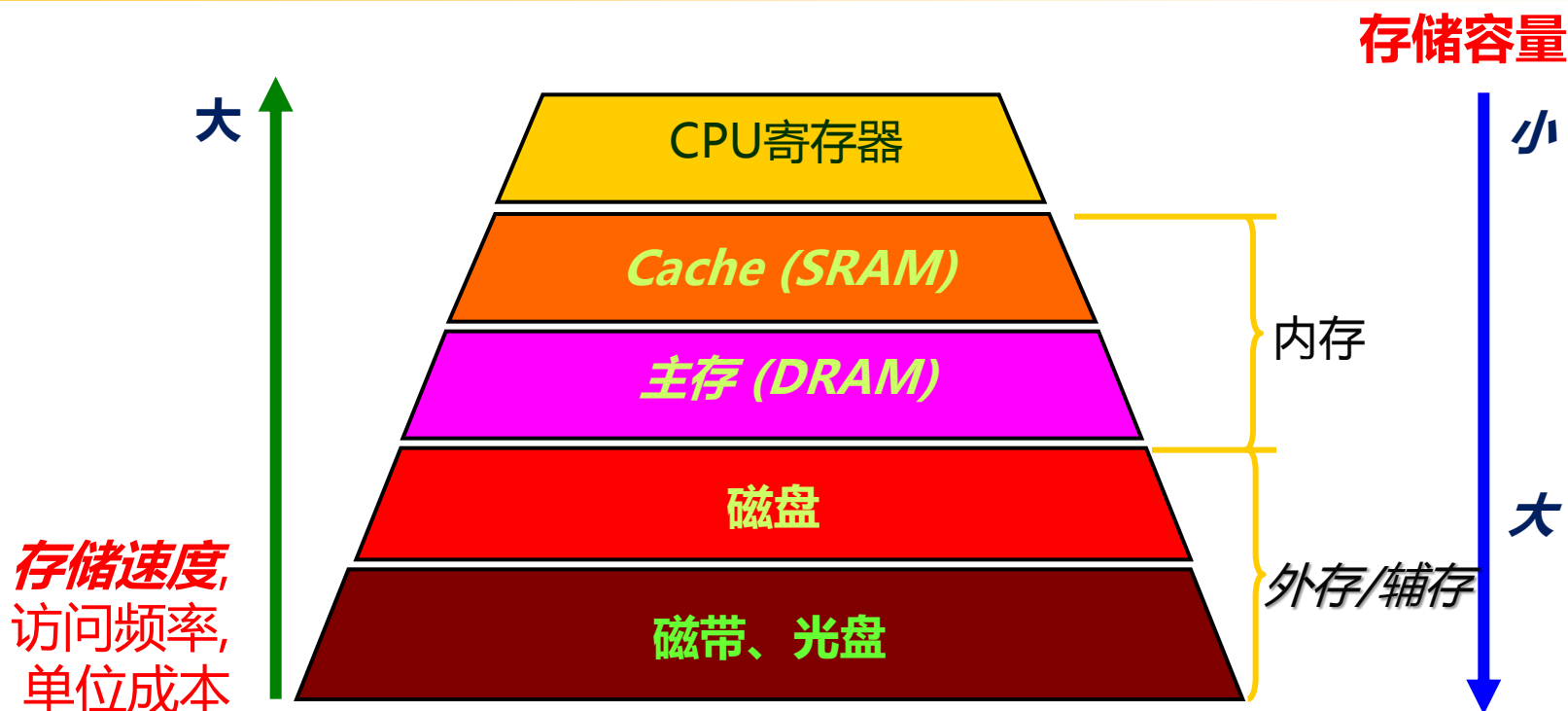
则: `&i=0; &k=4; &x=6; &c=14; &j=15; .....`

目前存储器的特点是：

- 速度快的存储器价格贵，容量小；
- 价格低的存储器速度慢，容量大。

在计算机存储器体系结构设计时，我们希望存储器系统的性能高、价格低，那么在存储器系统设计时，应当在存储器容量、速度和价格方面的因素作折中考虑，建立**分层次的存储器体系结构**，如下图所示。

存储系统分层结构 (✖)



三级存储体系

- **高速缓冲存储器** 简称cache，它是计算机系统中的一个高速、小容量的半导体存储器
- **主存储器** 简称主存，是计算机系统的主要存储器，用来存放计算机运行期间的大量程序和数据。
- **外存储器** 简称外存，它是大容量辅助存储器。

三级存储体系

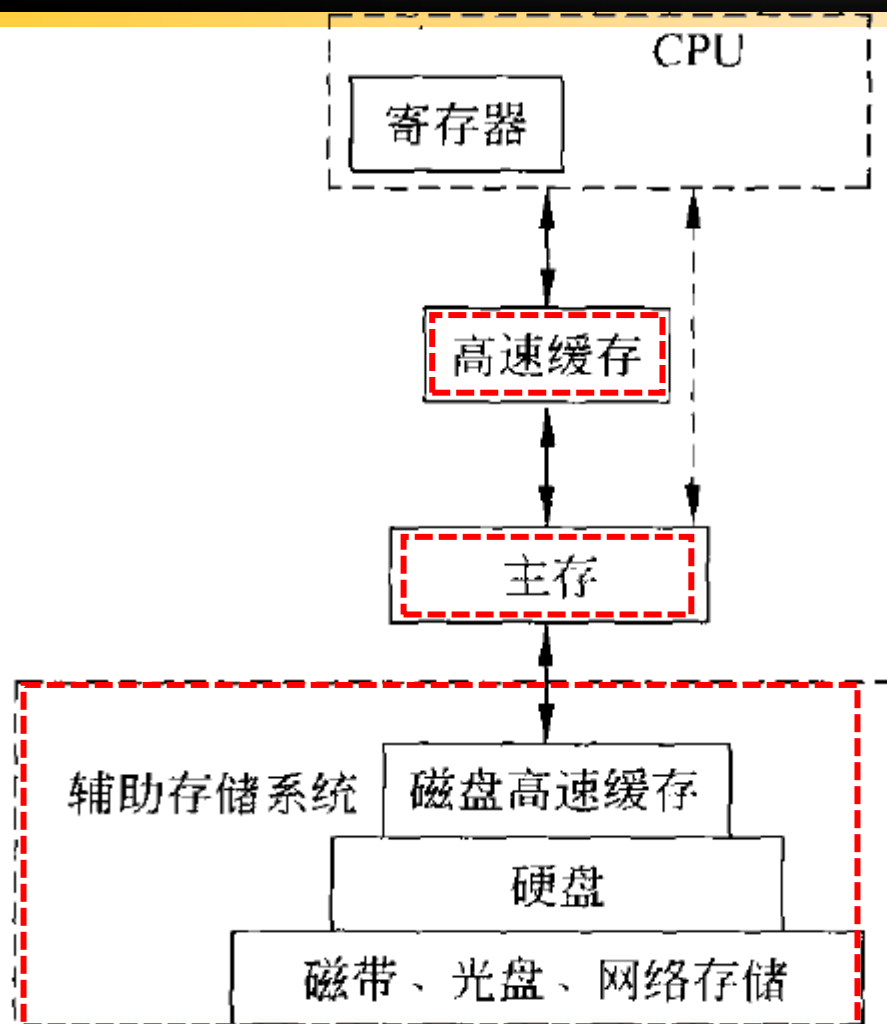


图 4.4 所示的是一个由高速缓冲存储器、主存和辅存三级存储体系构成的分级结构，3 种不同类型的存储器承担的职能各不相同。其中使用高速缓存的主要目的是提高 CPU 对存储系统的访问速度，缓解快速 CPU 与主存

之间的速度差异问题；使用辅存的目的是缓解主存容量不足的矛盾；使用主存是为了能容纳系统的核心软件和较多的用户程序及数据，因为指令和数据只有从外存调入内存才能被 CPU 访问。基于这种分级结构，就构成了一个满足应用需求的存储速度快、存储容量大、成本价格低的存储系统。

4.2.1 SRAM静态MOS存储器

目录

1. 基本的静态存储元阵列
2. 基本的SRAM逻辑结构。

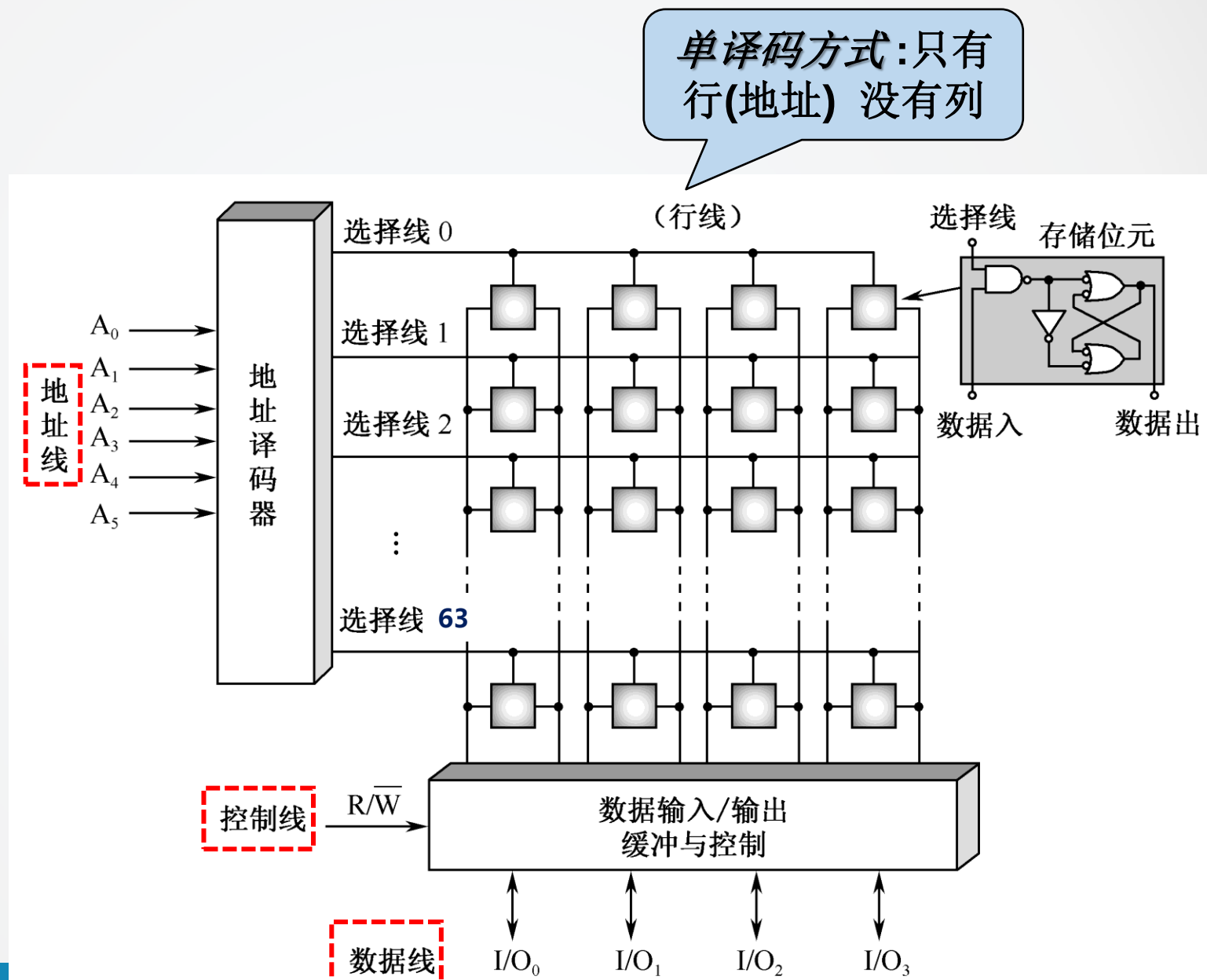
- **主存(内存)是半导体存储器。根据信息存储的机理不同可以分为两类：**
 - **静态读写存储器(SRAM): 存取速度很快**
 - **动态读写存储器(DRAM): 一般存储容量比SRAM大。**

基本的静态存储元阵列

1、存储位元

2、三组信号线

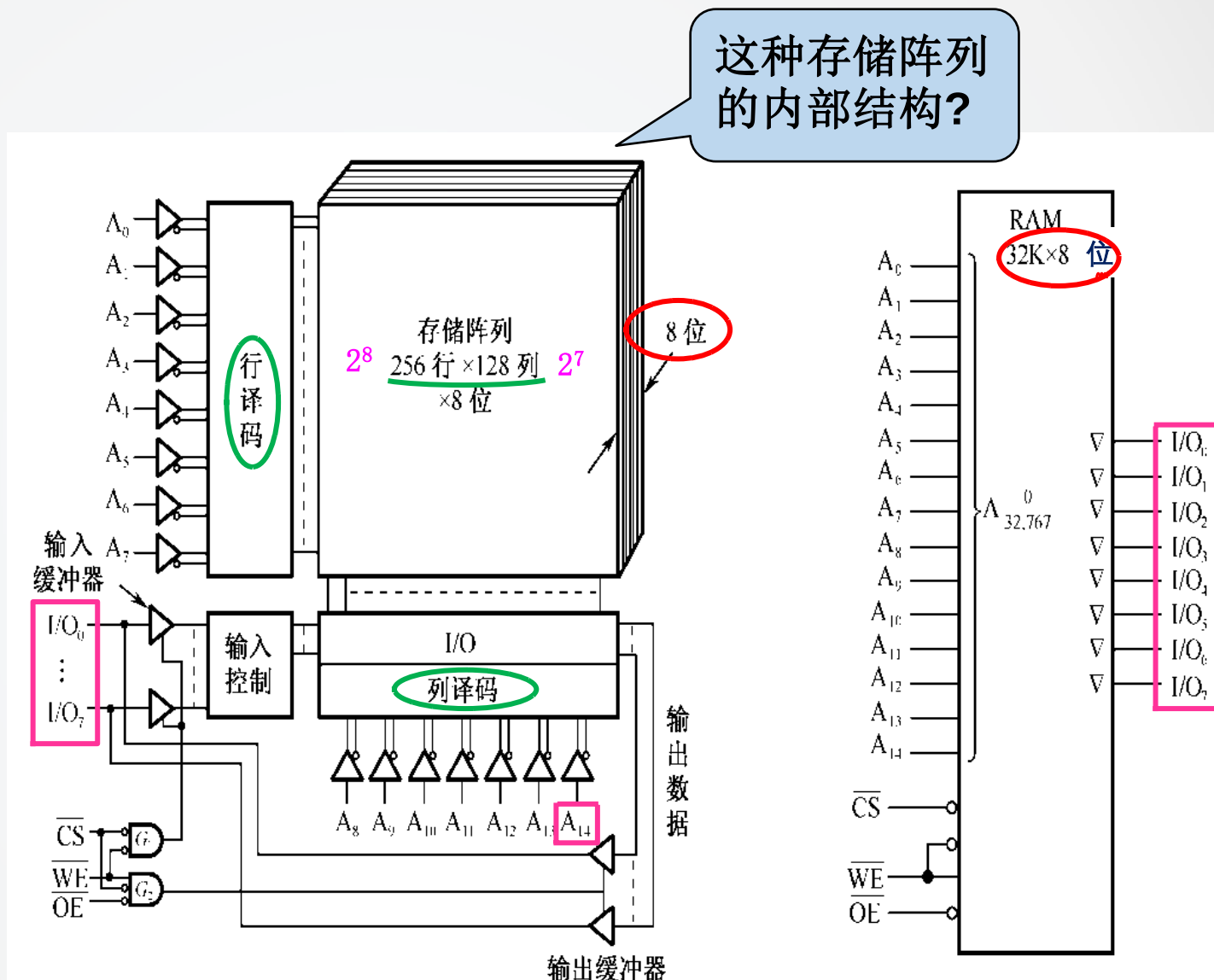
- 地址线
- 数据线
- 控制线。



基本的SRAM逻辑结构 (重+难)

P102

- 实际SRAM芯片大多采用**双译码方式**, 以便组织更大的存储容量。采用了二级译码: 将地址分成 ***x*向 (行译码)**、***y*向** 两部分如图所示。(a)结构图 (b)逻辑图。

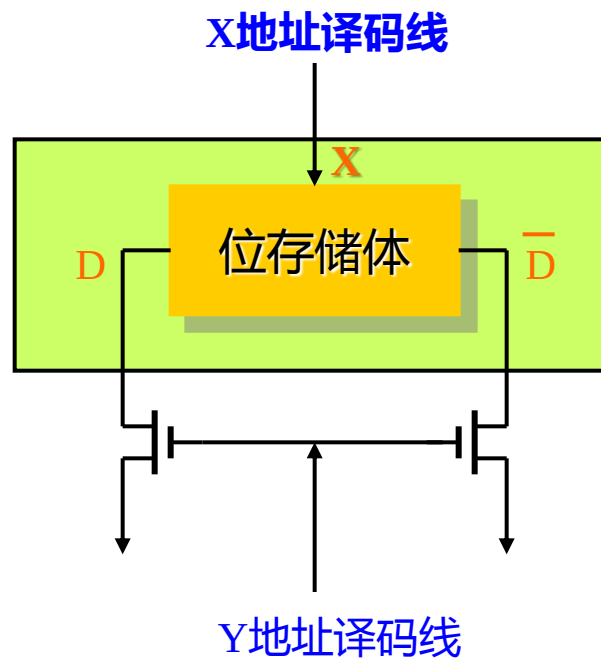


CS' 片选信号, 低电平有效; WE' 写使能信号; OE' 读出使能 (b)

基本的SRAM逻辑结构

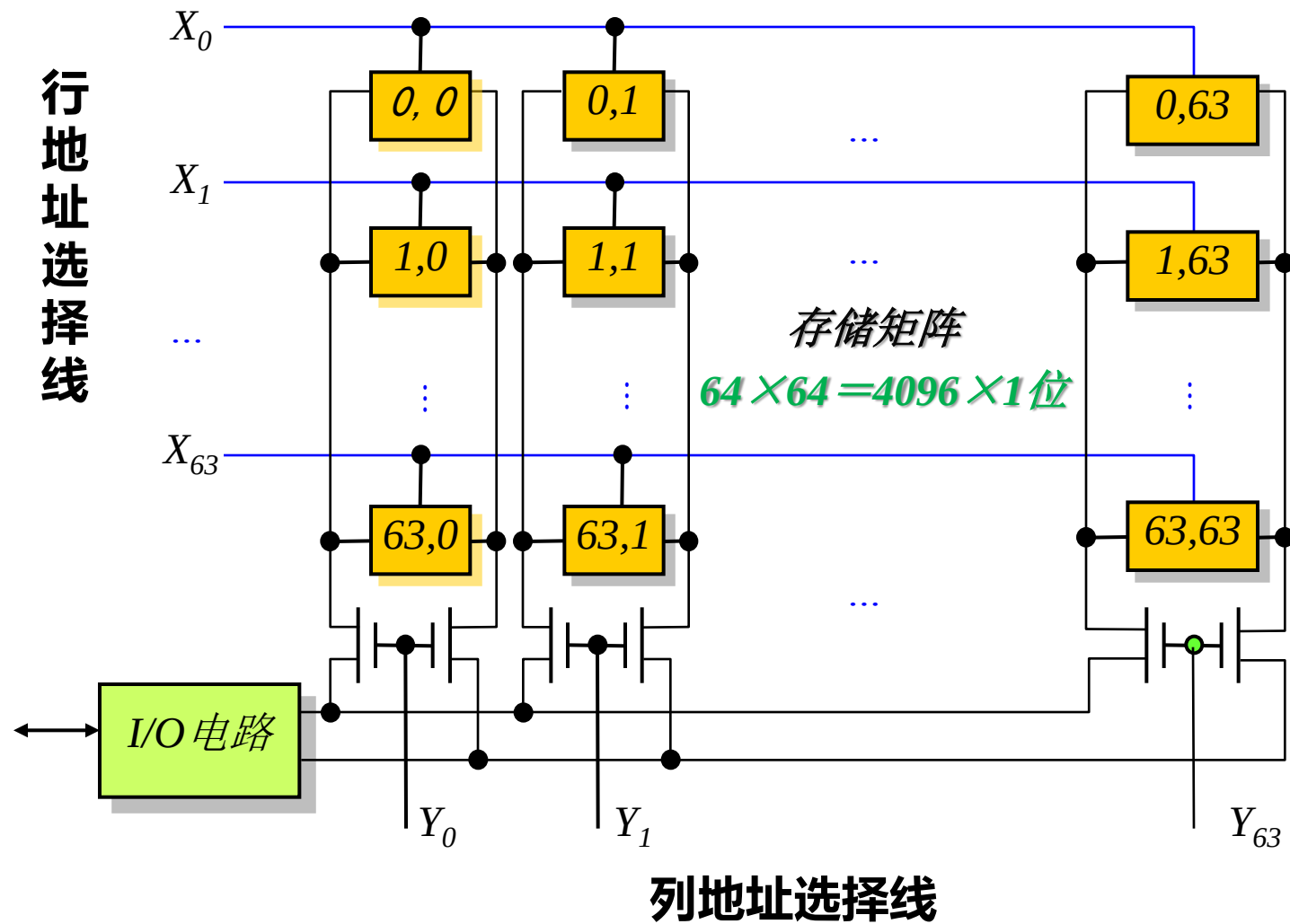
- 上页例：存储体（ $256\text{行} \times 128\text{列} \times 8\text{位}$ ）
 - 通常把各个字中，同一个字的同一比特位集成在一个芯片（ $32\text{K} \times 1\text{位}$ ）中（如上页图中的一层）， 32K 位排成 256×128 的矩阵。这样的8层就可构成 32KB 数据空间。
- 使用地址译码器：
 - 采用双译码方式
 - $A0 \sim A7$ 为行地址译码线
 - $A8 \sim A14$ 为列地址译码线。

双译码方式下的 1位存储体(存储位元)

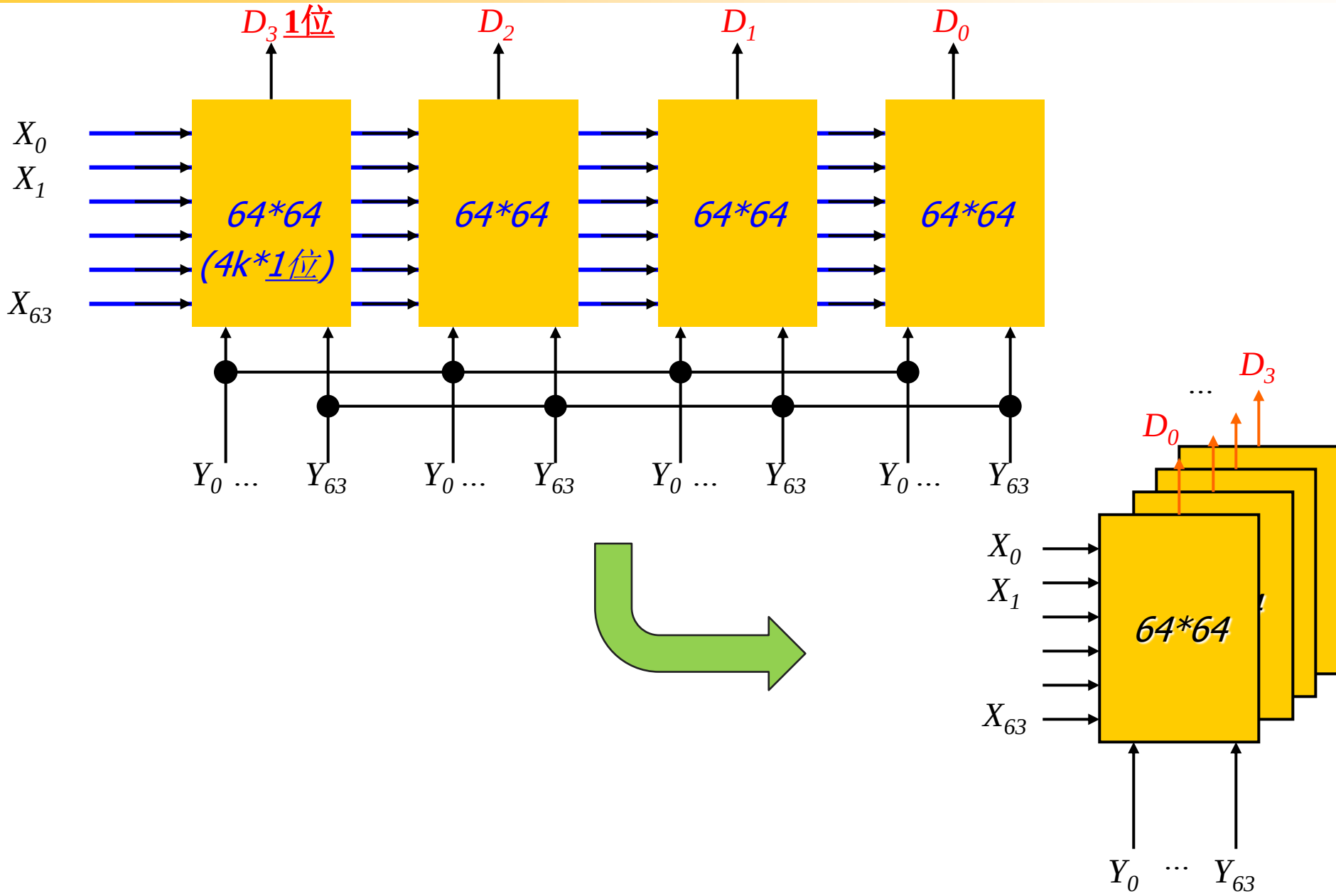


- 输入：X——行选择线
- Y——列选择线
- 输出：D——数据输出口
- 行、列选择线选中 位存储体, 方能读写数据。

64行x64列 存储矩阵

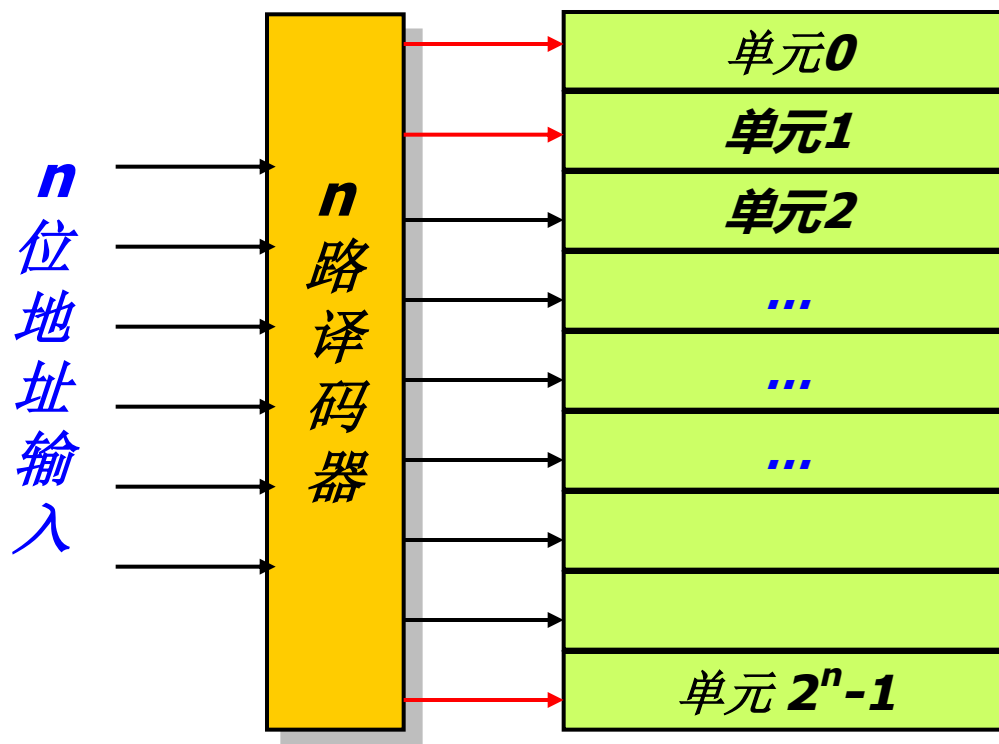


组成 4k*4位 存储体



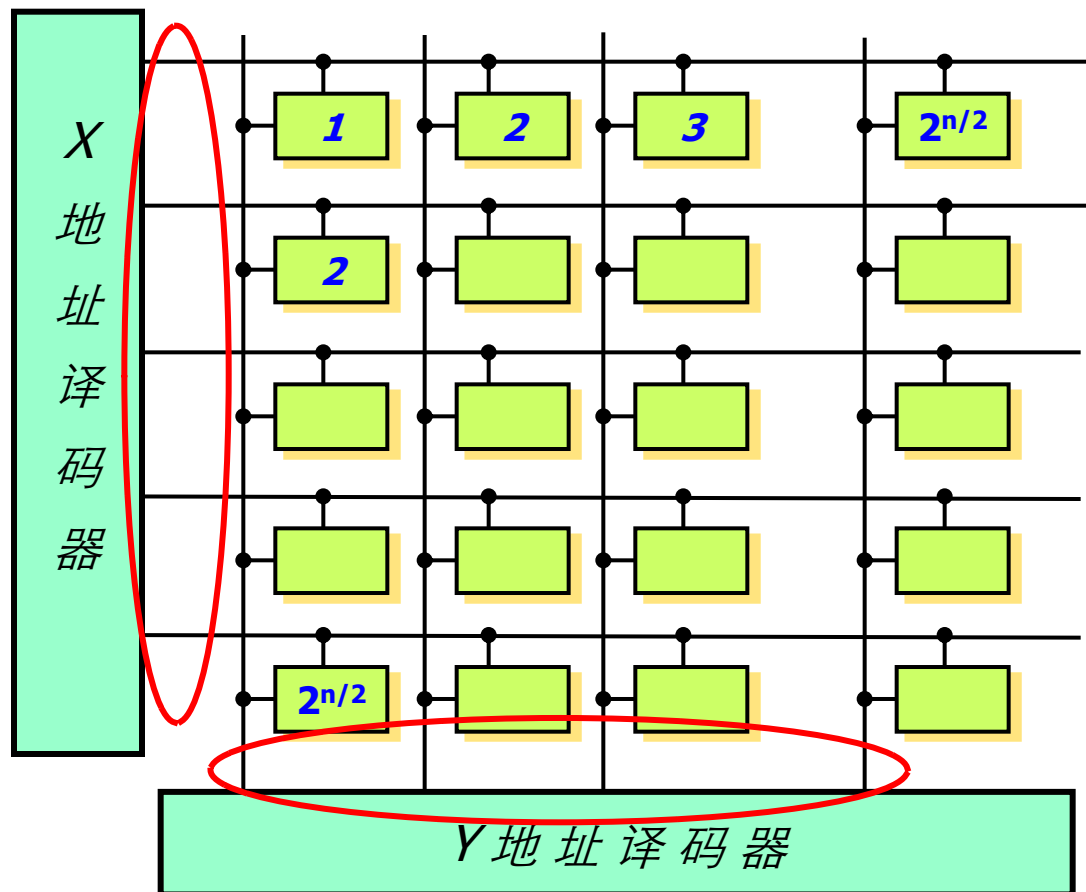
比较2种译码方式：

(1) 单译码方式



n 位地址，寻址 2^n 个存储单元，需 2^n 根译码线。

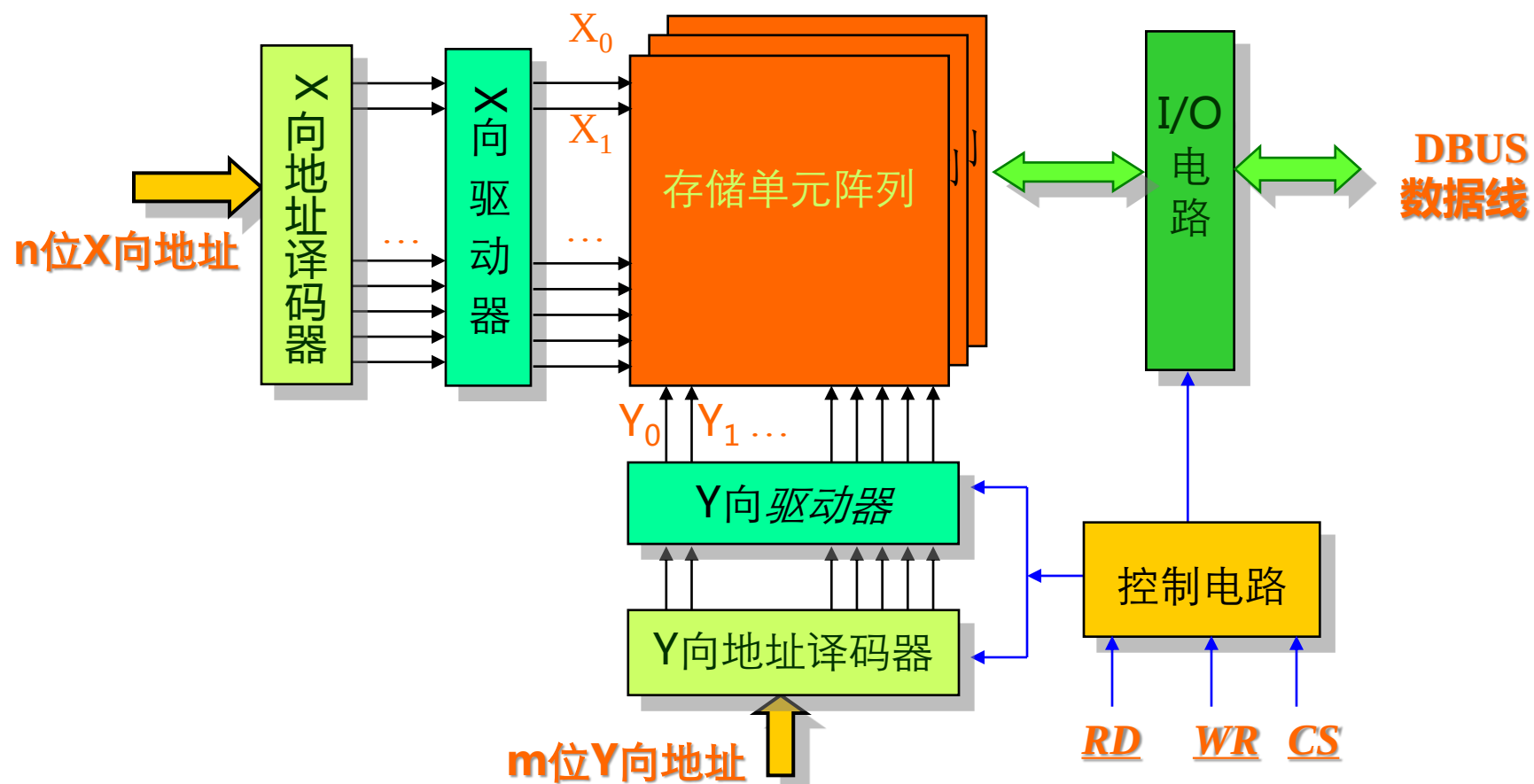
(2) 双译码方式



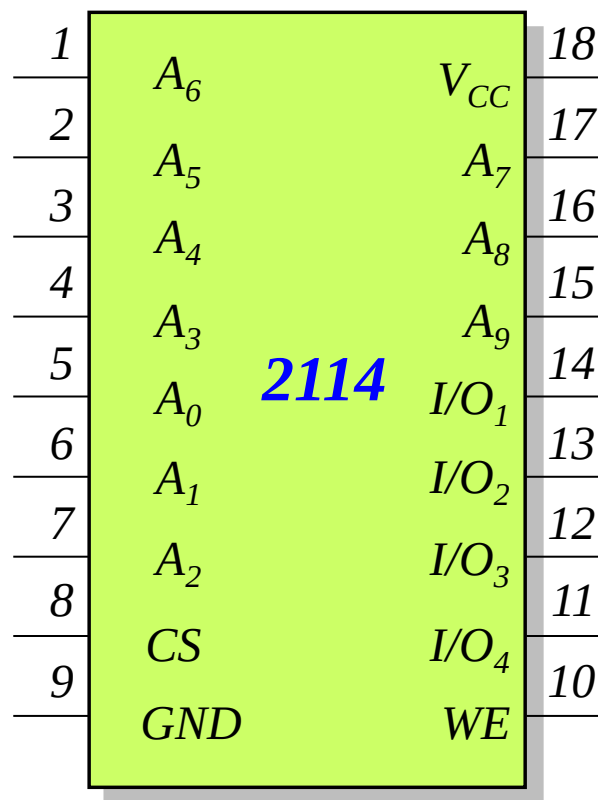
n 位地址，
待寻址 2^n 个存储单元，
只需 $2 * 2^{n/2}$ 根译码线，
对译码器件的要求更低
成本低，且行、列同时
译码速度更快。



静态存储器芯片结构

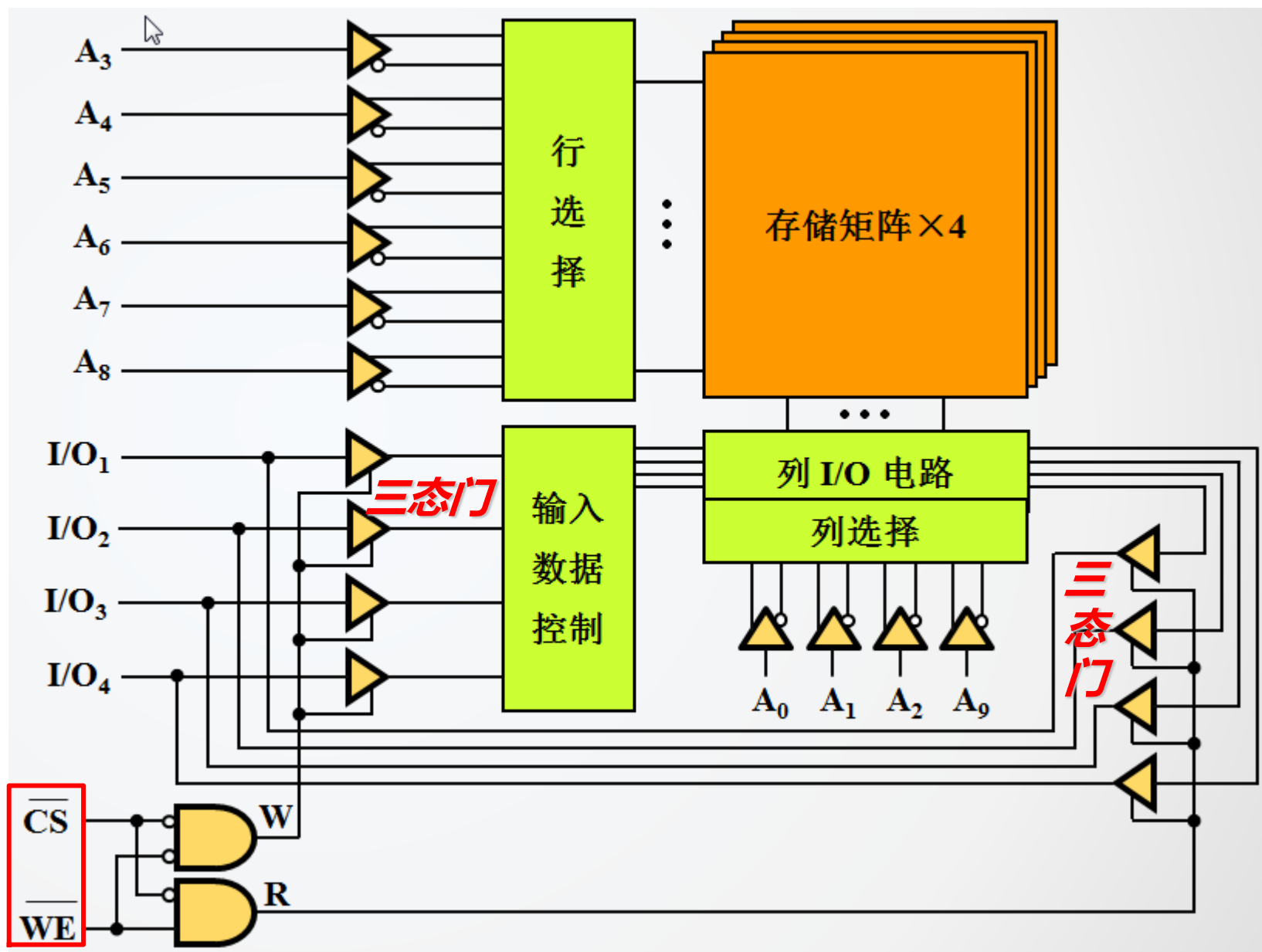


Intel 2114 引脚图



- 地址线 $A_9 \cdots A_0$
- 数据线 $I/O_4 \cdots I/O_1$
- 读写控制线 WE
- 片选线 CS
- 电源线 V_{CC}
- 地线 GND .

- 思考: 2114的存储容量 ?
- 2114的内部结构图如下页所示。

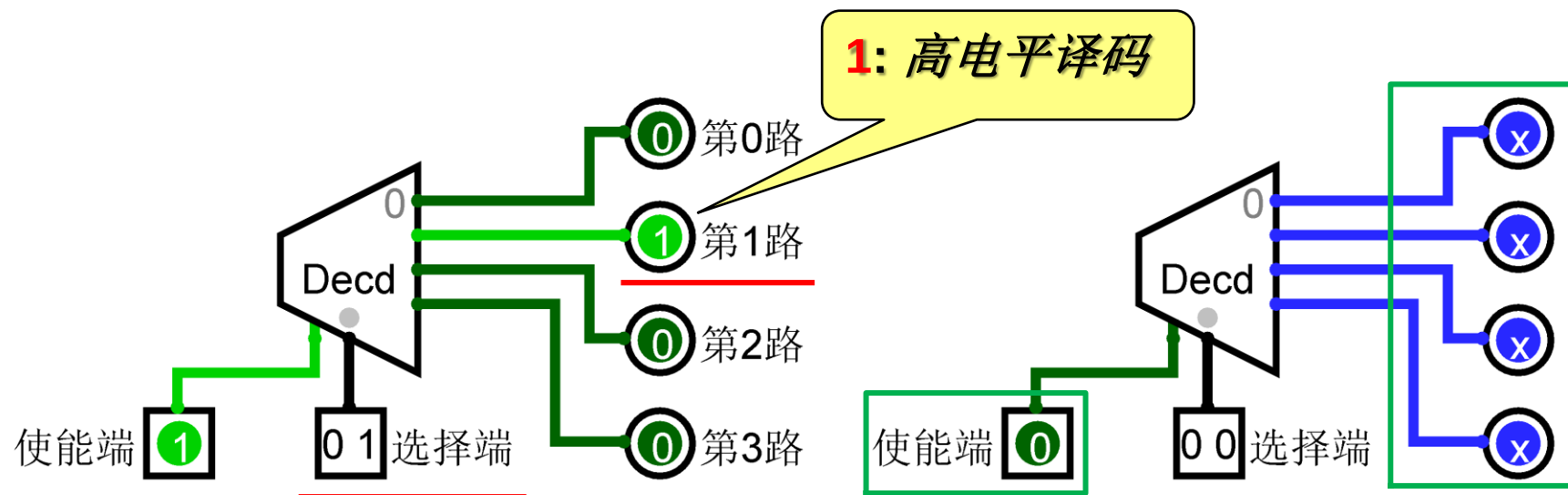


地址译码器 (可用于多个存储芯片中的片选)

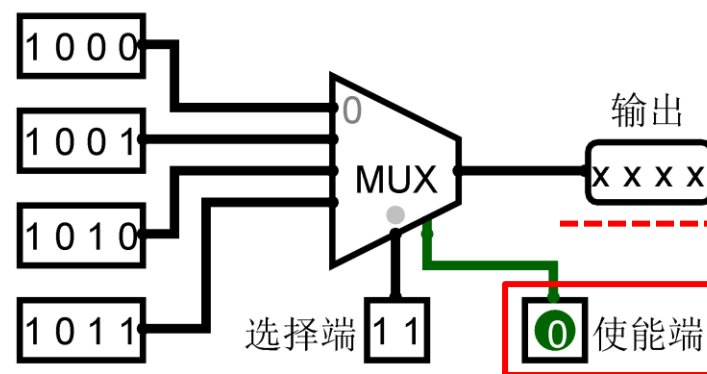
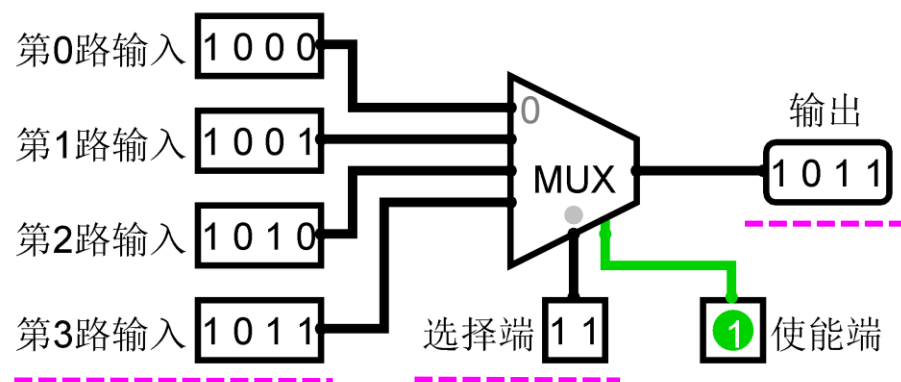


译码器 (logisim的 2:4 译码器)

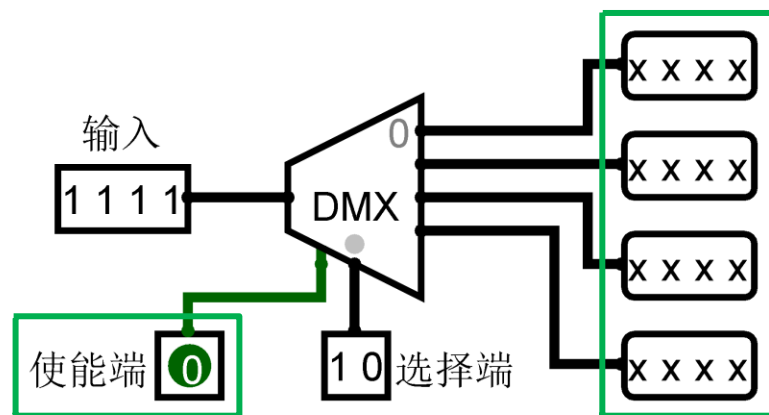
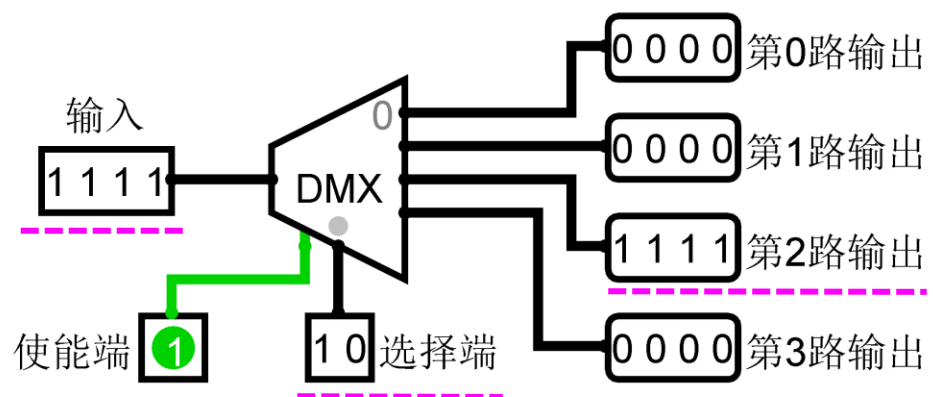
重



多路选择器MUX



Logisim 解复用器 (数据分配器)



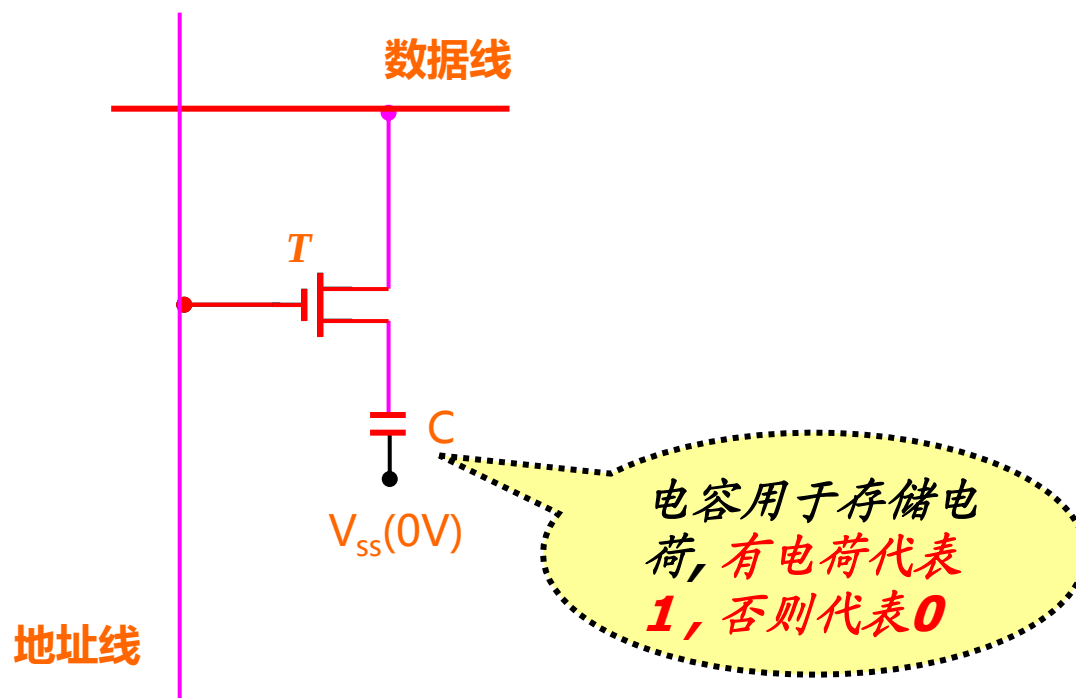
4.2.2 DRAM动态MOS存储器(※)

目录:

1. DRAM存储位元的记忆原理
2. 读/写周期、刷新周期
3. DRAM的刷新方式。

单管DRAM存储器的记忆原理

SRAM存储器的存储位元是一个触发器(锁存器)，它具有两个稳定的状态(0和1)。而**DRAM**存储器的存储位元是由MOS晶体管(若单管: 1个)和电容器组成的记忆电路。



- **读(写)周期**: 指连续两个读(写)周期的时间间隔。通常为控制方便, 读周期和写周期时间相等.
- 注意: 读操作和写操作是互斥的, 不会同时发生。

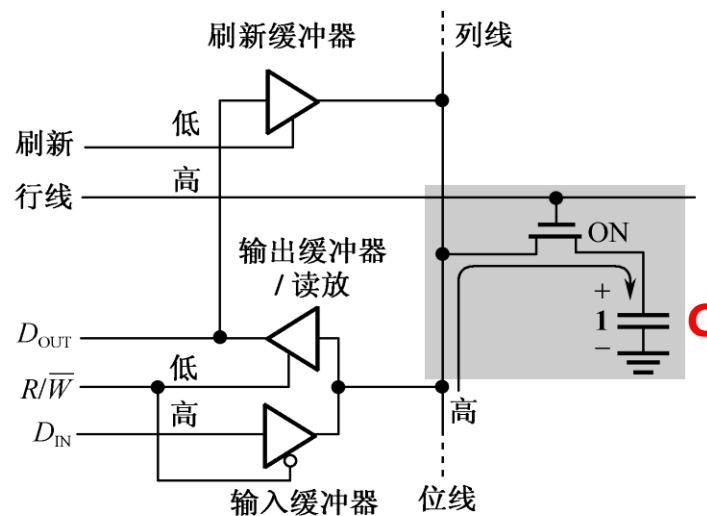
DRAM中 刷新的概念

- DRAM靠电容电荷存储信息，电荷易泄漏，需定期补充以保持信息不变，补充电荷的过程称为**刷新**。
- 电荷存储到泄漏完毕之间必须完成刷新。**刷新过程** 实质上是先将原存信息**读出**，再由刷新放大器形成原信息 并**重新写入**的再生过程。
- 从上一次对存储器刷新结束 到下一次对整个存储器刷新结束所经历的时间称为**刷新周期**。
- DRAM存储体采用双译码结构, *刷新按行进行, 每次刷新1行。*

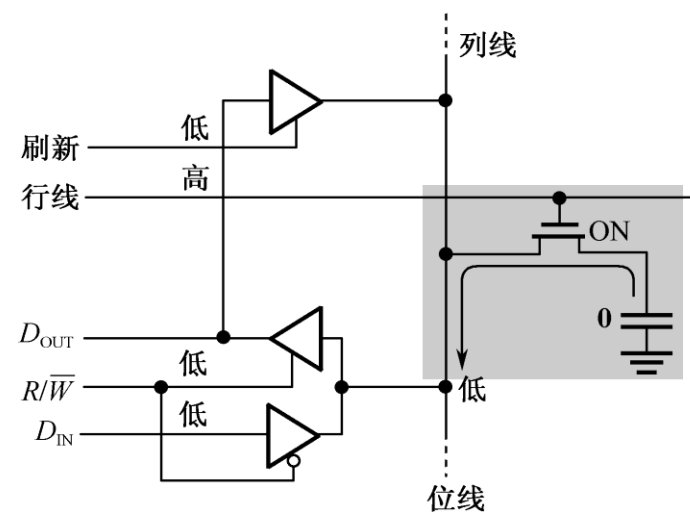
单管DRAM存储器的记忆原理



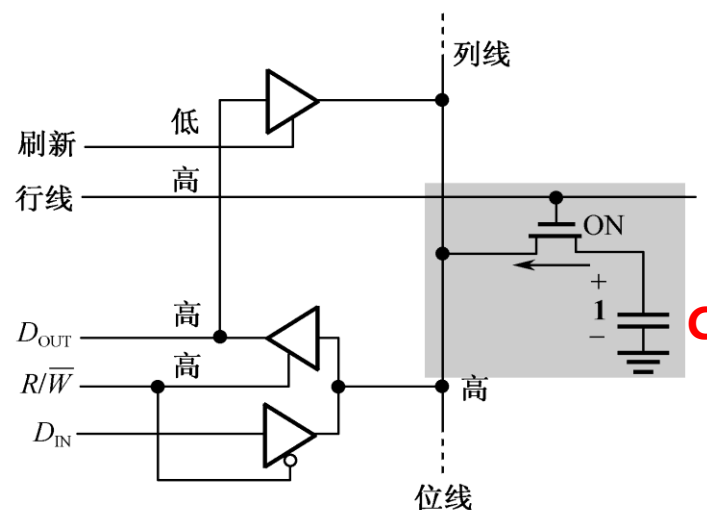
- 若原存储信息为1,则电容C将通过MOS管向位线放电,使位线电位上升到高电位1,送到Dout 读出1.
- 该读操作后,电容C上的电荷将丢失,属破坏性读出,需接着读后**重写—刷新**,见图(d)。



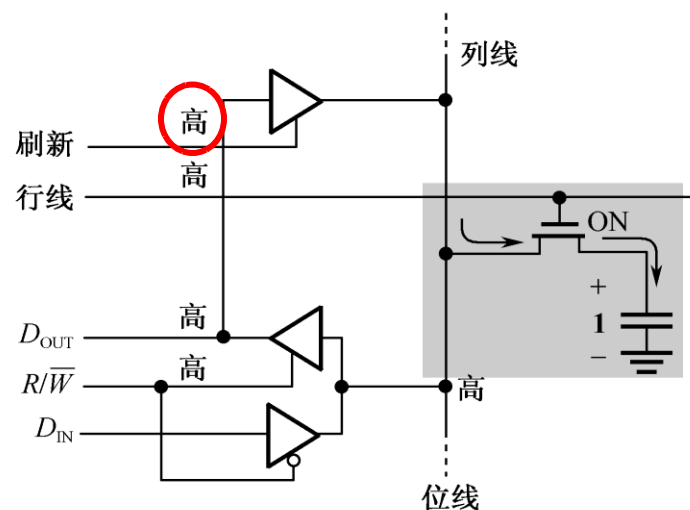
(a) 写 1 到存储位元



(b) 写 0 到存储位元



(c) 从存储位元读出 1



(d) 刷新存储位元的 1

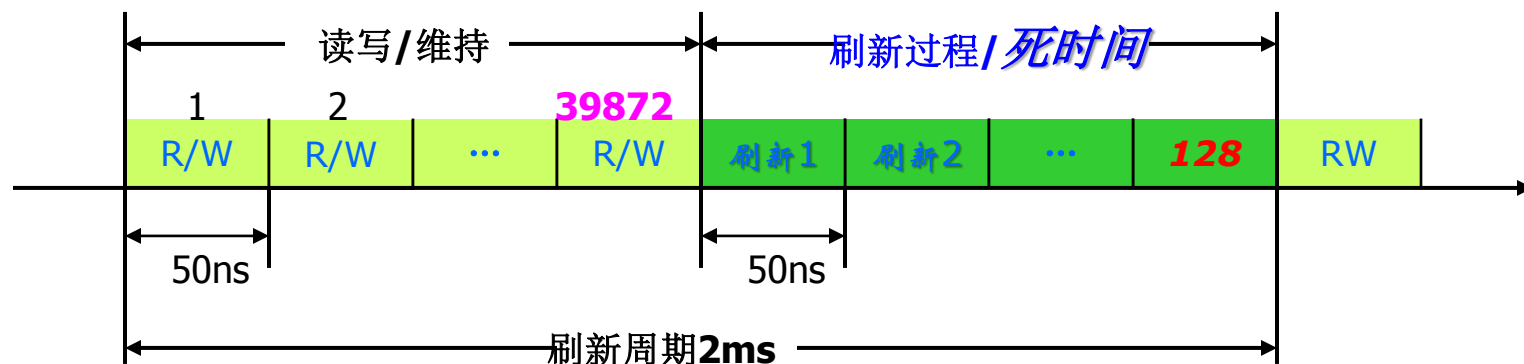
DRAM的刷新方式 (✕ P108)

- 1. 集中式
- 2. 分散式
- 3. 异步式。

集中刷新方式

- 为了便于比较3种不同刷新方式, 设DRAM存储体均为128行*128列 的结构, 读/写周期 $t_c=50$ 纳秒ns, 刷新周期为2毫秒ms

集中刷新, 即2ms内集中安排所有行的刷新, 一般在刷新周期的后段时间里集中做刷新。



采用集中刷新的存储器平均读写周期

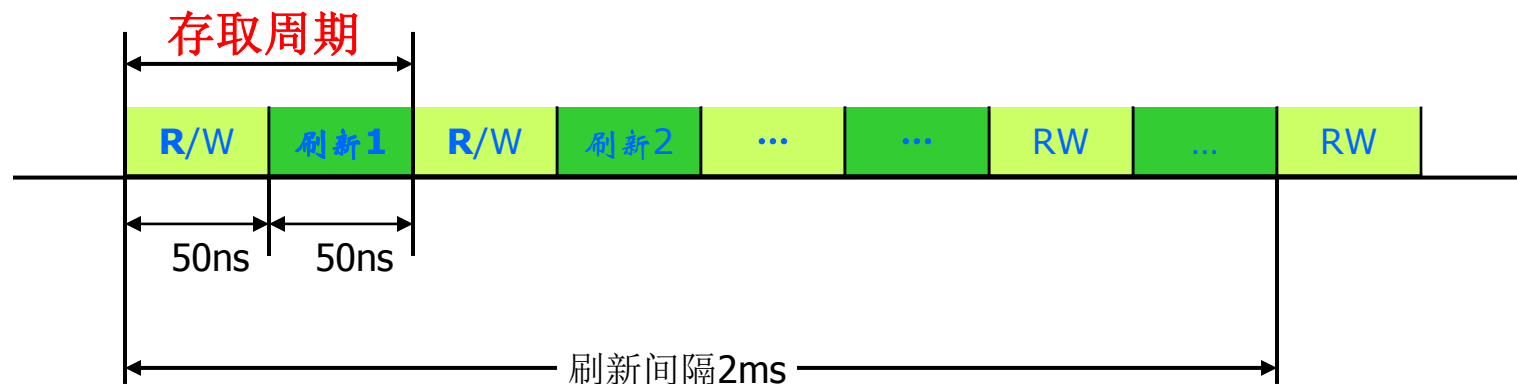
$$T_{\text{平均}} = \frac{2ms}{\frac{2ms}{50ns} - 128} = 50.161ns \approx t_c$$

此方式存在 死时间 (刷新过程的128个读写周期内 CPU 不能访存), 因此只用在实时要求不高的场合。

分散刷新方式

是各行的1次刷新 分散安排在每个存取周期中。

此实例中 $2\text{ms}/100\text{ns}=20000$ 次, $20000\text{次}/128\text{次(行)} \approx 155$ 遍
154遍不必要的刷新!



平均
读写周期: $T_{\text{平均}} = 100\text{ns} > t_c$

用在低速系统中。

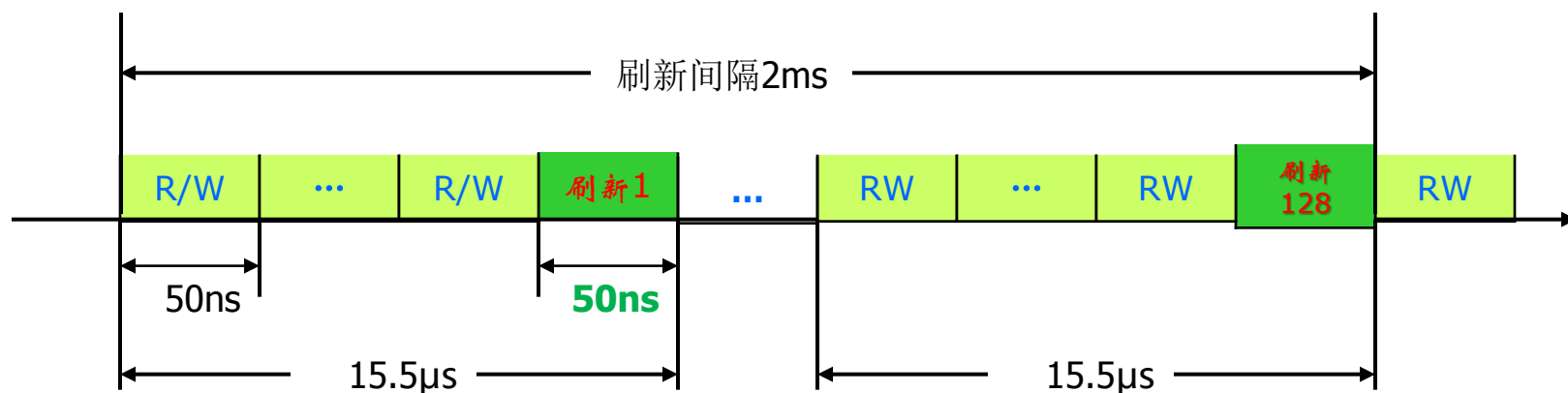


异步刷新方式

即各行的刷新 分散安排在整个刷新周期内 .

每隔一段时间刷新一行; ... $\frac{2ms}{128行} \approx 15.5 \text{ 微秒}\mu s$

即每隔15.5 μs 提1次刷新请求, 刷新1行; 2ms内刷新完所有128行.



平均
读写周期: $T_{平均} = \frac{2ms}{\frac{2ms}{50ns} - 128} = 50.161ns \approx tc$ $T_{平均}$ 与集中刷新方式的相同

这里死时间很短, 若将每次(异步)刷新安排在 CPU对指令的译码阶段(此时无需访存), 就无访存死区问题了; 因此它用于大多数计算机中。

- **DRAM发展历史**：最早Intel公司起家产品是DRAM存储器，后来才发展到CPU产业。而半导体存储产业发展经历了美国领先到日本领先，美日贸易摩擦广场协议后韩国迅速崛起，称霸全球。
- **中国半导体存储产业正在逐渐崛起，比如武汉“长江存储”（Flash闪存）！但中美贸易摩擦加剧背景下困难重重。**
- **同学们应提升民族自豪感和时代责任担当！**

4.3 主存的组织及与CPU的连接 (重+难)

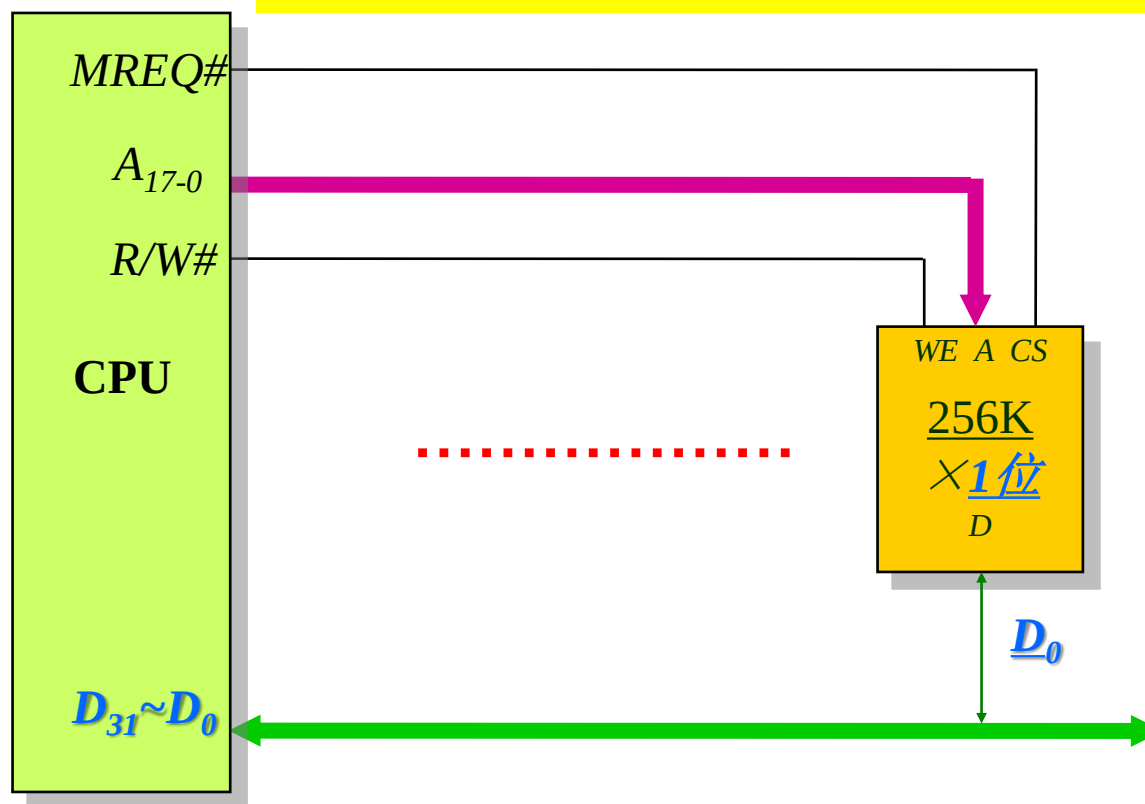
- 1. **地址线**的连接
- 2. **数据线**的连接
- 3. **控制(信号)线**的连接
- 4. 存储器的扩展。

主存储器与CPU的连接

- 地址线的连接
- 数据线的连接
- 控制信号线的连接
- 存储器扩展。

由256K*1位的芯片组成的256K*32位的存储体，三类信号线如何连接？

- 数据线 $D_{31} \sim D_0$
- 地址线 $A_{17} \sim A_0$
- 读写控制线 WE
- 片选线CS(32片)



存储器容量扩展

- 1、字长扩展 (数据总线的扩展)
 - 多个芯片同时并行工作
- 2、字数扩展 (地址总线的扩展)
 - 同一时刻仅一个芯片工作
- 3、字长、字数 综合扩展。

存储器容量扩展

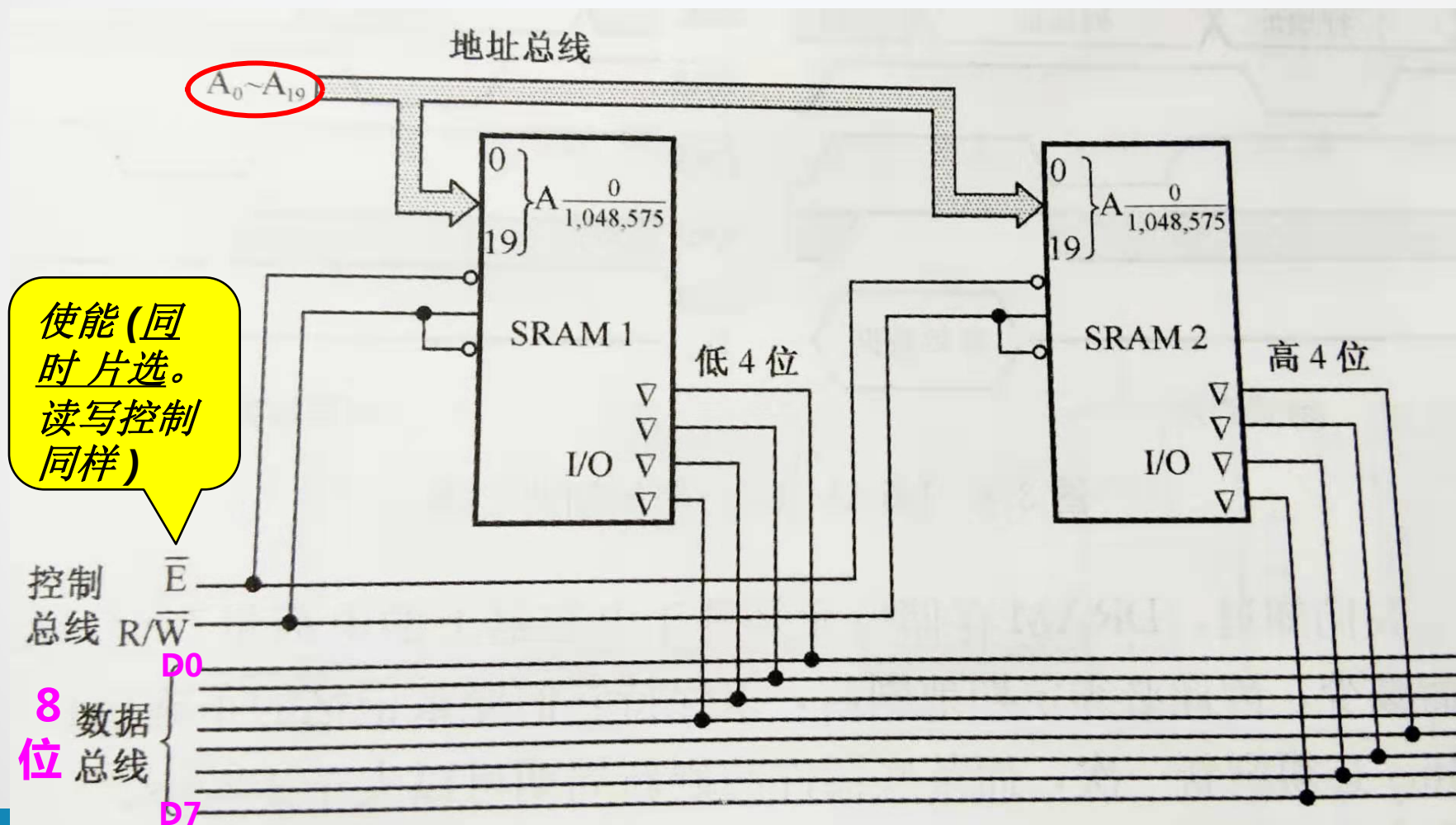
1、字长位数扩展（位扩展）

- 若给定的芯片字长位数较短，不满足设计要求的存储器字长，此时需要用多片给定芯片扩展字长位数。
- 此时三组信号线中，多片芯片上 **地址线** 和 **控制线** 公用，而 **数据线** 单独分开连接。
- 所需芯片数量：
$$d = \text{设计要求的存储器总容量} / \text{选择芯片之单个存储芯片容量}。$$

【例1】 利用 $1\text{M} \times 4$ 位的SRAM芯片, 设计一个存储容量为 $1\text{M} \times 8$ 位的SRAM存储器。

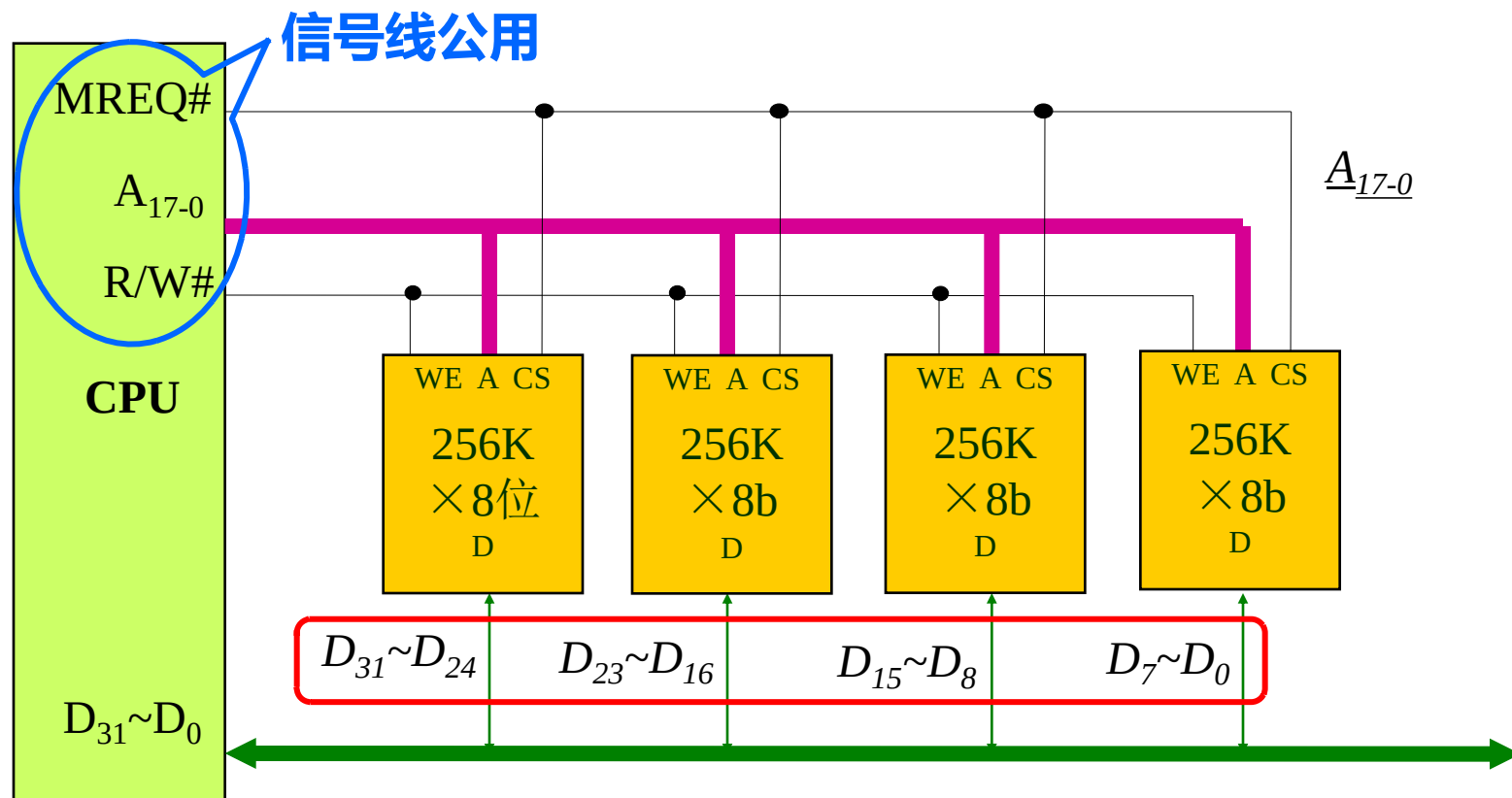
解：所需芯片数量 $= (1\text{M} \times 8\text{位}) / (1\text{M} \times 4\text{位}) = 2$ 片

- 设计的存储器字长为8位，存储单元数不变。连接的三组信号线：
地址线、控制线公用，数据线分高4位、低4位。如图所示：



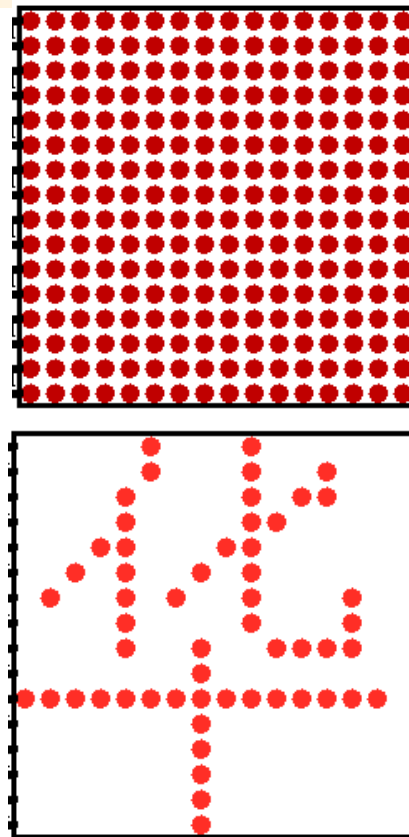
位扩展(DBUS) 例2：

用256K×8位的芯片，设计256K×32位的存储器：



实际例子: 16*16点阵字库设计(实验)

- 汉字字库：用于存放汉字字形码
 - 输入码: 区位码
 - 输出码: 对应汉字的 字形码(字模码)
- 16*16点阵需256 b才能显示一个汉字
 - 字库中存储单元 位宽 应该为: 256位
 - 但Logisim的ROM数据位宽最多:32位 , 怎么办?
 - 位扩展 :
 - ◆ 需8个 16K*32位 的ROM存储器
 - ◆ 256位的字形码 最终输出到8个32位ROM的输出引脚。



作业 1

- 本子上完成：
- 1. 教材P147习题4.1: 存取时间, 存取周期, 存储器带宽, 大端存储, 小端存储
- 2. 教材P149习题4.3 (1)
- 3. P150习题4.10 (设16K*1位芯片为128行*128列; 问题改成“ 若每块芯片的刷新间隔.....”)
- 4. 试题书3.2分析题: 3(**书上答案有错**)。

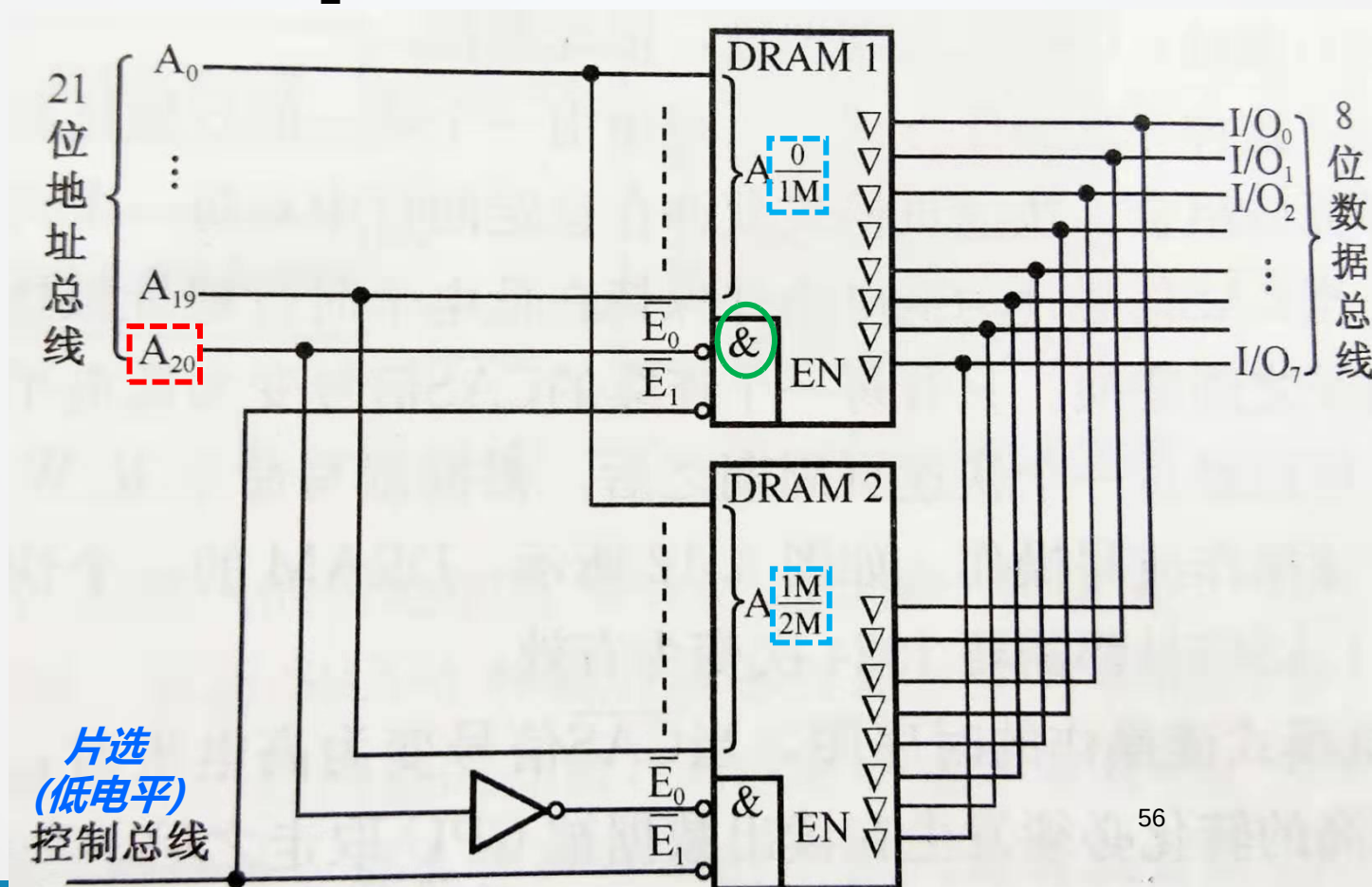
2、字(数) 扩展

- 若给定芯片中存储单元数目较小(字数少), 不满足设计要求的总存储容量, 此时需要用多片给定芯片来扩展字数.
- 此时三组信号线中, 给定芯片上 地址总线的低位段 和 **数据总线都是公用的**, 控制总线中 R/W公用, **使能端EN 不能公用 (因为不能同时片选多片)**, 由地址总线的高位段译码 来决定片选信号。
- 所需芯片数: $d = \text{设计要求的存储器总容量} / \text{选择芯片的存储容量}$ 。

例1. 利用1M×8位的DRAM芯片设计2M×8位DRAM存储器.

解：所需芯片数 $d = (2M \times 8\text{位}) / (1M \times 8\text{位}) = 2\text{片}$

- 设计的存储器见下图所示。字长位数8b不变, 地址总线中低位段 $A_0 \sim A_{19}$ 同时(公用)连接到2片1M×8位芯片的地址输入端, 地址总线最高位 A_{20} 和控制总线的 E_1' 连入与门, 得到片选信号EN。



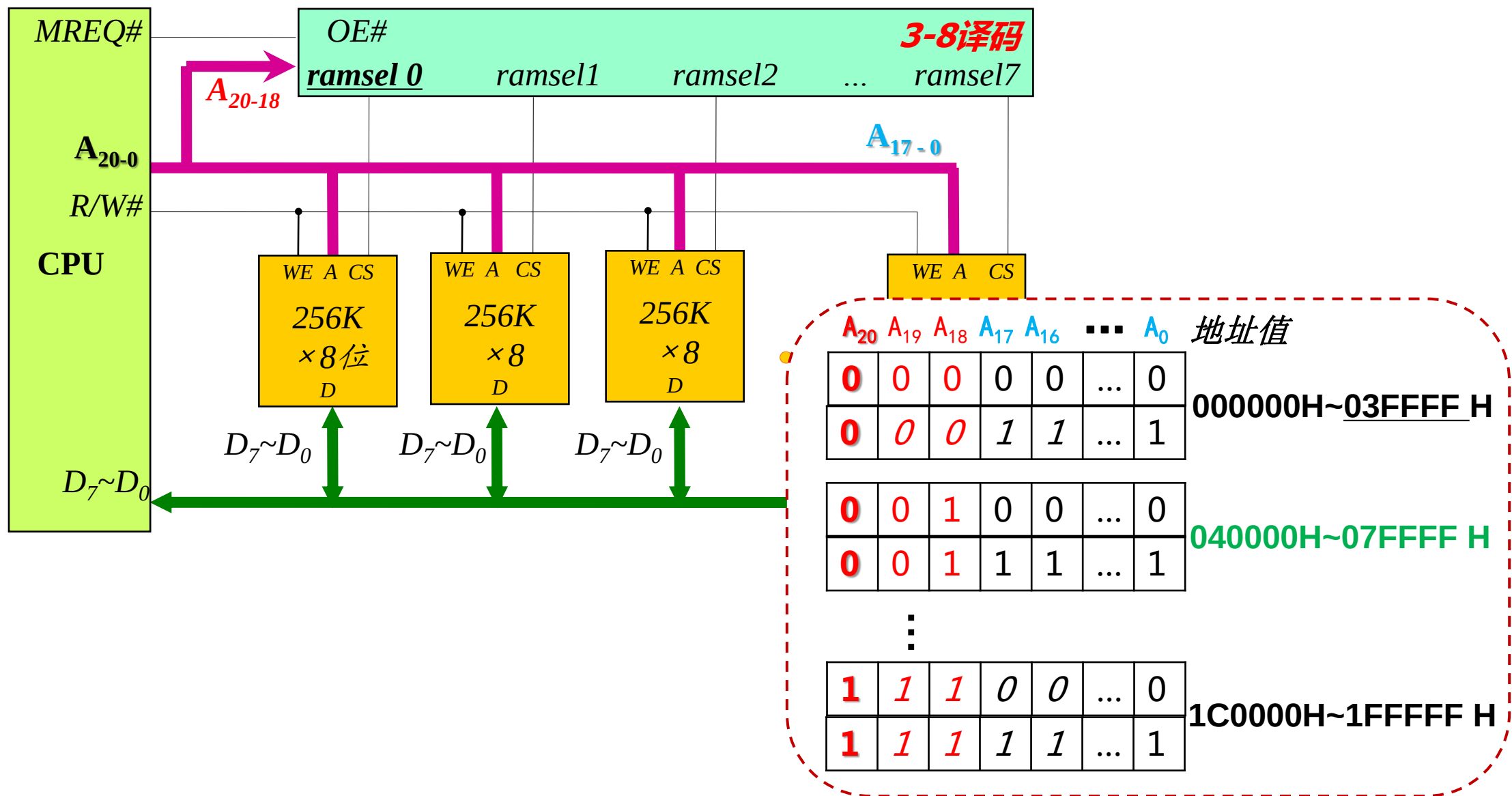


通知

- 今晚开始，信科的小伙伴们有实验课。注意，**18:45**开始上课（七八节下课后大家晚餐迅速一点），21:45下课。实验时带上理论课教材。

字扩展(ABUS) 例2：

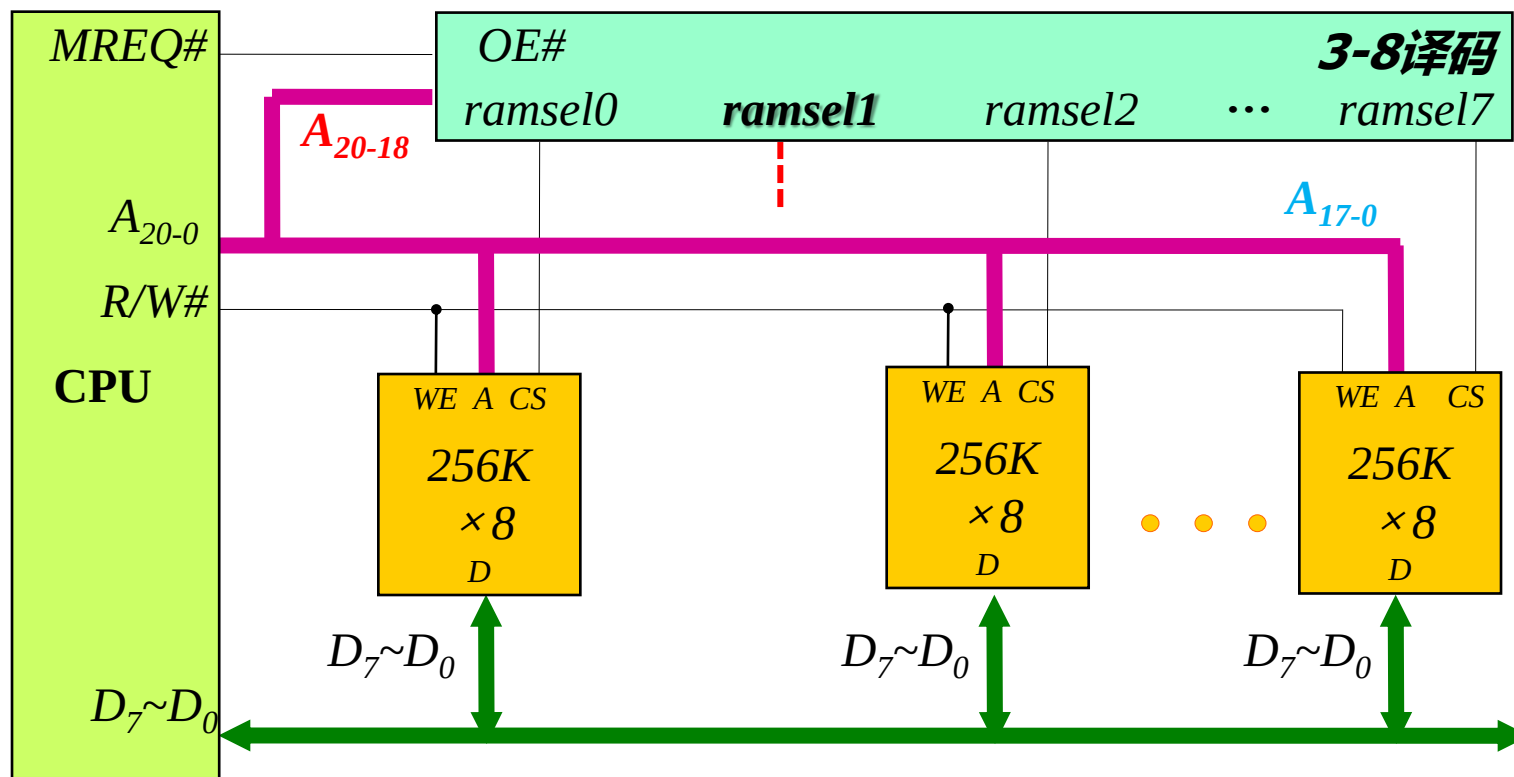
用256K×8位的芯片，设计 2M×8位的存储器。



字扩展(ABUS) 例3 :

用256K×8位芯片, 设计2M×8位的存储器, 其中地址
040000H~07FFFF H 的存储空间 保留不用。

- 解: $A_{20}A_{19}A_{18}A_{17}A_{16} \dots A_0$
 - 040000 H 即 **0 0 1** 0 0 ... 0 B
 - 07FFFF H 即 **0 0 1** 1 1 ... 1 B
- 第2片(*ramsel1*)的地址空间



3. 综合扩展 (字、位 同时扩展)

- 例1: 设有若干片 $256\text{K} \times 8$ 位的SRAM芯片, 问:
 - (1) 如何构成 $2\text{M} \times 32$ 位的存储器?
 - (2) 需要多少片RAM芯片?
 - (3) 该存储器需要多少地址位?
 - (4) 画出该存储器与CPU连接的结构图, 设CPU的接口信号有 地址信号、数据信号、控制信号 $\text{MREQ\#}$ 和 $\text{R/W\#}$ 。

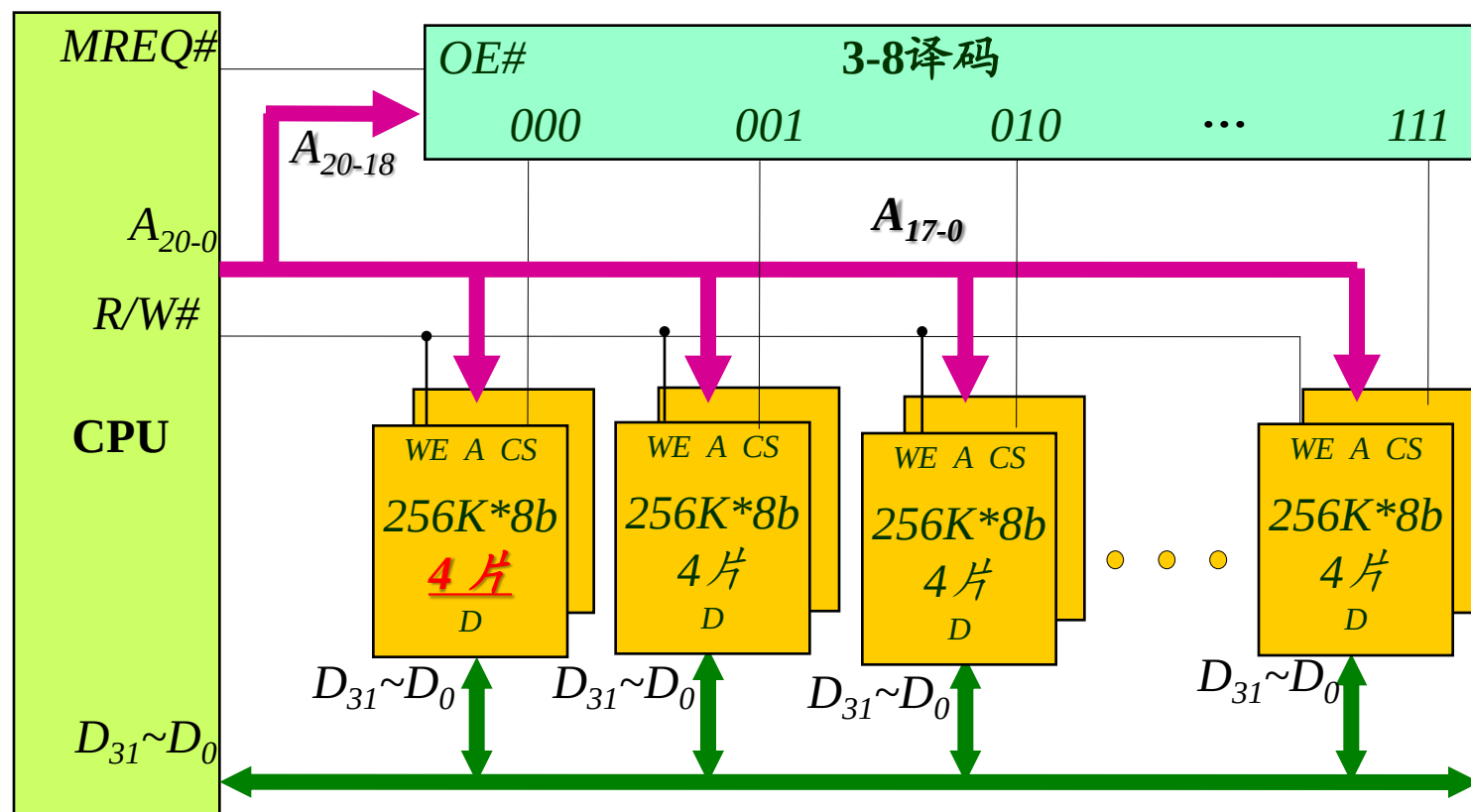
■ 解: (1) (2) 需芯片数 $d = (2\text{M} \times 32\text{位}) / (256\text{K} \times 8\text{位})$

■
$$= (2\text{M} / 256\text{K}) \times (32\text{位} / 8\text{位})$$

■
$$\begin{array}{ccc} \text{字扩展} & = 8(\text{组}) \times 4\text{片} = 32\text{片} & \text{位扩展} \end{array}$$

■ (3) $2\text{M} = 2^{21}$, 因此需要21位地址: A_{20-0} 。

(4) 连接图：

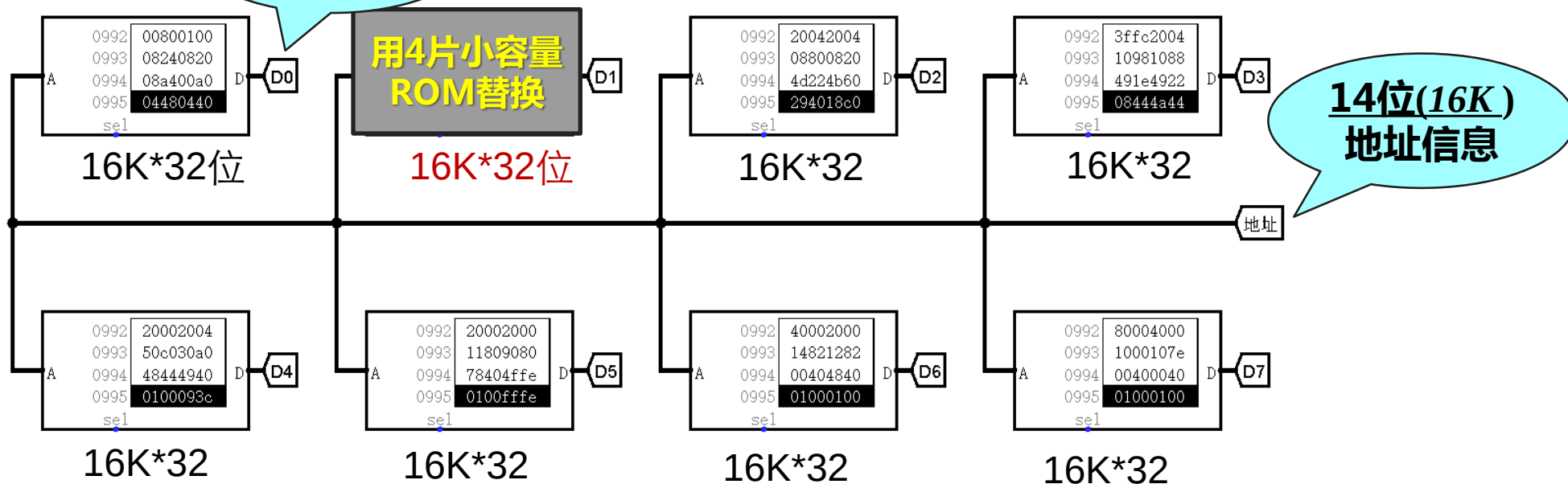


结论：一个存储系统容量为 $M \times N$ 位，若使用 $L \times k$ 位的芯片 ($L < M, k < N$)，共需要 $(M/L) \times (N/k)$ 个芯片。

(多组)
字扩展

(组内)
位扩展

例2. 字库设计 (回顾) 8个 $16K * 32$ 位的ROM存储器中：



■ 如何用 **4片 $4K * 32$ 位** 的ROM 替换阴影部分的 **$16K * 32$ 位** 器件？

□ 容量扩展: 地址总线扩展 (字扩展)。

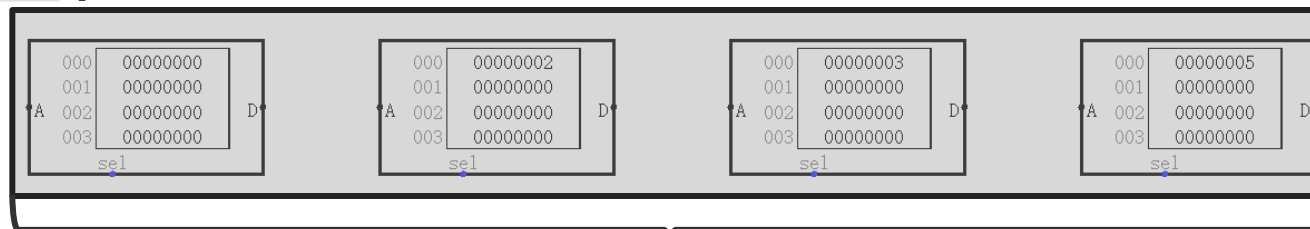
存储扩展实验（汉字字库）

第0片
4K (*32位)

第1片
4K

第2片
4K

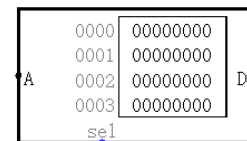
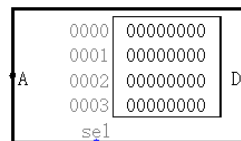
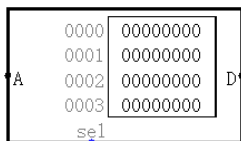
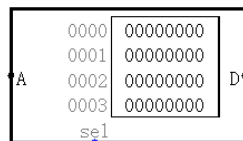
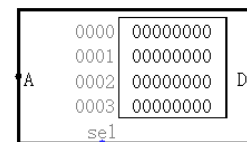
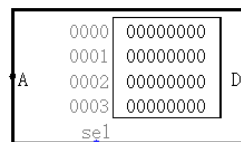
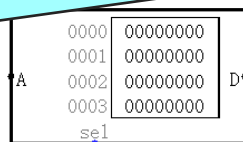
第3片
4K



提示: 14
位地址中
低12位这
4片公用,
高2位用于
2:4译码
(器)。

?

用4片小容
量ROM替换



综合扩展--各芯片地址范围

例3：某计算机的主存地址空间中

0x0000到0x3FFF为ROM(只读不写)存储区域

0x4000到0x5FFF为保留(不用)地址区域

0x6000到0xFFFF为RAM地址区域。

RAM的控制信号为CS#和WE#，CPU地址线A₁₅—A₀，数据线D₇—D₀，控制信号有读写控制R/W#和访存请求MREQ#。

若ROM和RAM都采用8K×1位的芯片，试画出与CPU的连接图。

0x0000 ~ 0x3FFF 即16K(×8位) ROM = 连续 2 组 8K×8位芯片组

地址编号:
地址线A₁₅-A₀

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1

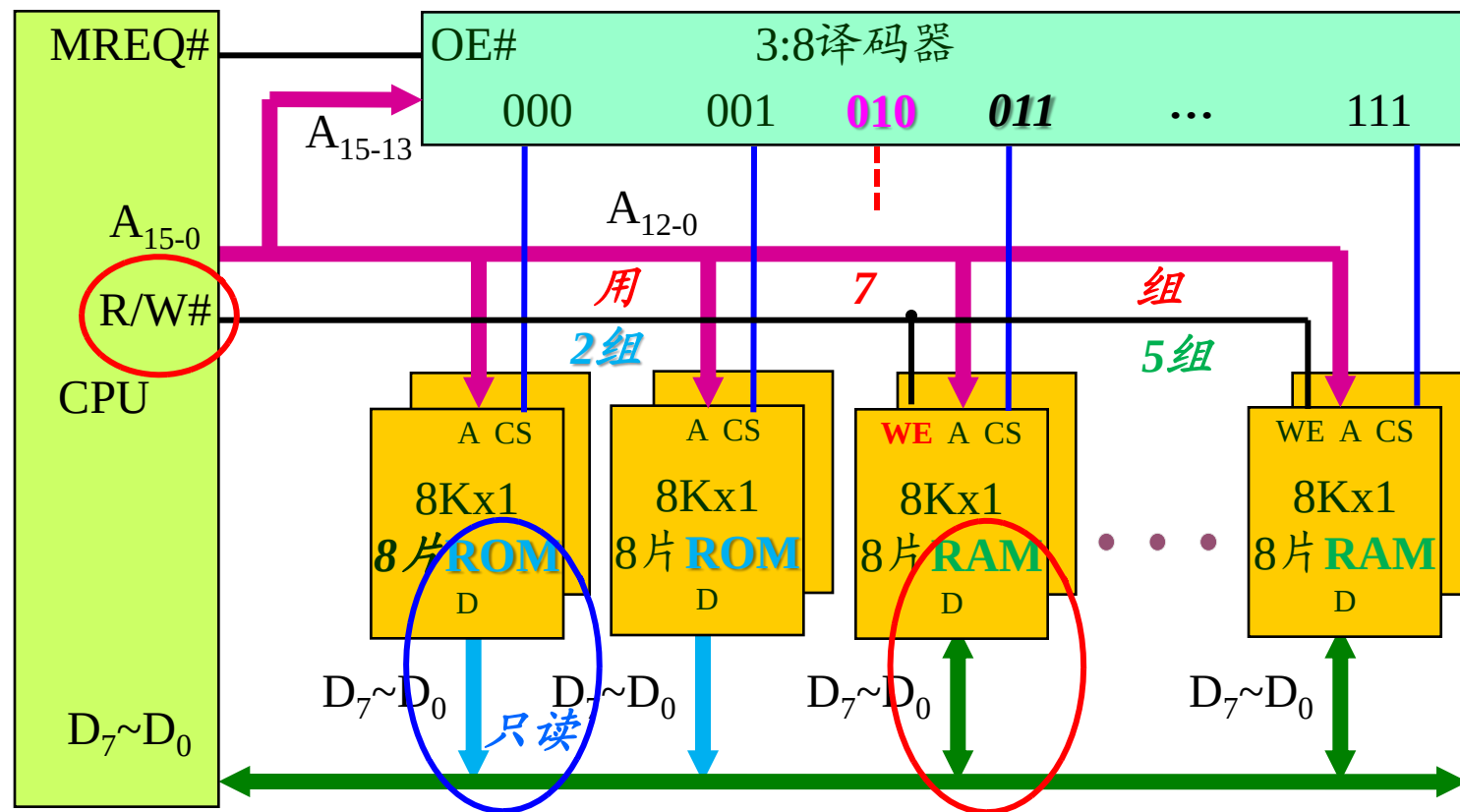
0x4000~0x5FFF 8K×8位 RESERVED = 1组

0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1

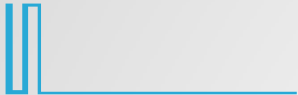
0x6000 ~ 0xFFFF 40K×8位 RAM = 连续 5组 8K×8位芯片

0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

ROM和RAM采用8K×1位的芯片：



- 在试题书上完成P30: 3.1选择题 1,2,3,5,6,8,9, 13,14 .
- 以下在本子上完成 :
- 1. 试题书3.3设计题: 3 (“CPU地址总线为A14 --A0” ,
图3.16(a)有错) ; 4 (**去掉P51第3行末尾 “按字节编址”** ;
图3.18有错)
- 2. 教材P149习题4.6。



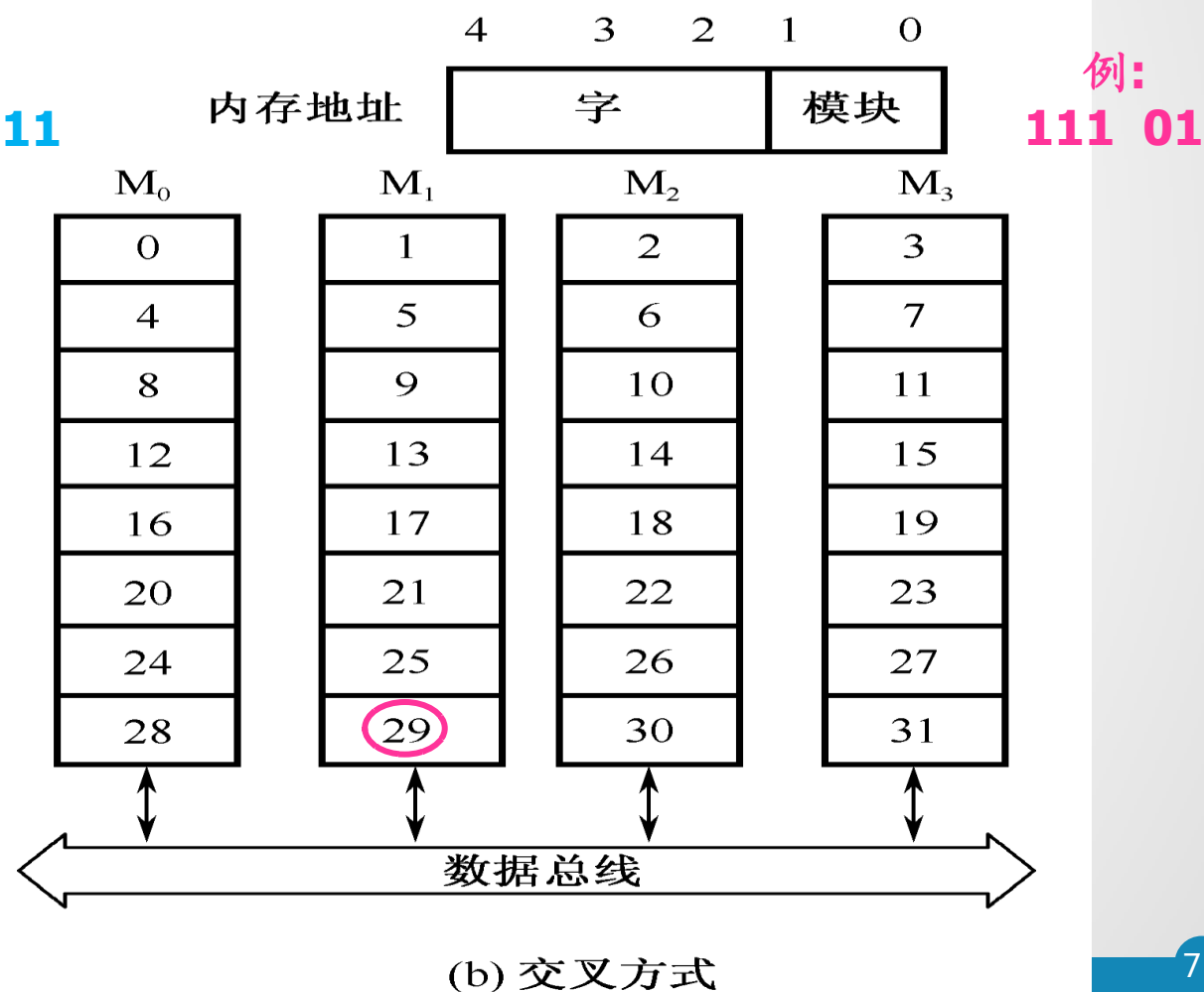
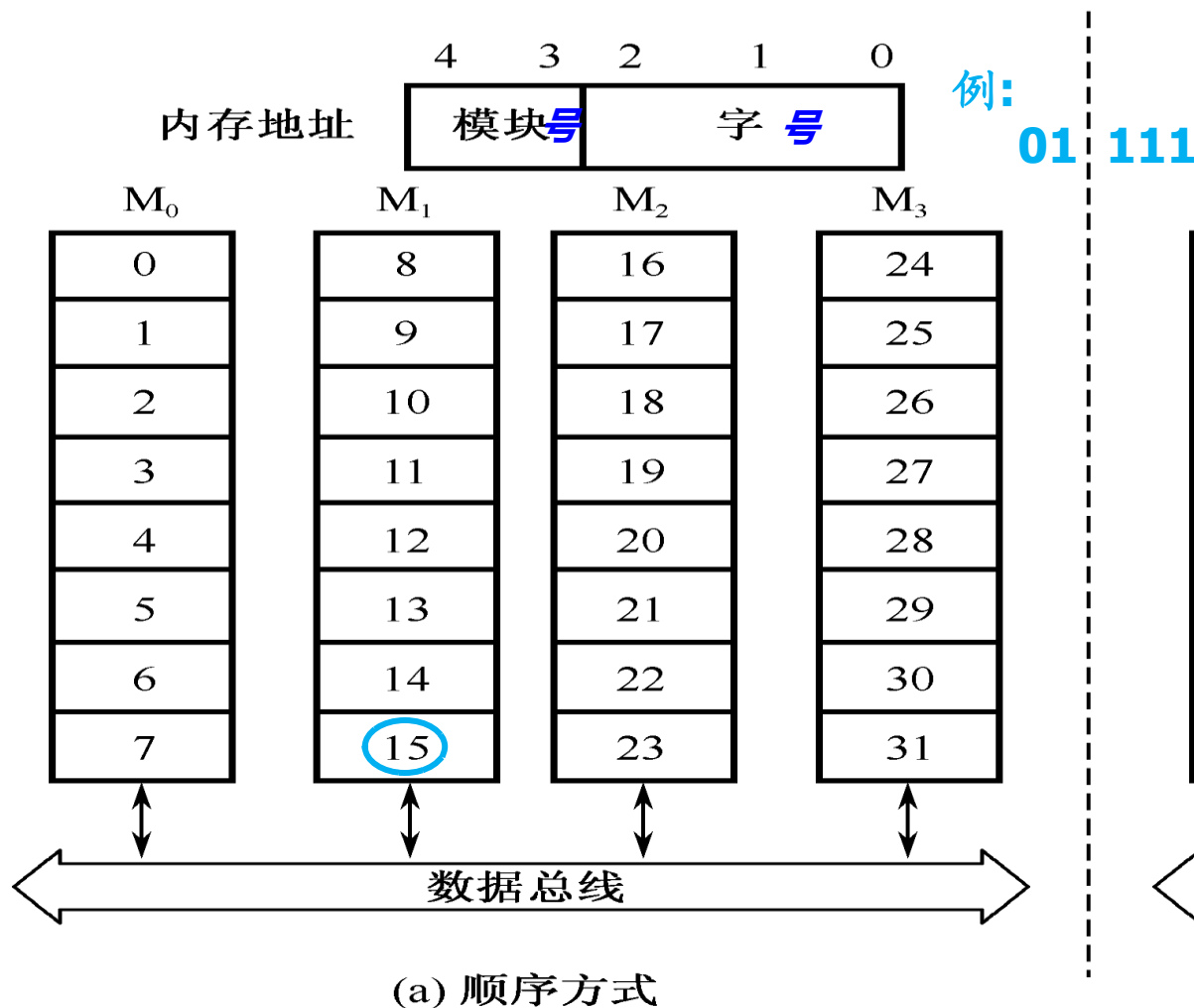
4.4 并行主存系统

由于CPU和主存储器之间在速度上是不匹配的，这种情况便成为限制高速计算机设计的主要问题。为了提高CPU和主存之间的数据传输率，除了主存采用更高速的元器件外，还可以采用 并行技术的存储器。

多体交叉存储器

1. 两种组织方式

一个由若干个存储体模块组成的主存储器是线性编址的。这些地址单元在各模块中如何安排,有两种方式:一种是顺序方式(高位交叉方式),一种是(低位)交叉方式。



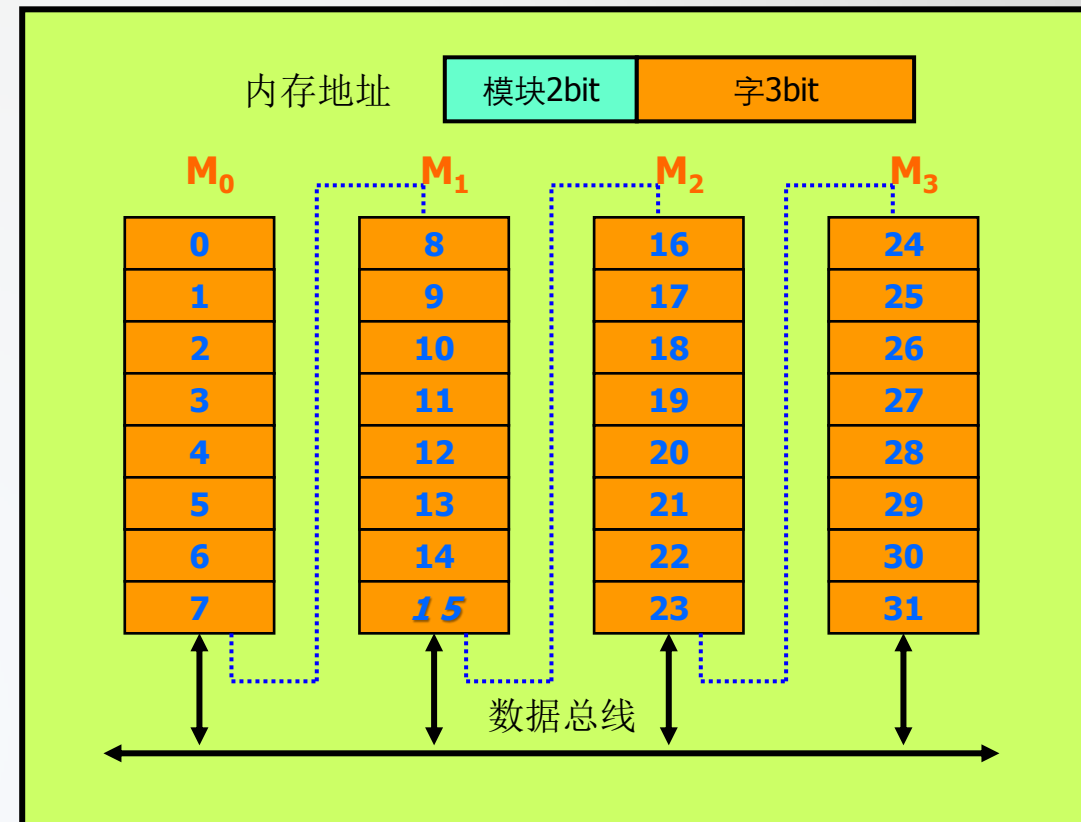
1) 顺序方式 (高位交叉方式)

[例]如图，M0 - M3共四个模块，每个模块8个字

- 5位地址组织：

X	X	X	X	X
高位选模块		低位选块内地址		
- 例：

01	111
----	-----

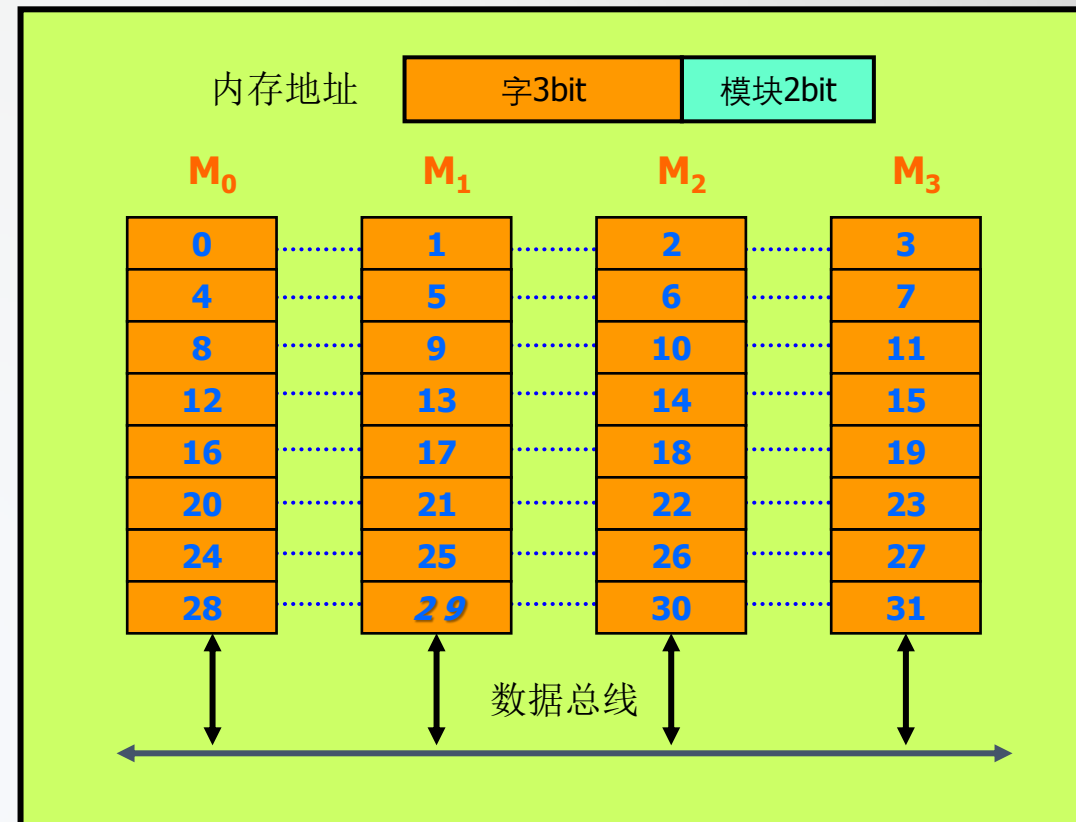


- 该方式特点：某个模块进行存取时,其他模块不工作;优点是某一模块出现故障时,其他模块可以照常工作;通过增添模块来扩充存储器容量比较方便。缺点是各模块串行工作，存储器的带宽受到了限制。

2) 低位交叉方式

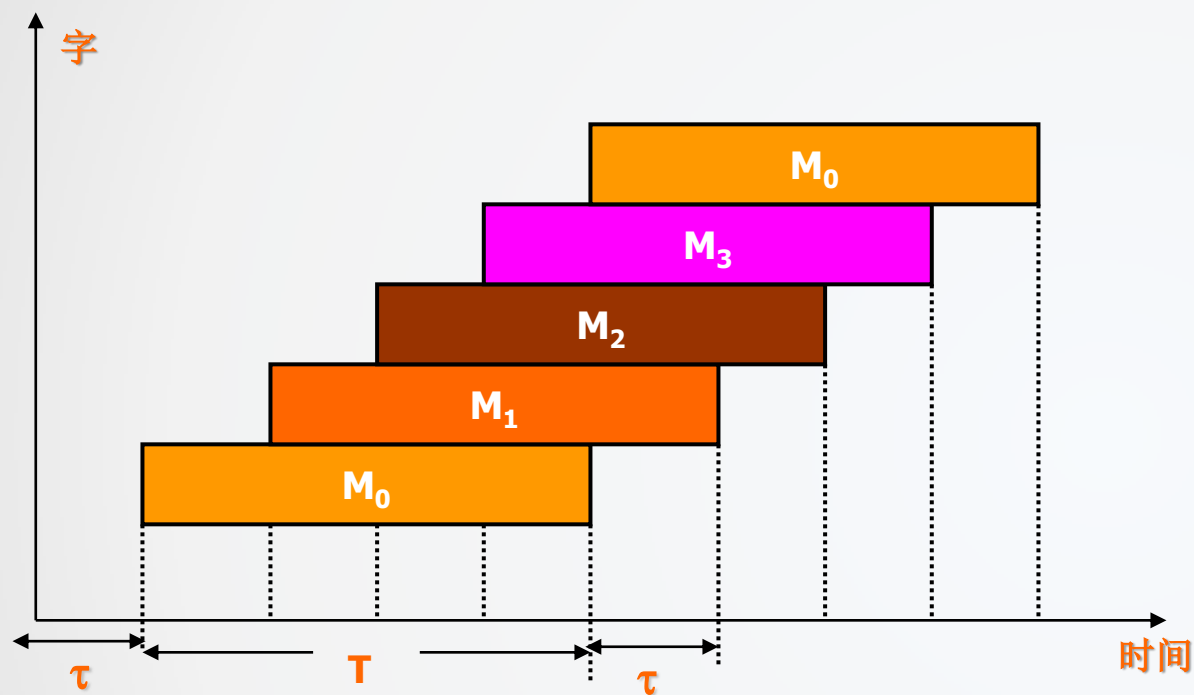
[例]如图，M0 - M3共四个模块，则每个模块8个字

- 5位地址组织： $\begin{matrix} X & X & X & X & X \end{matrix}$
 $\begin{matrix} \text{高位选块内地址} & \text{低位选模块} \end{matrix}$
- 例： $\begin{matrix} 111 & 01 \end{matrix}$



- 特点：连续地址分布在相邻的不同模块内；同一个模块内的地址都是不连续的。**优点**是对**连续字(单元)**的成块传送可实现多模块**流水式并行**存取，大大提高存储器的带宽。

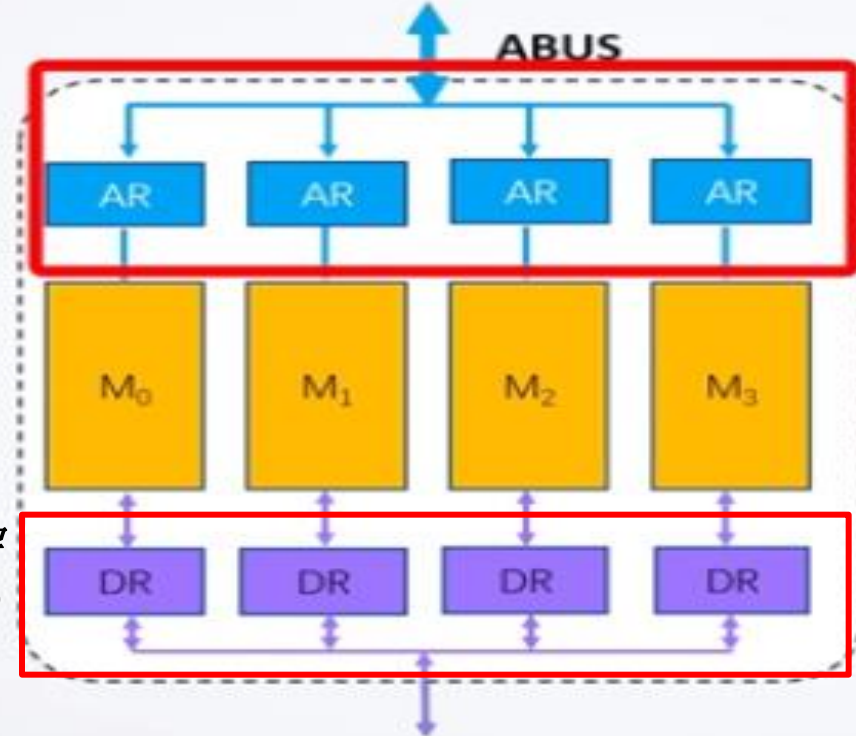
低位交叉存储器 访问时可按流水方式存取:



T : 模块的存取周期 τ : 总线传输周期
 m : 存储器交叉模块数

地址
寄存器AR

数据
寄存器DR



- 需满足 $T = \tau \times m$
- $m = T/\tau$ “交叉存取度”
- 连续读取 n 个字的时间:
- 流水方式 $t_1 = T + (n-1) \times \tau$
- 非流水线 (顺序) $t_2 = n \times T$

$$t_1 < t_2。$$

例: 设存储器容量为32字, 字长64位, 模块数 $m=4$, 分别用顺序方式(高位交叉)和(低位)交叉方式进行组织。存取周期 $T=200\text{ns}$, 数据总线宽度为64位, 总线传送周期 $\tau=50\text{ns}$ 。若连续读出4个字, 问高位交叉存储器(顺序方式) 和低位交叉存储器的 带宽(P97) 各是多少?

解: 由题, 每个模块上有 $32\text{字} \div 4 = 8\text{字}$

高位交叉存储器和低位交叉存储器 连续读出 $n=4$ 个字的信息总量都是:
 $q=64\text{b} \times 4 = 256\text{b}$

由题, 这里满足 $T = m \times \tau$ 的条件

$\therefore$ 顺序存储器和低位交叉存储器 连续读出4个字所需的时间分别是:

顺序 $t_1 = n \times T = 4 \times 200\text{ns} = 800\text{ns} = 8 \times 10^{-7}\text{s}$

低位 $t_2 = T + (n-1) \times \tau = 200\text{ns} + 3 \times 50\text{ns} = 350\text{ns} = 3.5 \times 10^{-7}\text{s}$

顺序存储器和低位交叉存储器的带宽分别是:

$W_1 = q / t_1 = 256\text{b} \div (8 \times 10^{-7})\text{s} \approx 320\text{Mb/s}$ ($1\text{Mb} \approx 10^6\text{b}$)

$W_2 = q / t_2 = 256\text{b} \div (3.5 \times 10^{-7})\text{s} \approx 730\text{Mb/s}$ 。

4.5 cache存储器 (重)

- 是由SRAM组成的**高速**缓冲存储器，为了解决CPU和主存之间速度不匹配问题。
- 其原理基于**程序访问的局部性原理**：一般在较短时间间隔内，程序产生的**地址**多集中在存储器逻辑地址空间的很小范围内。
- 利用以上局部性原理，可在主存和CPU的寄存器之间设置cache，把正在执行的指令地址**附近的 一部分指令、数据 从主存调入(拷贝到)cache**，供CPU一段时间内使用，以提高CPU访存速度。
- 程序继续执行时，程序访问的局部性会随时间变化，新的数据需不断从主存调入cache，**替换**掉旧数据。

如何在存储器中找到待访问内容

- 用于查找的关键字Key: 主存地址
- 如何将: **主存地址** → **Cache 地址**
 - 按程序员的软件思路
 - ◆ **查找表** (类似于索引表) --- 解决如何快速查找
 - 架构师的 硬件思路
 - ◆ **相联存储器** --- 硬件 并发查找。



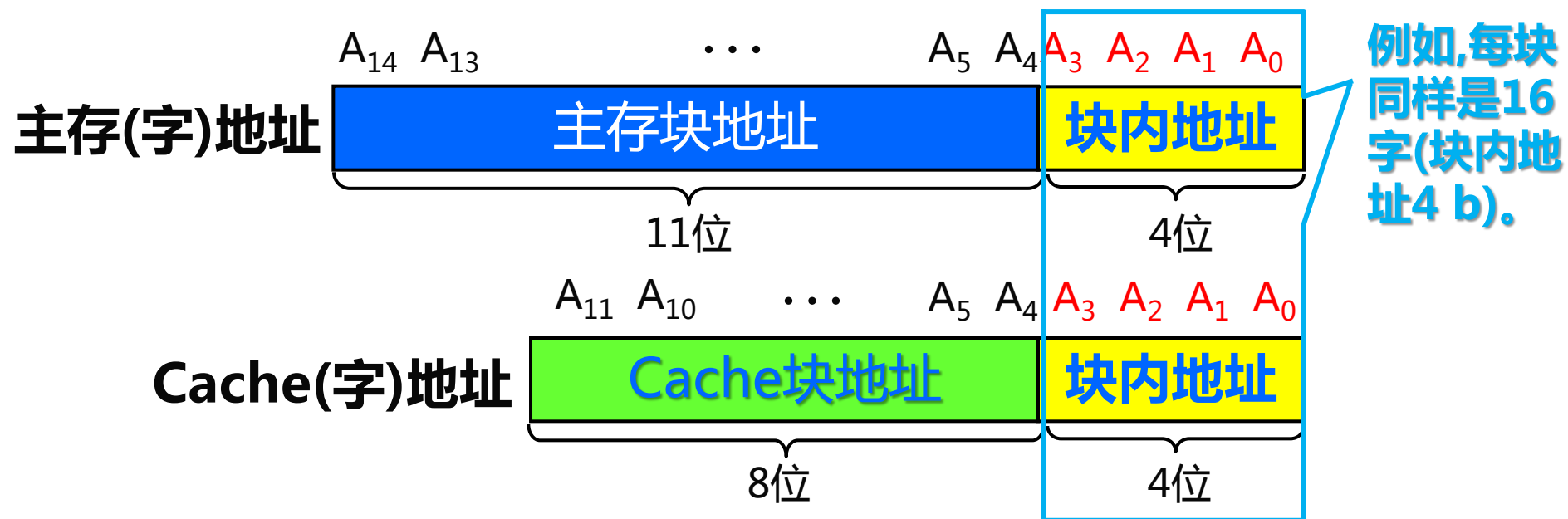
4.5.5 相联存储器CAM

- 是独立硬件, 不在Cache中 (位于CPU)
- 特点: 按内容进行访问
 - 以关键字 作全局并 发 比较
 - CAM存有 查找表, 表项 如下:
 - ◆ (valid, Key , Value)
 - ◆ 即(有效位, 主存块地址 , Cache块地址)
 - 有效位: 表示该表项数据是否有效 .
 - 主存块地址 : 即拷贝到cache的原主存块地址。 (见P125图中例子)。

Cache基本组织方式 (重)

- cache与主存间,以 **块**为单位交换数据
- **cache与主存 各均分为若干数据块 (两者中的块大小相等)**
- 用 相联存储器存放查找表
- CPU给出的主存块地址 与查找表中某表项相同, 则 **命中**
Cache (即 hit)。

块地址 与 块内地址

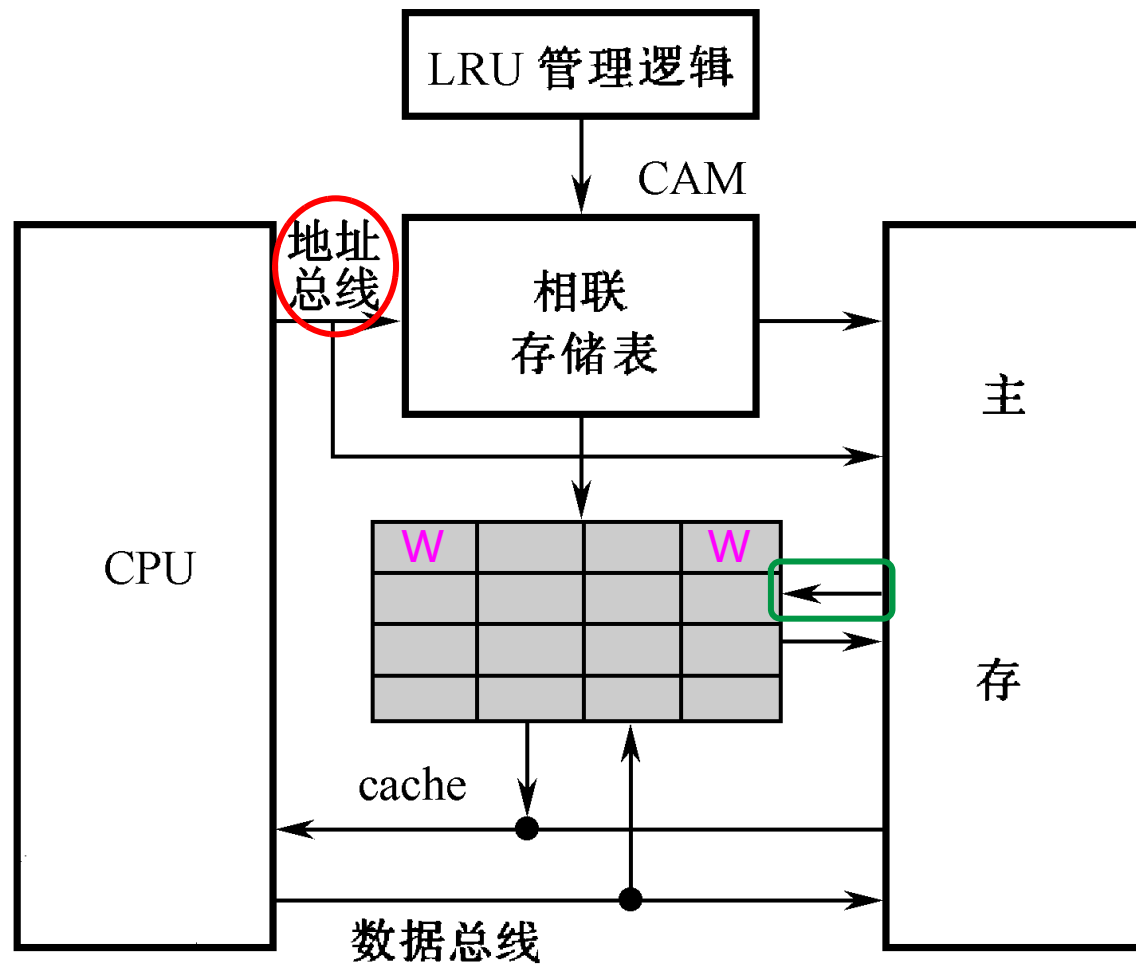


cache系统 读流程：(重)

- 1) CPU给出 主存地址(**主存块地址** , 块内偏移地址)
- 2) 将**主存块地址**作为 关键字Key, 查找CAM中的查找表 .
- 3) 如相符 表示**副本**在cache中, 已命中 , 则根据CAM查找表中记录的cache块地址 去访问cache;
- 否则, 表示数据**缺失**, 则去访问主存 :
 - 并将数据所在块的**副本**调入cache (局部性原理)
 - 载入副本过程可能引起**替换**。

Cache读的基本过程

- 假设cache分为4块(行), 每行 4个字W (主存分块同样大).
- CPU执行访存指令时, 把所要访问的字的地址送到CAM, 若该字W不在cache中(缺失), 则将W从主存读出传送到CPU。与此同时, 把包含该W由前后相继的4个字所组成的1块(行)数据 拷贝送入cache。



Cache的命中率：

从CPU来看，增加cache的目的，就是在性能上使主存的平均读出时间尽可能接近cache的读出时间。为了达到这个目的，在所有的存储器访问(次数)中**由cache满足CPU需要的次数 应占很高的比例**，即cache的命中率 应接近于1。

cache命中率公式：

- 设 N_c 为某程序运行期间 命中cache的次数, N_m 为从主存中访问信息的次数, t_c 表示命中时cache的访问时间, t_m 表示未命中时 主存的访问时间：

$$\text{Cache命中率 } H = \frac{N_c}{N_c + N_m}$$

$$\text{Cache-主存系统的平均访问时间 } t_a = Ht_c + (1 - H)t_m$$

$$\text{系统的访问效率 } e = \frac{t_c}{t_a}$$



例: CPU执行一段程序时, cache完成存取的次数为1900次, 主存完成存取的次数为100次, 已知cache存取周期为50ns, 主存存取周期为250ns。求cache-主存系统的访问效率和平均访问时间。

解：

■ 命中率 $H = N_c / (N_c + N_m) = 1900 / (1900 + 100) = 0.95$

■ $t_a = H \times t_c + (1 - H) t_m = 0.95 \times 50 + (1 - 0.95) \times 250$
 $= 60 \text{ ns}$

■ $e = t_c / t_a \times 100\% = 50 \text{ ns} / 60 \text{ ns} \times 100\% = 83.3\%$

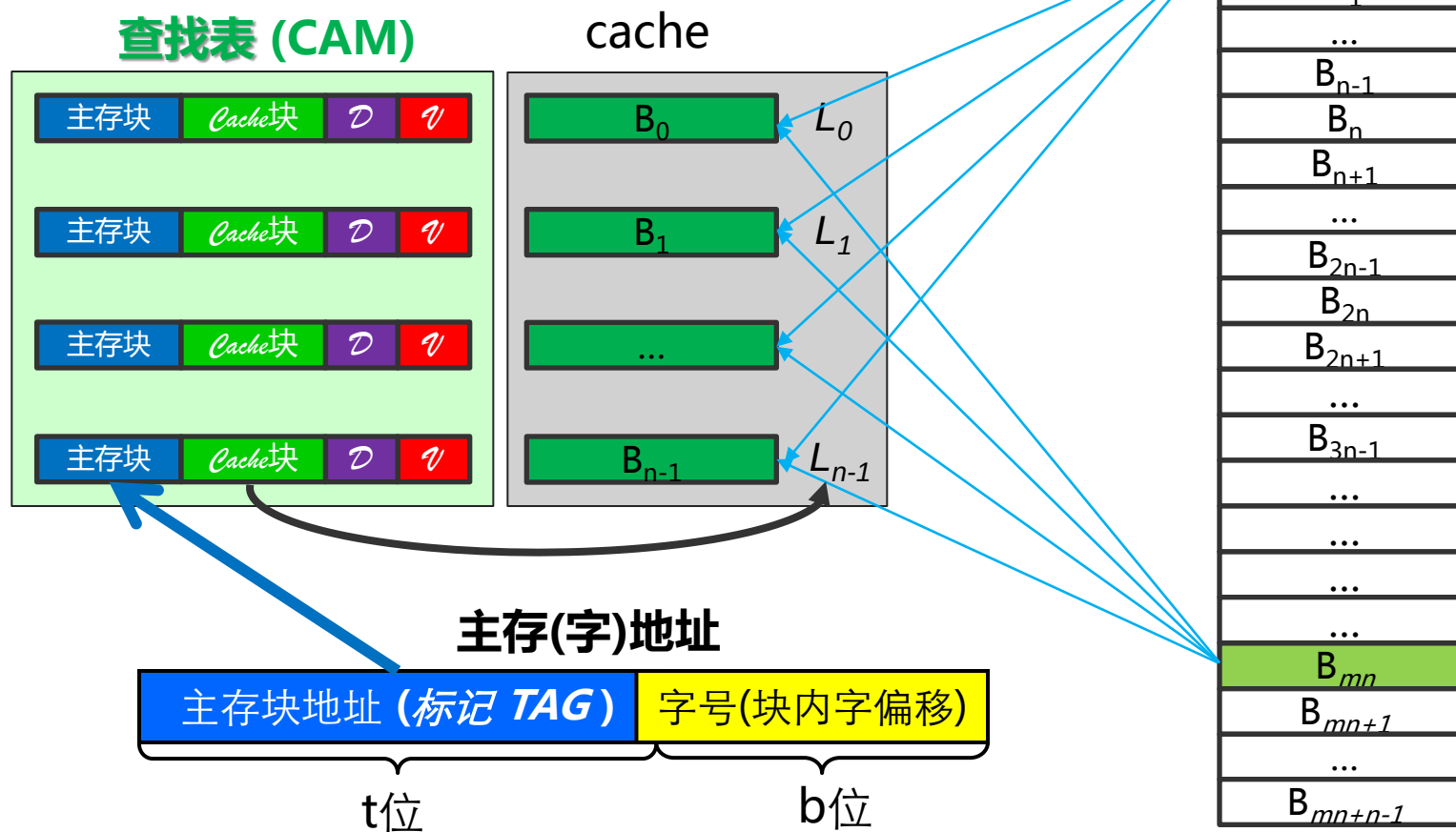
4.5.6 主存与cache的地址映射 (重+难)

- 1、全相联的映射方式
- 2、直接相联映射方式
- 3、组相联映射方式。

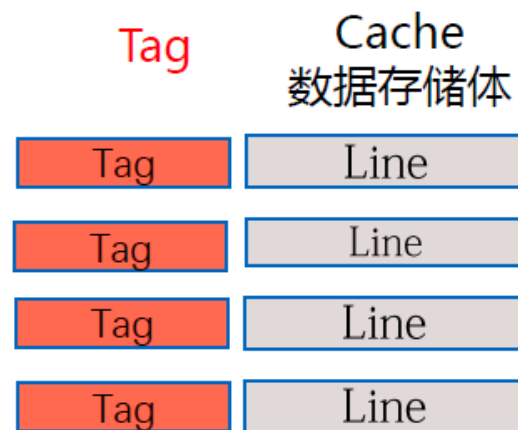
✓ **注意:** 无论选择那种映射方式, 都是把**主存和cache**划分为同样大小的“块”。

全相联映射 (P126)

- **特点:** 主存块可拷贝到任意 cache 行
- 寻址是利用相联存储器的**查找表: 查找慢**



全相联映射



主存

B_0
B_1
...
B_{n-1}
B_n
B_{n+1}
...
B_{2n-1}
B_{2n}
B_{2n+1}
...
B_{3n-1}
...
B_{mn}

例：

- 设每块4个字，主存大小为1024个字，则第61个字的主存地址为：

共 $2^8 = 256$ 块

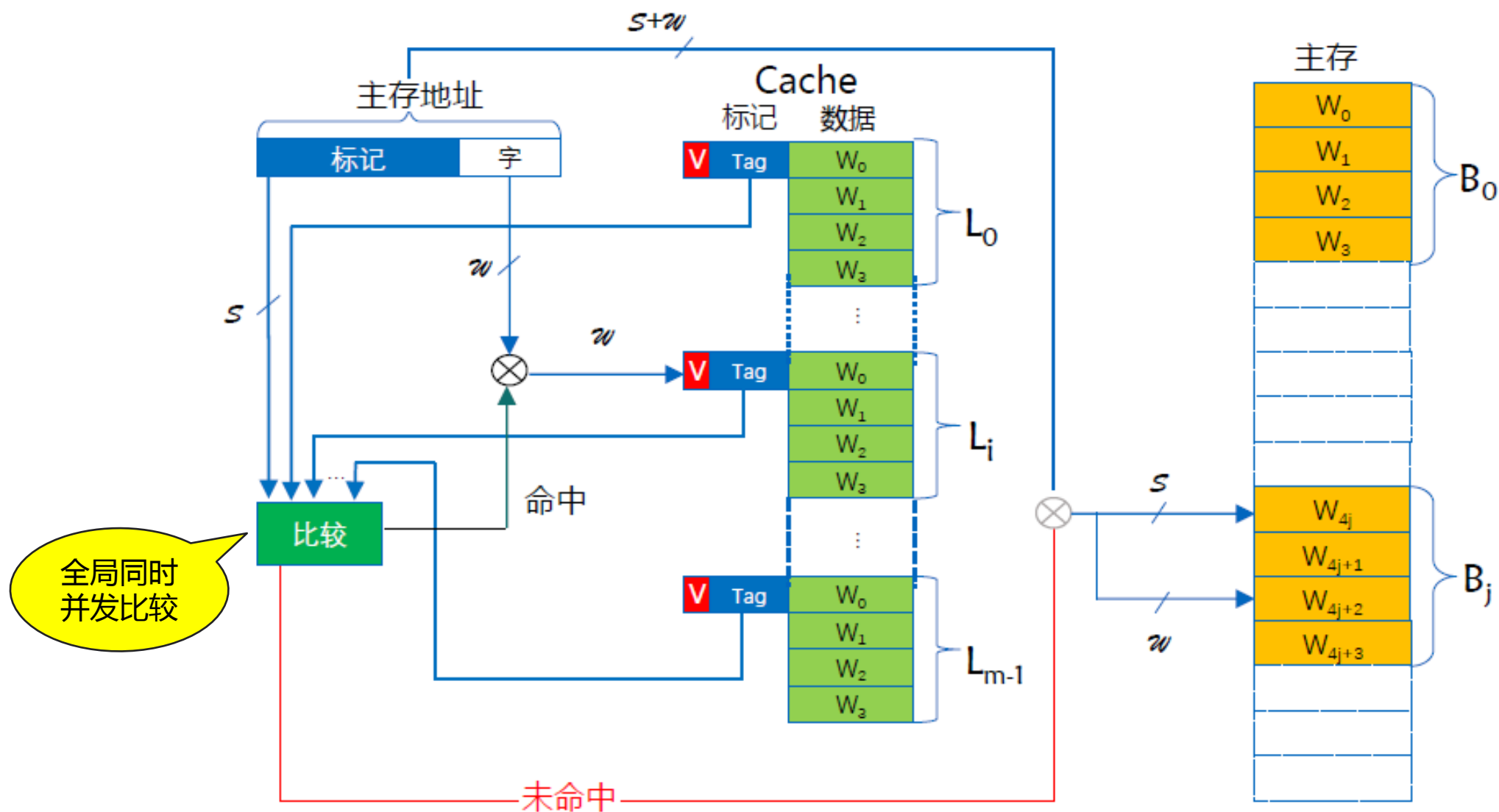
00001111 01 (块号 块内地址)

- 映射算法：主存的数据块可映射到Cache任意行，

映射后同时将该数据块地址号(例 00001111) 送入对应行的标记存储体Tag中保存 (见上图中的Tag)。

全相联映射的工作原理

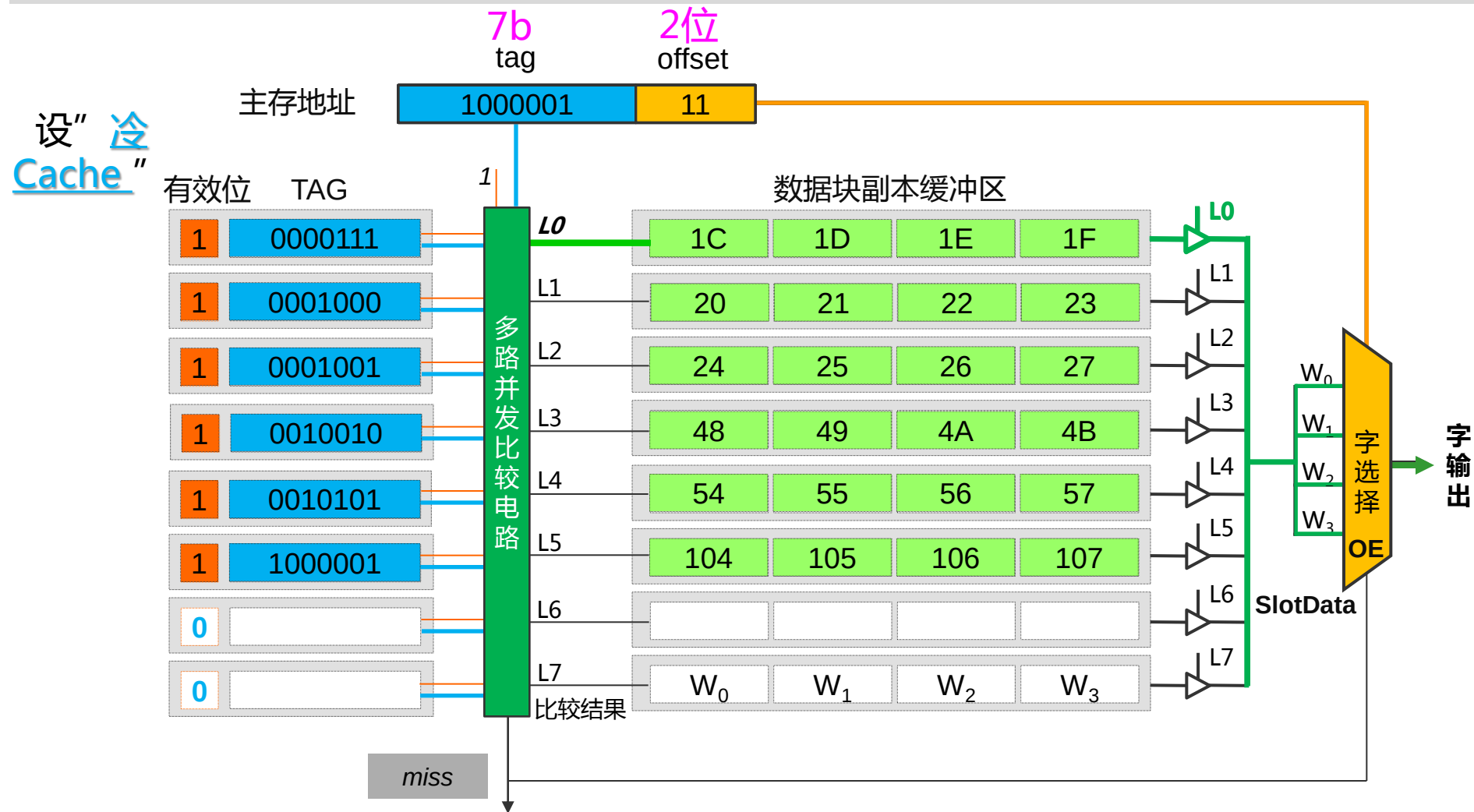
下图: 块大小 4字 W





全相联映射动态载入过程

Cache 8行, 块大小4W, 主存 2^9 W



访问序列

1F

20

24

1E

48

54

107

全相联映射 应用场合

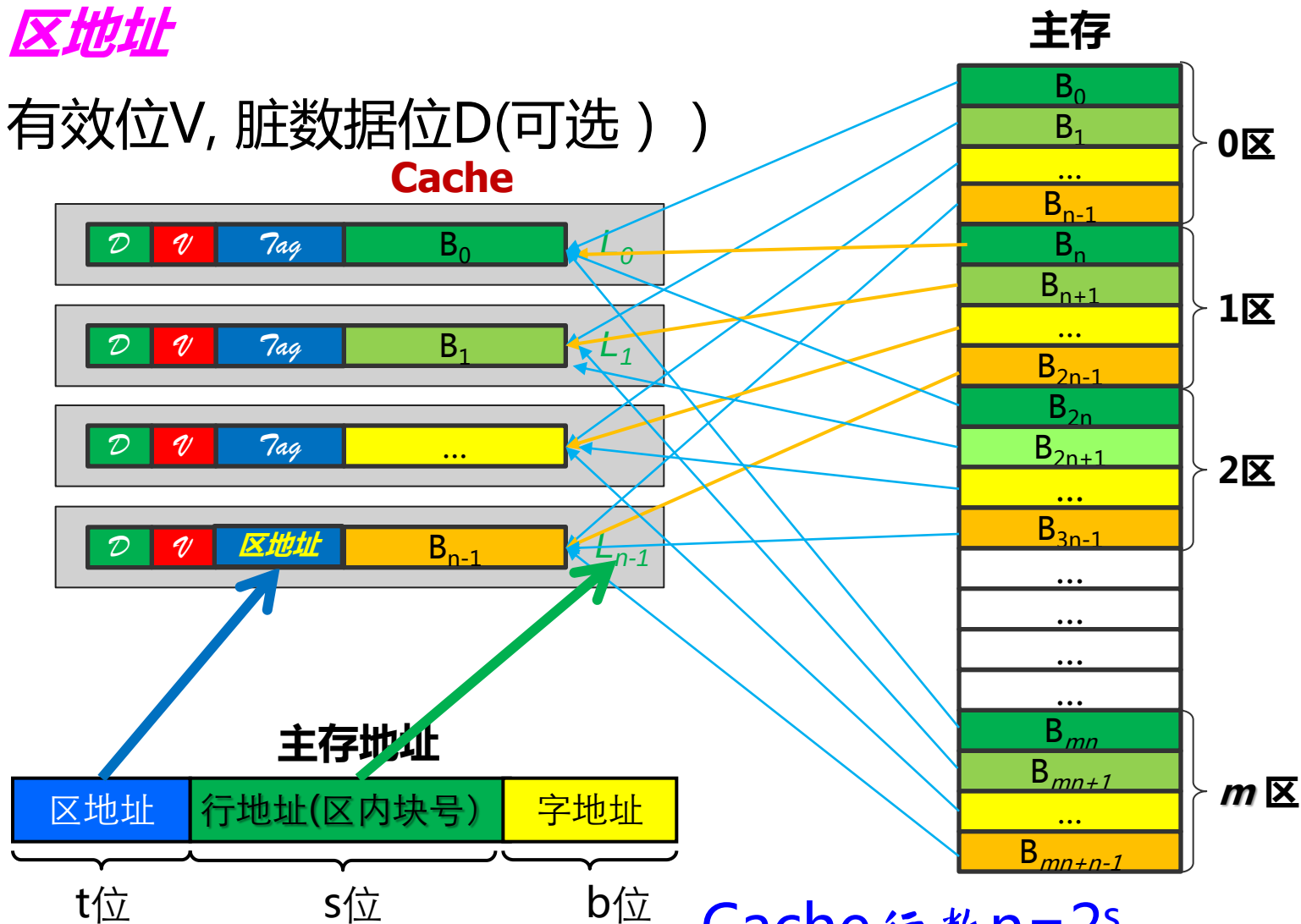
- 可灵活地进行 块的映射; 一对多映射
- cache全部装满后才会出现 块冲突(指: 按映射算法得到的cache块地址, 但该块已用); 块冲突的概率低, cache利用率高.
- 每次地址映射时, 要比较cache所有行的TAG值, 使比较电路复杂, 因此只适合于小容量的cache。

直接相联映射

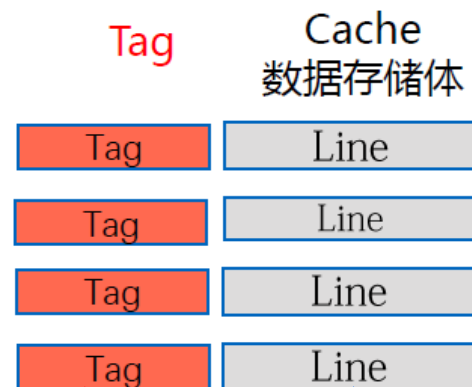
- 主存分块，Cache行（Line），两者大小相同；
- 主存分块后还将以Cache行数为标准进行分区，

➤ 标记位Tag = 区地址

➤ 其他标志位(有效位V, 脏数据位D(可选))



直接相联映射



例 000011 11 01

本例中每块4个字，每4块1个区。主存大小为1024字，Cache分为4行，第61个字的主存地址为：

000011 11 01 (区号，区内块号，块内地址)

•映射算法：

Cache共n行，主存第j块号映射到Cache的行号为：

$$i = j \bmod n \text{ (上例中 } j = 11 \text{ B, } n = 4 \text{)}$$

即主存的数据块映射到Cache特定行

主存

B ₀
B ₁
...
B _{n-1}
B _n
B _{n+1}
...
B _{2n-1}
B _{2n}
B _{2n+1}
...
B _{3n-1}
...
B _{mn}

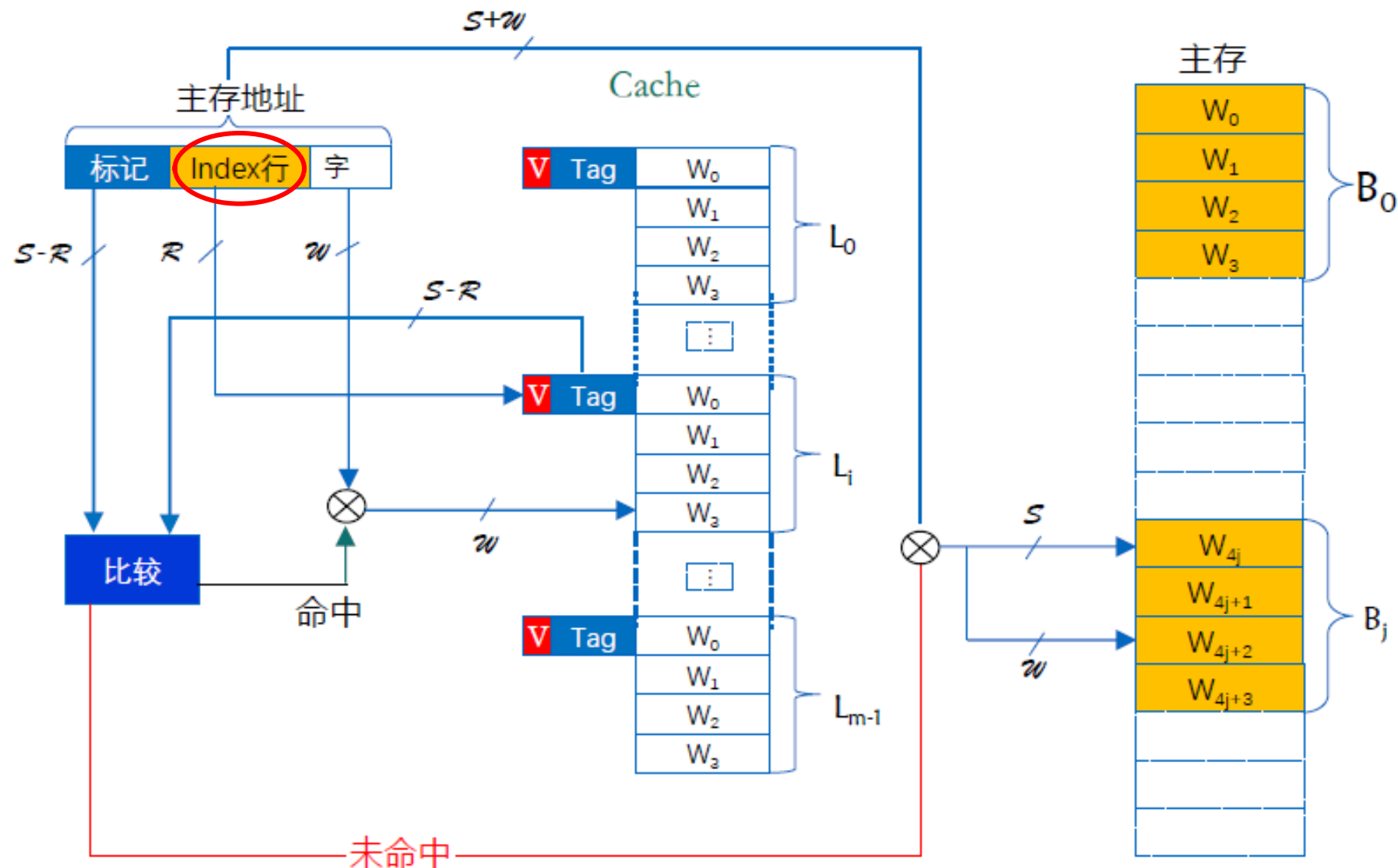
直接相联映射工作原理

主存地址

区地址-Tag

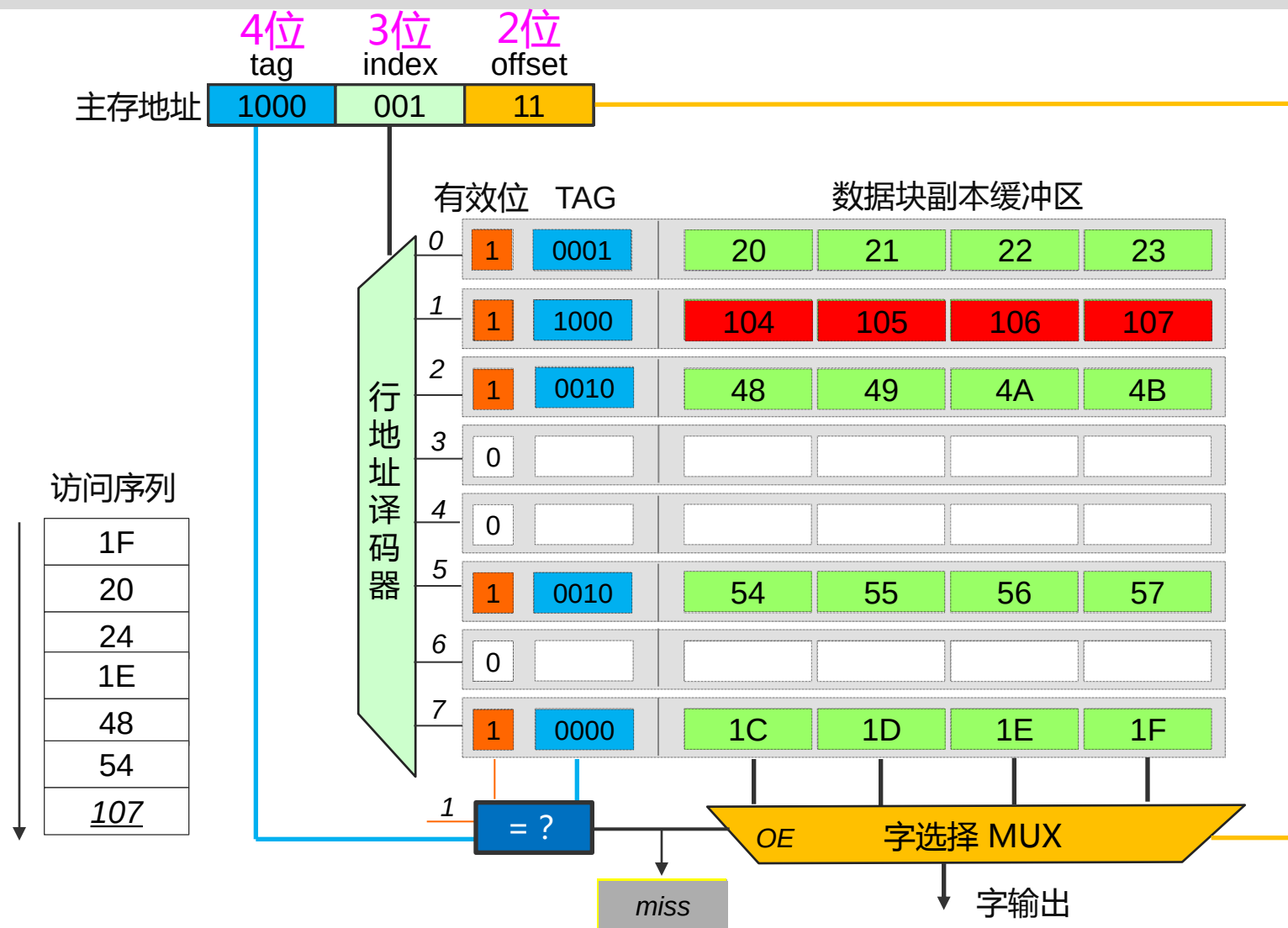
行地址-索引 (区内块偏移)

字(块内字偏移)



直接相联映射的载入过程:

Cache **8行**, 块大小4W, 主存 2^9 W



直接相联映射 应用场合

- 块映射速度快，一对一映射，无须用CAM全局查表
 - 利用索引字段index 直接对比相应Cache行的标记位Tag即可
- 主存块只能映射到Cache中特定行, 因此Cache利用率低, 命中率低
- Cache容易块冲突 (因index相同的所有主存块映射到cache中同一行)
- 每次查找时只需 1个比较器, 硬件成本低, 适合 大容量 Cache。

作业 3

- 本子上完成:
- 1、教材P147习题4.1: 相联存储器
- 2、教材P149习题4.3 (4)(5)
- 3、试题书P39: 13
- 4、试题书P168: 12(“请给2个十六进制表示的内存地址01234 ...”)。先独立完成, 再订正答案。

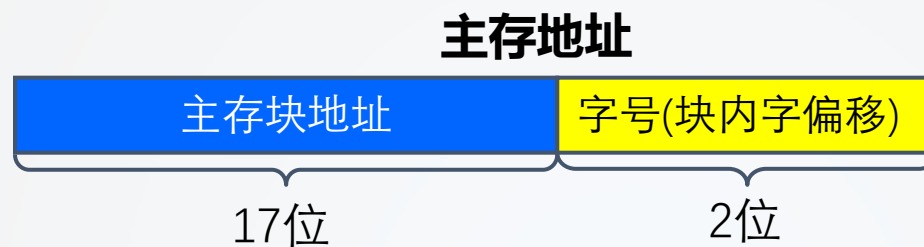


例: 假设主存容量为 $512\text{K} \times 16$ 位, cache容量为 4096×16 位, 块长为4个**16位的字**, 访存地址为字地址 (**以字编址**)。

- 1) cache地址有多少位? 可容纳多少块?
- 2) 主存地址有多少位? 可容纳多少块?
- 3) 在全相联映射方式下, 设计主存的地址格式。
- 4) 在直接映射方式下, 设计主存的地址格式。

- 解: 1) cache容量为 $4096\text{字} = 2^{12}\text{字}$, cache字地址的长度为12位。每个块4个字, 故 cache中有 $4096\text{字} / 4\text{字} = 1024\text{块(行)} = 2^{10}\text{行}$
- 2) 主存容量为 $512\text{K 字} = 2^{19}\text{字}$, 故主存字地址共19位。主存中有 $2^{19} / 4 = 2^{17}\text{块}$ 。

■ 3) 全相联

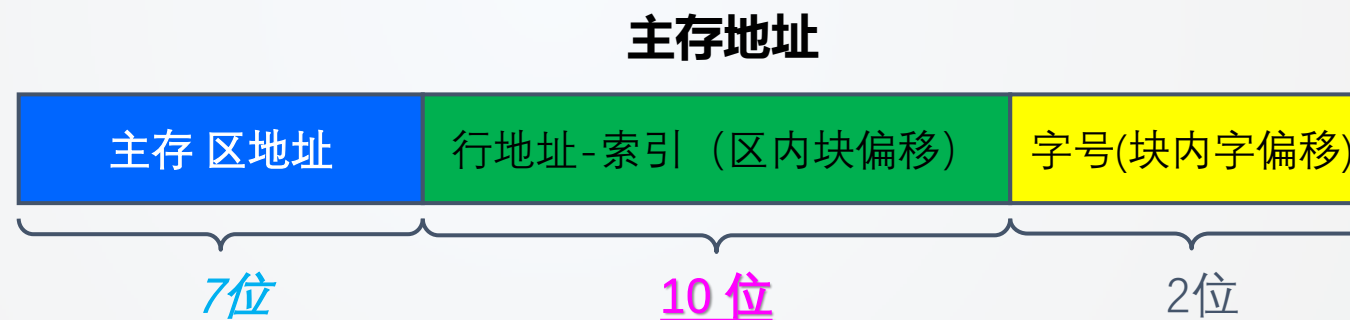


● 4) 直接相联

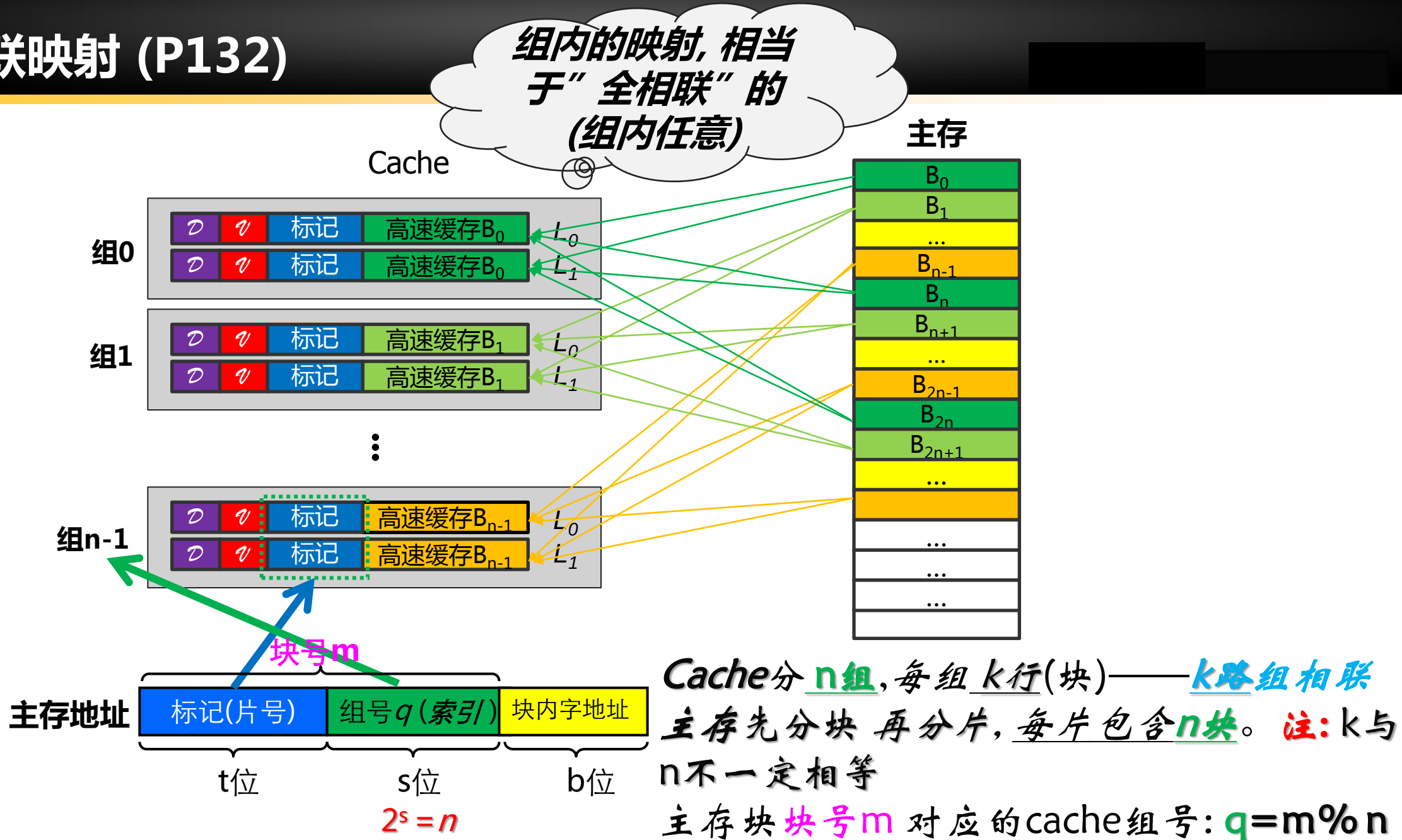
● 主存分区数

$$= 2^{17} \text{块} / 2^{10} \text{行}$$

$$= 2^7$$

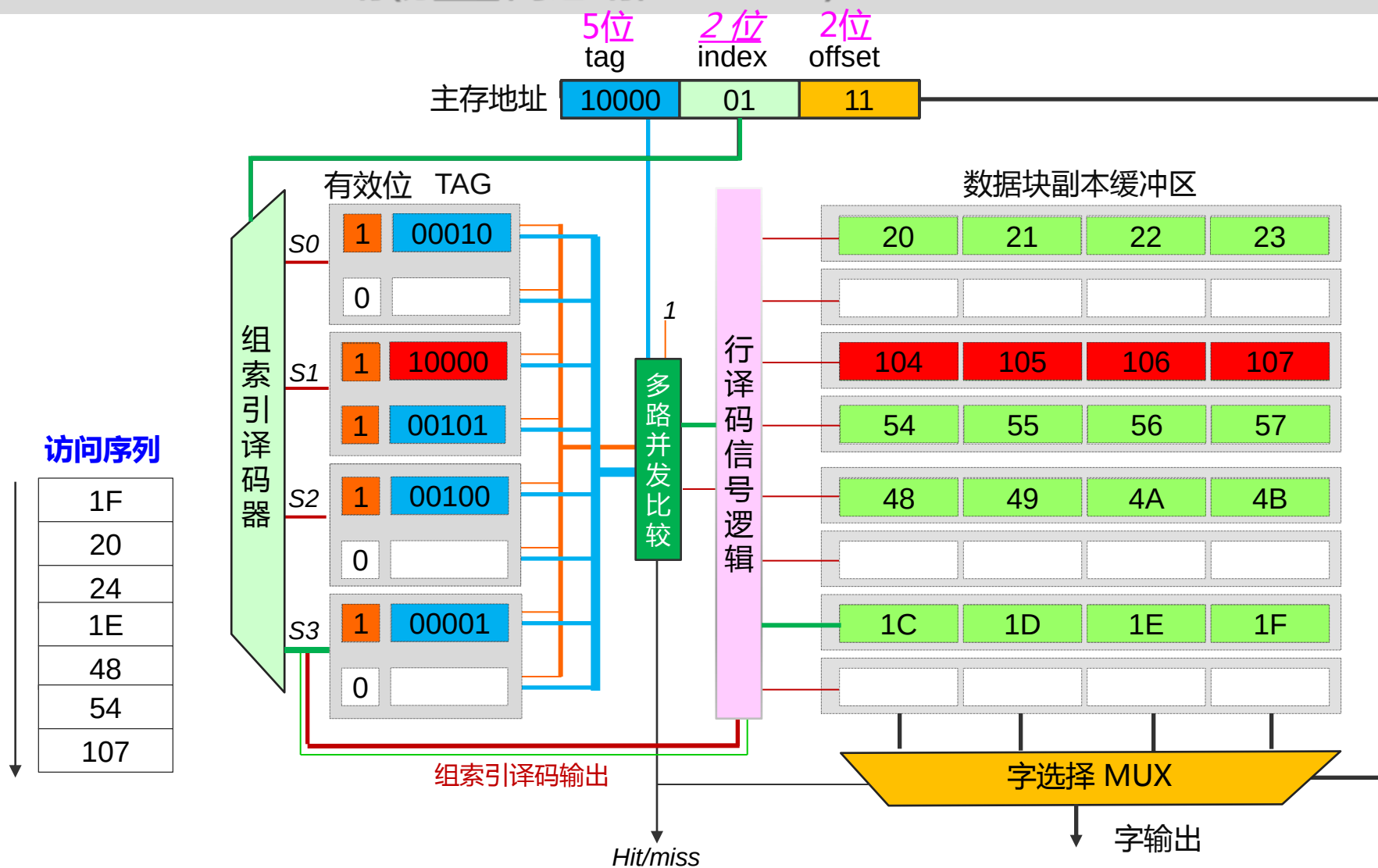


组相联映射 (P132)



组相联映射动态载入过程

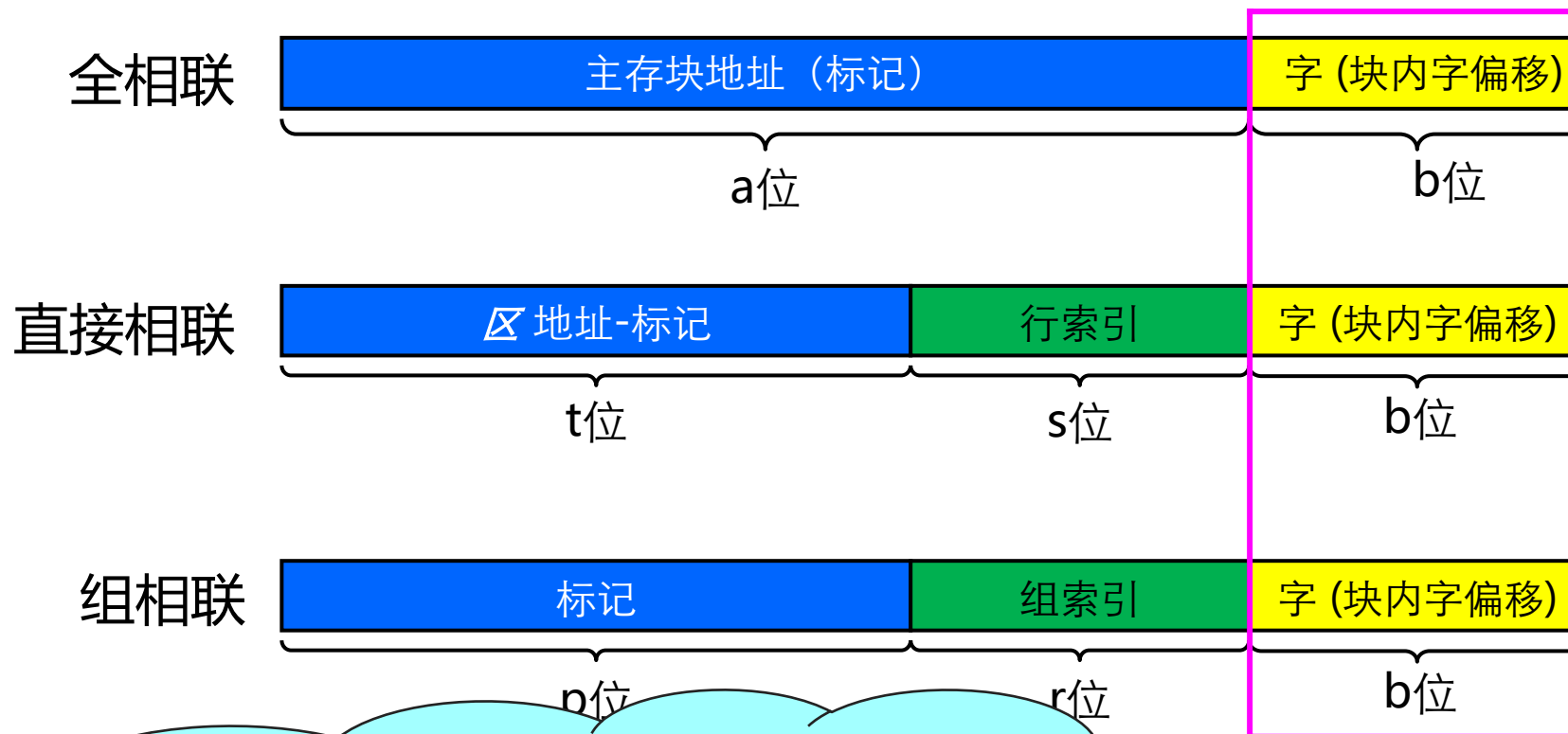
Cache **8行(分4组,每组2行,二路组相联)**, 块大小 $4W$, 主存 $2^9 W$



组相联映射 应用场合

- 容量不大的cache可采用全相联映射 或 组相联映射 (k 路组相联, 则需 k 个比较器)
- 容量大的cache可采用直接相联映射：
 - 查找速度快，但命中率相对低
 - 但cache容量大时可提高命中率。

小结--不同映射方式下主存地址划分



- 组相联映射中, 若cache只分1组, 则组相联映射变成了 全相联;
- 组相联中, 若cache每组只含1行, 则组相联变成了 直接相联。

(解答见 试题书P191)

例: 某计算机的主存地址空间大小为256MB , **按字节编址**。指令Cache 和 数据Cache 分离,均有8个Cache行, 每个Cache行大小为64B; **数据Cache**采用 **直接映射方式**。现有两个功能相同的程序A、 B , 其伪代码如下所示 : (2010研究生统考)

```
int a[256][256];  
for (i = 0; i < 256; i++)  
    for (j = 0; j < 256; j++)  
        sum += a[i][j];
```

程序 A

```
int a[256][256];  
for (j = 0; j < 256; j++)  
    for (i = 0; i < 256; i++)  
        sum += a[i][j];
```

程序 B

假定int类型数据用32位补码表示 , 程序编译时 i、 j、 sum均分配在寄存器中 , 数组a 按 **行优先**方式 存放 , 其(首)地址为320(十进制)。

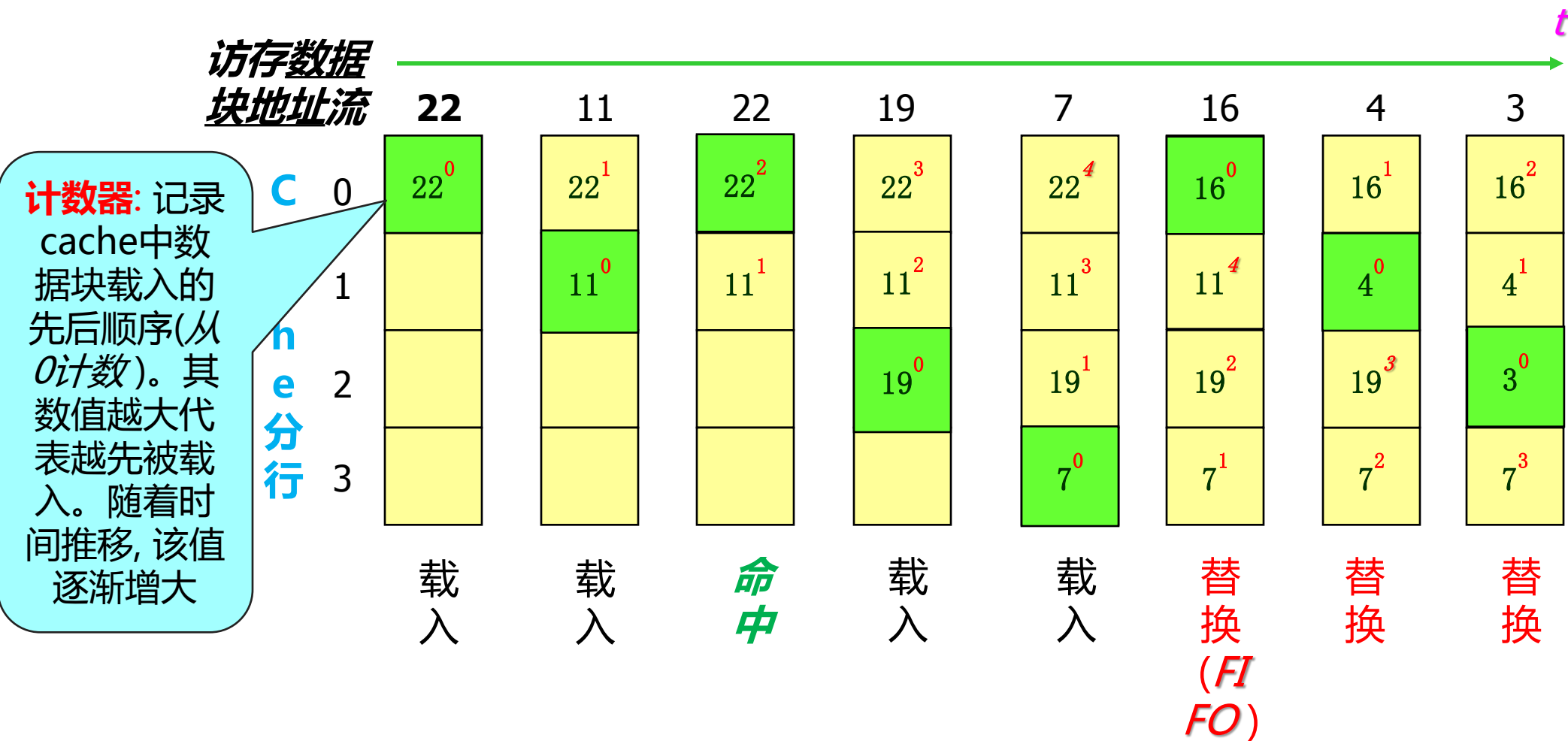
- (1) 若不考虑用于Cache一致性维护和替换算法的控制位 , **数据cache**的总容量是多少 ?
- (2) 数组元素a[0][31],a[1][1]各自所在的主存块对应的cache行号分别是多少 (**cache行号从0开始**) ?
- (3) 程序A、 B的数据访问(Cache)命中率各是多少 ? 哪个程序的执行时间更短 ?

对程序B的解答, 见谭教材P132第2段。

- 若cache中已装满数据，当新的数据块要装入时，须将cache中原数据块替换掉。选择替换掉哪一行, 常用替换算法有:
 - 先进先出算法 FIFO
 - 最近最不经常使用算法 LFU
 - 近期最少使用算法 LRU
 - 随机替换法: 即需要进行替换时, 随机选取一行进行替换。

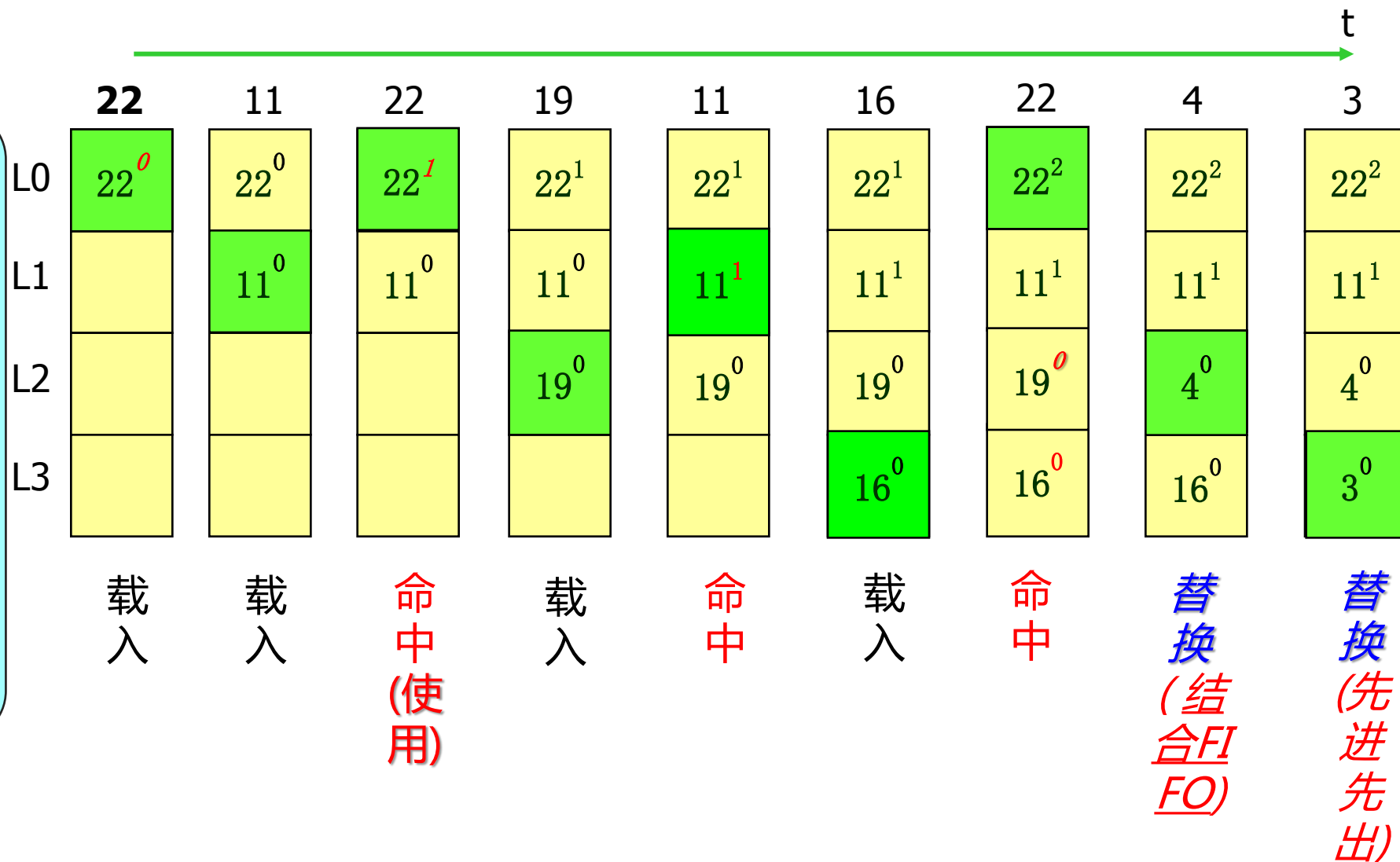
先进先出替换 (FIFO)

假设这里采用
全相联映射, 下同



最不经常使用算法(LFU)

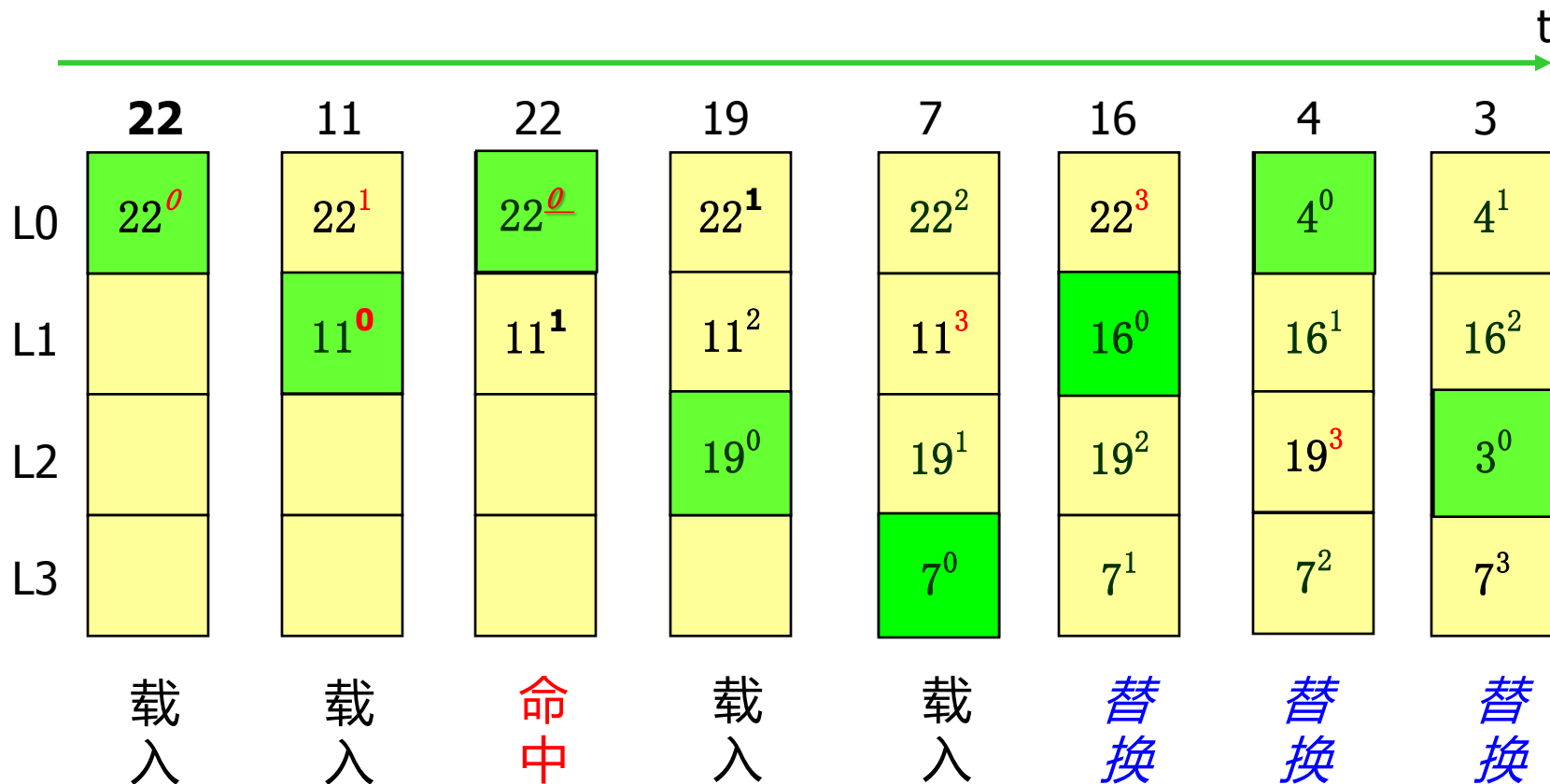
计数器: 记录cache中数据块被使用的次数, 从0计数, 每被使用1次则计数器值增1。需要替换时, 选择计数器值最小的行换出。遇到有多个可选行时, 可再综合FIFO算法



近期最少使用算法(LRU)

未访问次数计数器: 初载入的行, 其计数器值为0。

cache每命中一行, 该命中行的计数器值清0, 其他各行的计数器值增1。需要替换时, 将计数器值最大的cache行换出





4.5.8 cache系统 写的流程

- 1) CPU给出写主存字的地址(主存块地址 , 块内字偏移)
- 2) 以主存块地址为关键字 , 在相联存储器的查找表中查找
 - 若相符 则表示命中 , 即数据副本在cache里 , 转3)
 - 若缺失 则根据写策略 决定是否将该主存地址对应数据块调入cache .
- 3) 根据cache字地址 (见教材P125图), 写入数据字到cache ;
- 4) 根据写策略 决定cache数据是否写回主存。

写策略

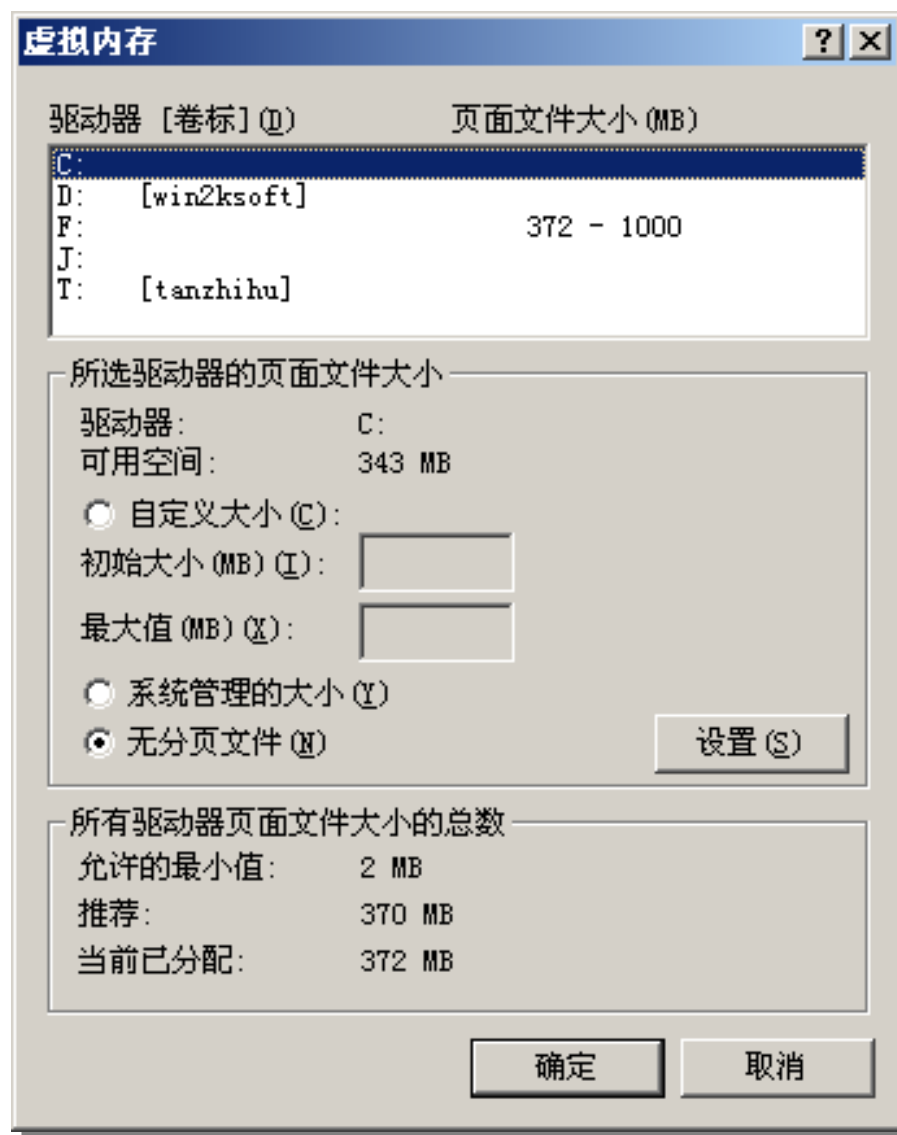
- 由于cache的内容只是主存部分内容的拷贝, 它应当与主存内容保持一致。而CPU对cache的写入更改了cache的内容。
- 如何使cache块与主存内容保持一致? 可选用如下3种写操作策略:
 - **写回法:** 修改cache时,不立即写入主存,只设置修改位; 被替换出时,对cache行的修改位进行判断, 决定是写回主存还是舍弃掉 (详见教材P136)。
 - **全写法 (写穿法):** 写命中时, Cache与主存一起写; Cache未命中(缺失)时, 直接向主存写。
 - **写一次法:** 与写回法基本一致, 但第一次Cache命中时, 采用全写法 (两者一起写)。

4.6 虚拟存储器

- 如何在有限的主存空间运行较多的较大的用户程序？
 - 初衷： 采用虚存来扩大寻址空间
 - 现在： 采用虚存进行存储保护、程序共享
- 主存、辅存 在操作系统和硬件的管理之下，提供比实际主存 大得多的存储空间——**虚拟存储器**。
- 虚存空间是靠 **辅存(硬盘)**来支持的。
- **虚拟存储器**采用与Cache类似原理，提高速度，降低成本，用户感觉到的是一个**速度接近主存 而容量极大的存储器**。

虚拟内存实例

- Linux
 - 交换分区swap
- Windows
 - 页面文件。

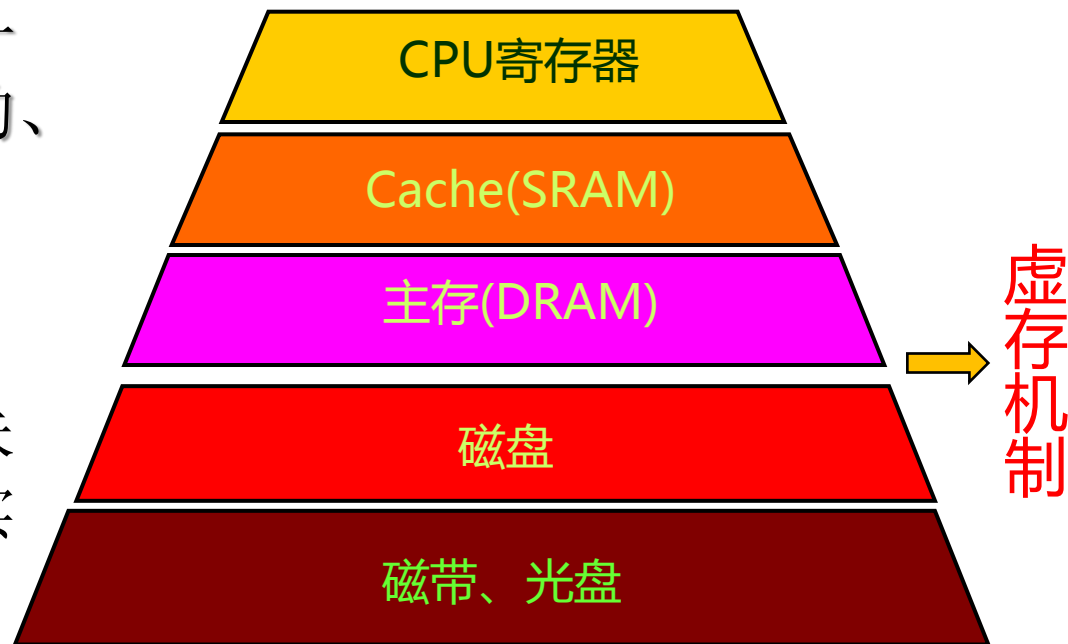


1、实地址与虚地址:

- 用户编写程序时使用的地址称为**虚地址**或 逻辑地址，其对应的存储空间称为 **虚存空间** 或**逻辑地址空间**；而计算机物理内存的访问地址则称为**实地址**或 物理地址，其对应的存储空间称为 **物理存储空间**。
- 每个程序的虚地址空间：
 - 可以远大于实地址空间→以扩大存储容量为目的
 - 也可远小于实地址空间。

层次性结构

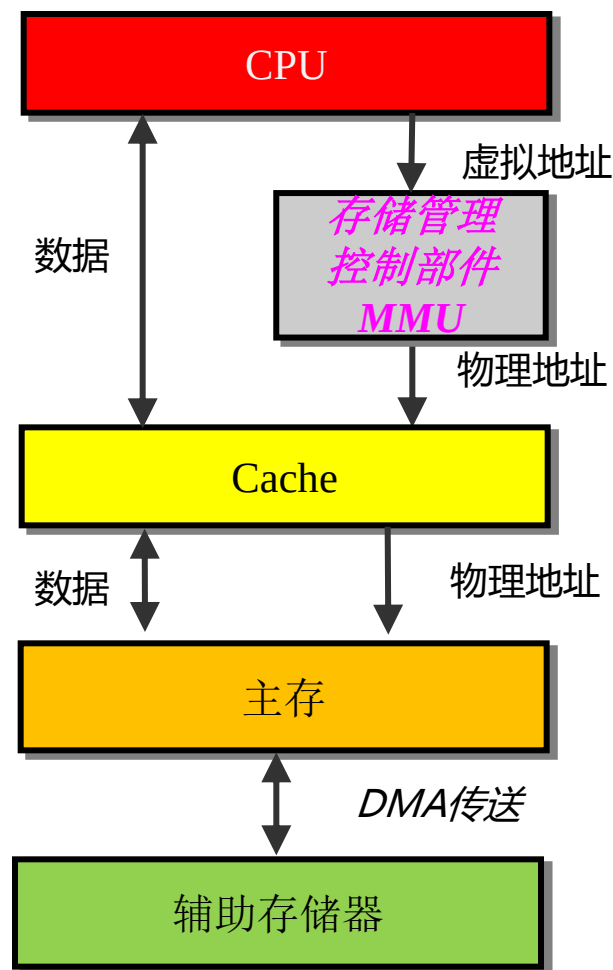
- 虚拟存储器处于“主存-辅存”存储层次, 它使辅存和主存构成一个有机整体, 就像一整个单一的、可供CPU直接访问的大容量主存.
- 程序员可用虚存提供的地址进行编程, 这样在实际主存空间未增加情况下, 用户编程不再受实际主存空间大小的限制.



- **cache**和**主存**之间 以及 **主存**和**辅存**之间, 各级存储器组成了有机的**三级存储体系**。**cache**和主存构成了系统的内存, 而主存和辅存构成了虚拟存储器。工作原理见后页。

2、虚存的访问过程

- 虚存空间的*用户程序*按照虚地址编程并存放在辅存中。程序运行时,由**地址变换机构MMU**依据当时分配给该程序的*实地址空间*把程序的一部分调入主存:
- 每次访存时,首先判断该虚地址所对应的实地址是否主存中;若是,则进行**地址转换(映射)**并用实地址访问主存;否则(即:实地址在辅存),则按照某种算法将辅存中的部分程序(一页等)调度进主存,再按同样的方法访问主存。



Cache-主存 和 主存-辅存 两个存储层次的相似之处

请自学
以下3页

- 将程序中常用的部分驻留在 高速存储器
 - 程序载入时按需载入（不是全部载入）
 - 高速存储器分块或者分页（粒度问题）
 - 主存空间满后，将不常用程序或数据淘汰或交换到辅存中。
- 数据的换入换出由硬件或操作系统完成，对用户透明。
- 利用程序局部性，使存储系统的性能接近于高速存储器；价格接近于低速存储器，并扩充主存容量。

Cache-主存和主存-辅存 两个存储层次的不同之处：

- 1) **侧重点不同。** cache主要解决主存与CPU的**速度**差异问题；而就性能价格比的提高而言，虚存主要是解决存储**容量**问题。
- 2) **数据通路不同。** CPU与cache和主存之间均有直接访问通路，cache不命中时可直接访问主存；而虚存所依赖的**辅存与CPU之间不存在直接的数据通路**，当主存不命中时只能通过调页解决，CPU最终还是要访问主存。

Cache-主存和主存-辅存 两个存储层次的不同之处：

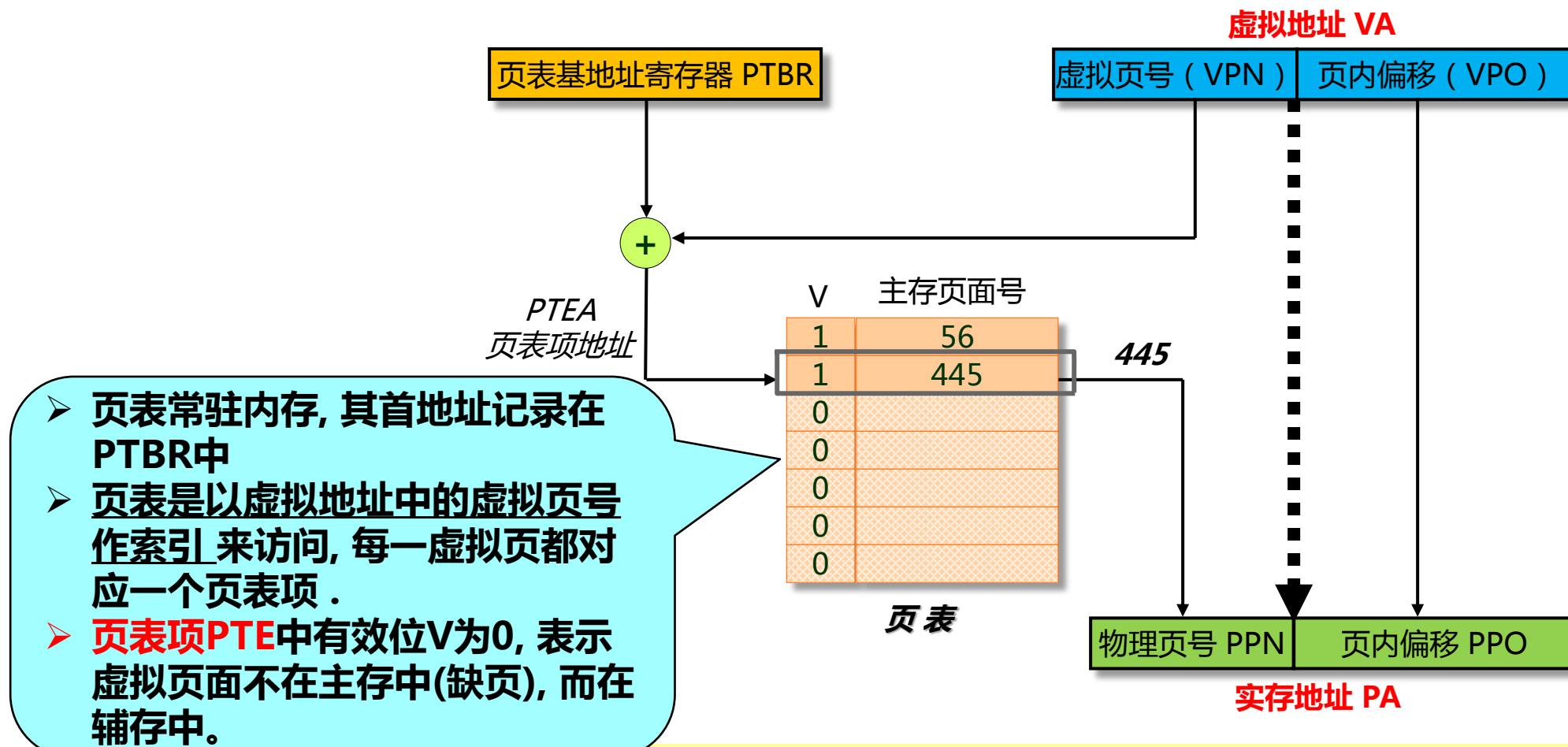
- 3)透明性不同。** cache的管理完全由硬件完成，对系统程序员和应用程序员均透明；而虚存管理由软件(操作系统)和硬件共同完成，因此 虚存对实现存储管理的系统程序员不透明，而只对应用程序员 透明。
- 4)未命中时的损失不同。** 由于主存的存取时间是cache的存取时间的5 ~ 10倍，而主存的存取速度通常比辅存的存取速度**快上千倍**，故主存-辅存存储层次中,主存未命中时系统的性能损失 远大于cache未命中时的损失。

4.6.3 页式 虚拟存储器 (重)

1、页式管理

- 信息交换的单位是**定长的页** (即 数据块).
- **主存与辅存 均划分成等长的页面.**
- **页式管理的优点**: 便于维护,便于生成**页表**; 不容易产生碎块, 存储空间浪费小。
- **虚拟地址**分为两个字段: **高字段** 为 (逻辑)**虚拟 页号**, **低字段**为 **页内地址**(偏移量); **实存地址**也分两个字段: **高字段** 为 **物理页号**, **低字段**为**页内地址** .
- 通过页表可把 虚拟地址(逻辑地址) 转换成 物理地址。**页表** 是一张保存 **虚拟页号** 和 **物理页号** 间对应关系的**查找表** (OS软件实现), 是一个由若干表项组成的数组。基于页表的地址映射过程见下页图。

2、页式虚存地址映射



■ 图中几个英文简称:

■ 虚拟(逻辑)页号: VPN

物理页号:

PPN

■ 页表项:

PTE

页表项地址:

PTEA

P141例4.6

例 4.11 设主存容量为 16MB, 按字节寻址。虚拟存储器容量为 4GB, 采用页式虚拟存储器, 页面大小为 4KB。完成下列各问:

- (1) 计算物理页号、页内偏移字段、虚拟页号字段各为多少位。
- (2) 计算页表中页表项的数量。

解: (1) 由于页面大小为 4KB, 因此页偏移量字段为 12 位, 物理页号字段的位数

: $16\text{MB} / 4\text{KB} = 4\text{K} = 2^{12}$ 个物理页 $\rightarrow$ 物理页号12位

虚拟页号字段: $4\text{GB} / 4\text{KB} = 2^{20}$ 个虚拟页 $\rightarrow$ 虚拟页号20位 .

(2) 页表项的数量 即虚拟页的数量 $= 2^{20}$ 项

第(3)问见下页。

(3) 若部分页表内容如表 4.9 所示,求对应于虚拟地址 $(00015240)_H$ 和 $(03FFF180)_H$ 的物理地址。

表 4.9 部分页表内容

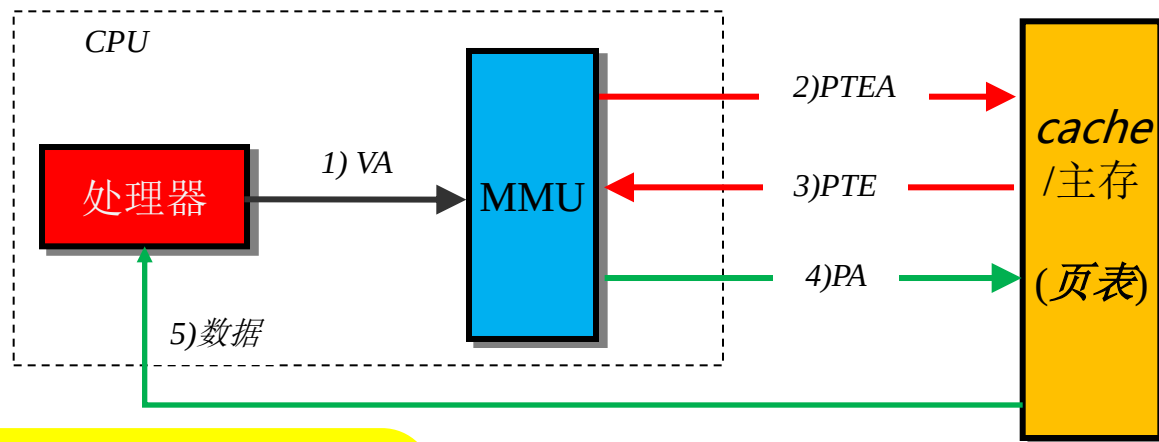
虚页号	有效位	物理页号	虚页号	有效位	物理页号
00010H	0	002H	03FFFH	0	153H
00015H	1	035H	...	...	...
...	...	...			

■ 解：

(3) 从虚拟地址 $(00015240)_H$ 和 $(03FFF180)_H$ 中分离出的 20 位虚页号分别为 00015H 和 03FFFH。查页表可知与虚页号 00015H 对应的物理页号为 035H,而与虚页号 03FFFH 对应的页不在主存中,因为对应的有效位为 0。

与虚拟地址 $(00015240)_H$ 对应的物理地址为 $(035240)_H$,其中的物理页内偏移字段 240H 直接来自虚拟地址的虚拟页内偏移字段。

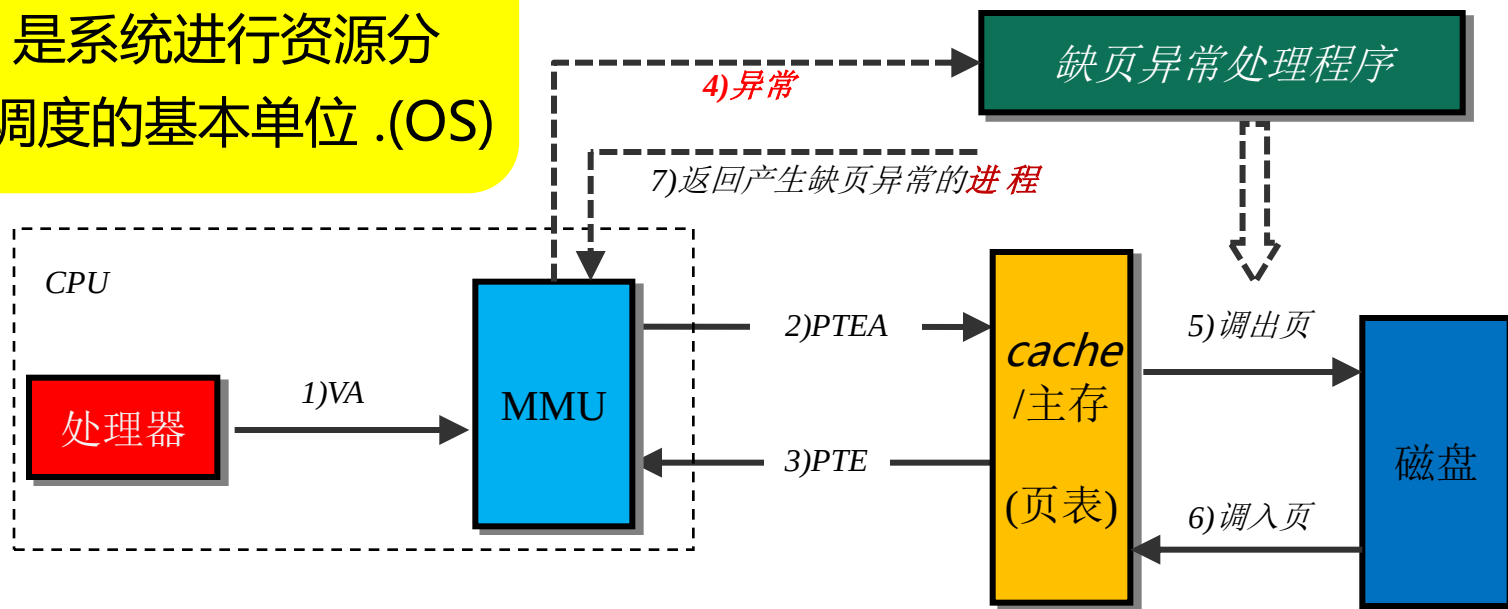
虚拟存储器访问流程：



(a) 页面命中

- 虚存+cache 的系统中, 得到物理地址后, 多访问 cache (教材P143 "4.")
- 访问流程见教材 P142.

■ **进程:** 是程序的基本执行实体, 是系统进行资源分配和调度的基本单位。(OS)

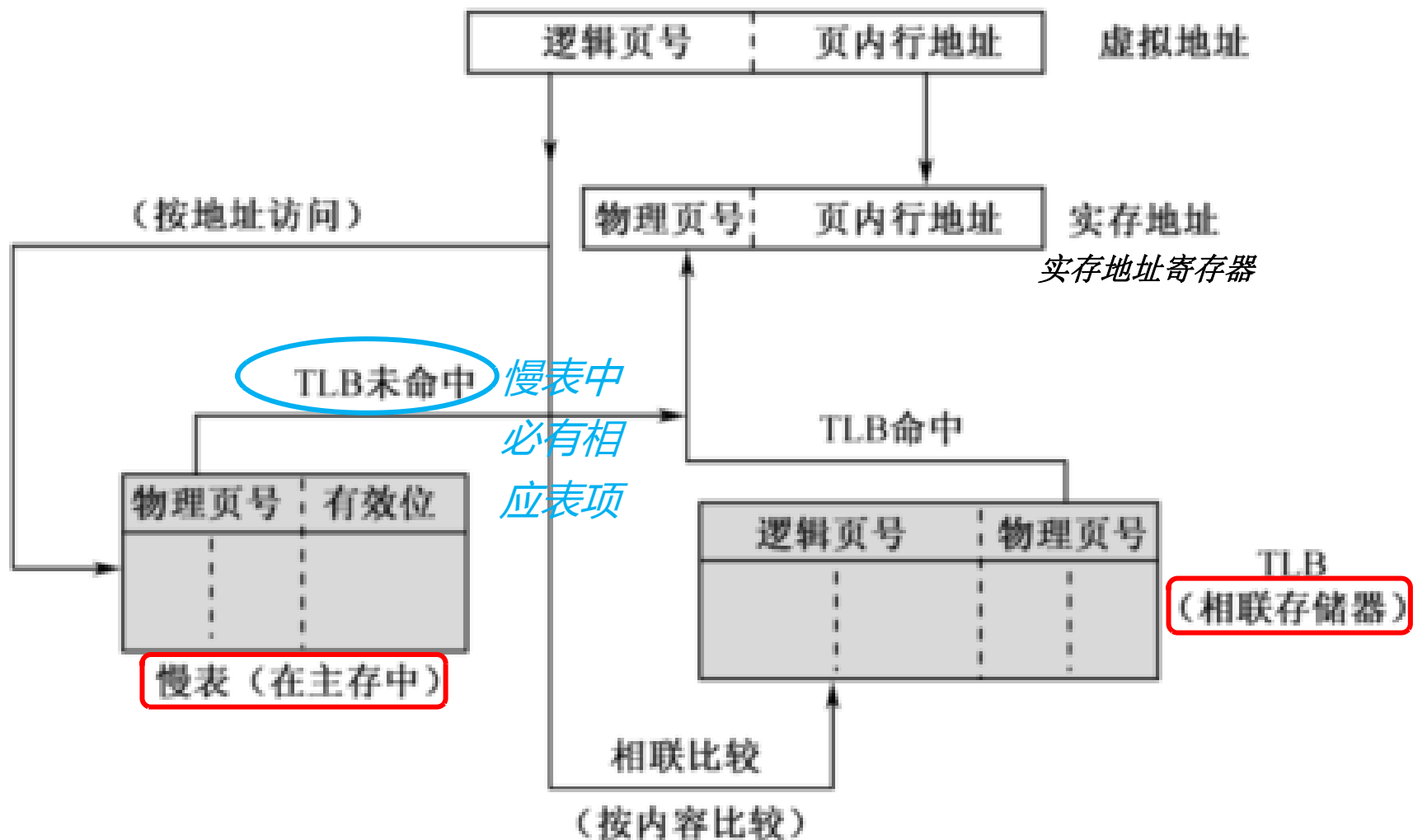


(b) 缺页



- 由于页表通常放在主存中，因而即使虚拟页已经在主存，也至少要访存2次物理存储器才能实现一次访问，这将使虚拟存储器的存取时间加倍。
- 为提高页表查找速度，采用类似cache的思想，引入一个空间小的**快表** (**是整个页表的子集, 是页表映射的一部分**)，来存放**页表中经常被访问的表项**。这个**专用于页表缓存**的小空间的高速存储部件通常称为**转换后援缓冲器(TLB)**。
- 保存在**主存中的完整页表**又可称为**慢表**。
- TLB离CPU更近, 且引入了相联存储器机制，提高了查找速度。

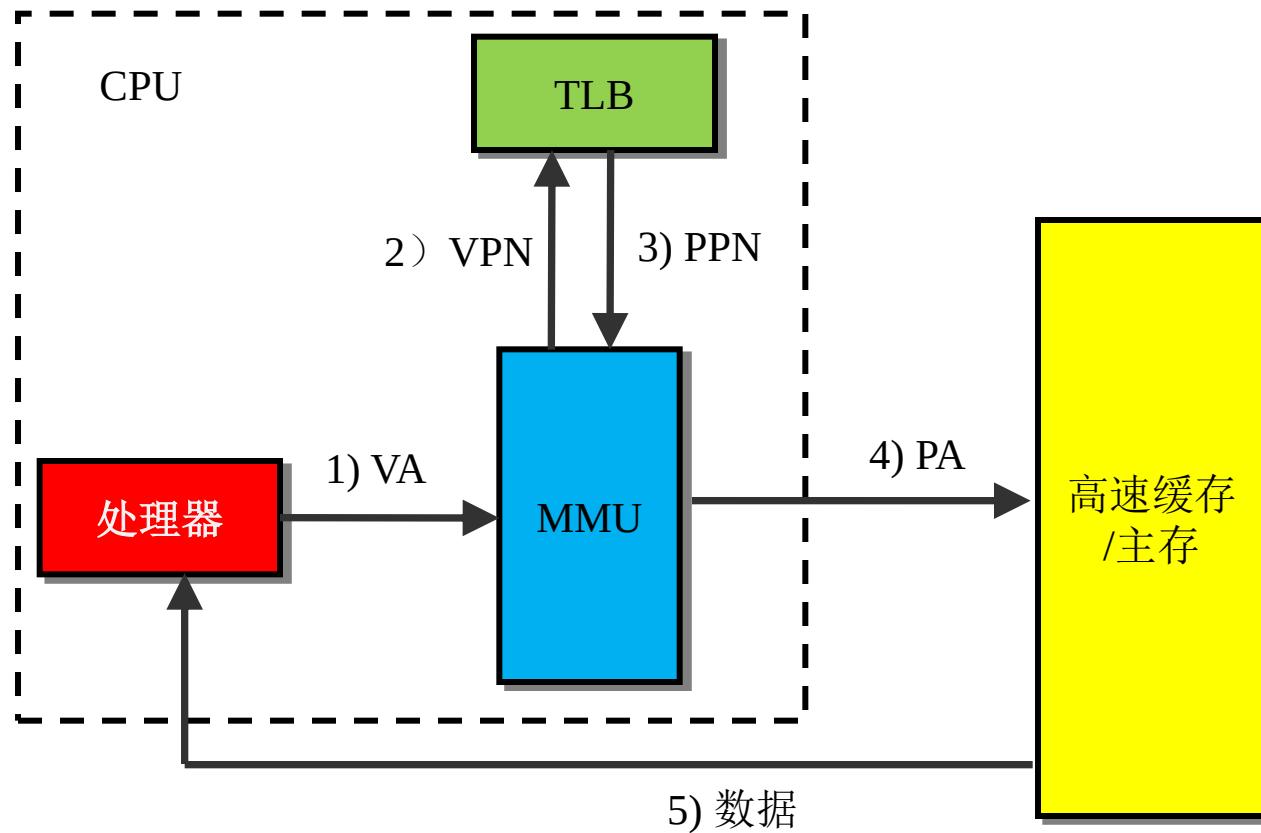
TLB的地址映射过程如下(P144第2行例)：



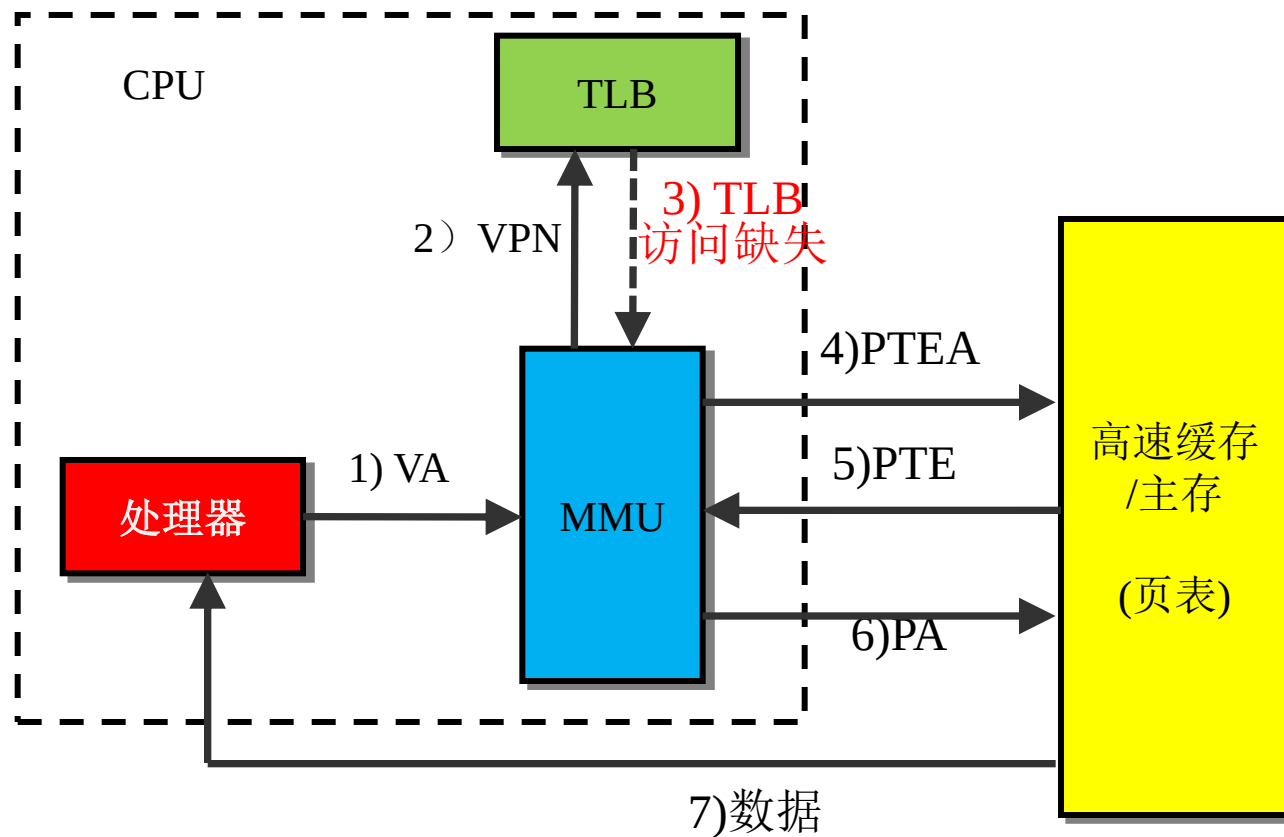
TLB的组织方式：

- 它与cache组织方式类似，多采用 全相联 或 组相联 映射方式，将页表项*映射* 载入到TLB中。
- 当采用*组相联* 方式时，也会将**虚页号** 划分成**TLB标记**和**TLB组索引**两部分，便于快速判断所访问的页表项是否在TLB中。

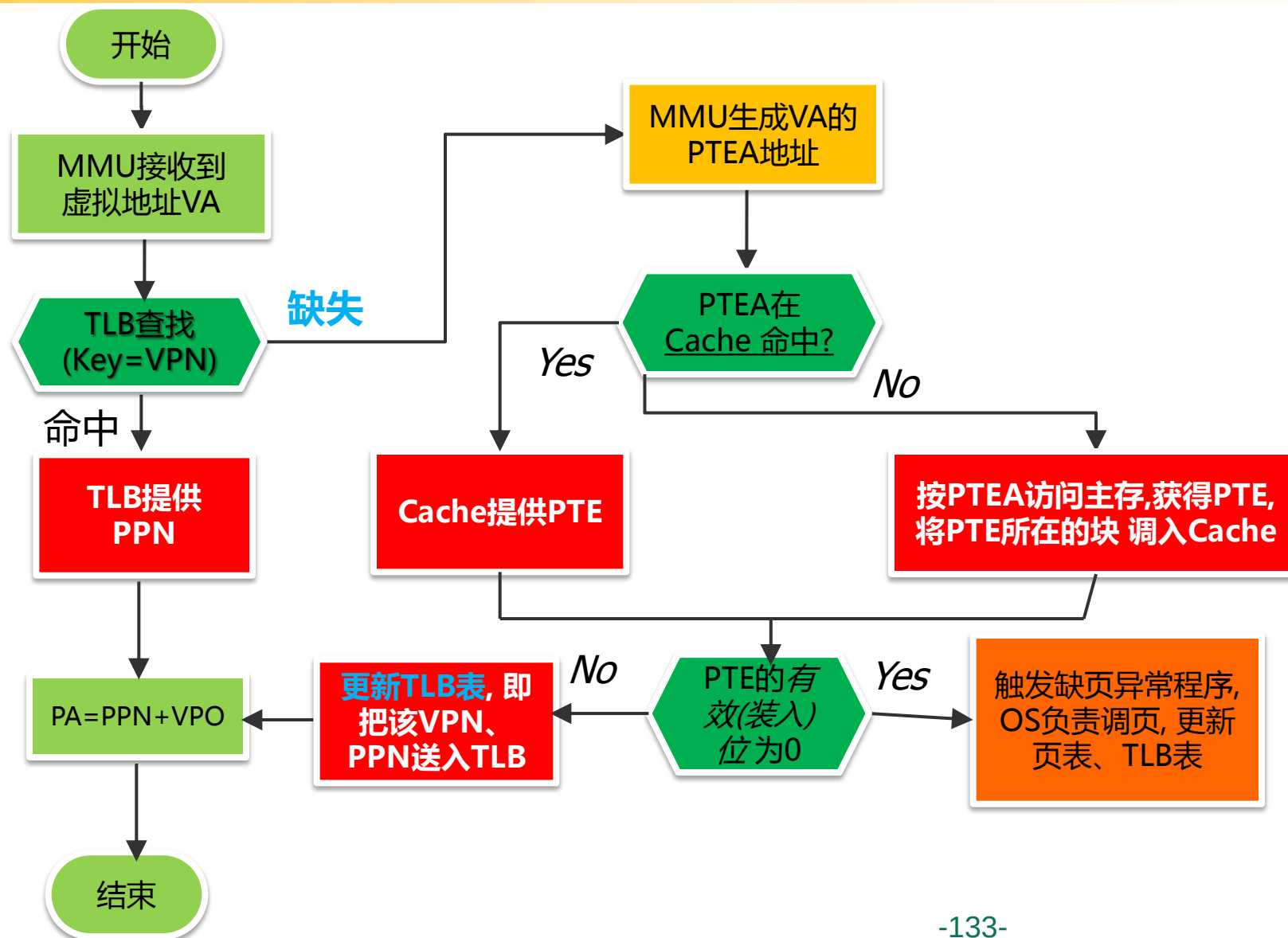
TLB命中：见P144流程步骤



TLB缺失：见P144底下的流程步骤



总结 虚拟地址→物理地址的流程：



小结 虚拟页面的载入过程：

- 程序按需载入，以*页面*为单位载入主存
- 载入页面同时更新*主存/cache*中的(慢)页表
- 主存满时(缺页 即主存满)，完成数据*替换*：
 - 将不经常使用页面淘汰掉(*修改位*)
 - 从辅存把程序或数据调入(替换)得新的页面
 - 页面替换采用**OS软件** 实现 (*cache数据替换由硬件实现*)
- 主存满后的系统性能？
 - *虚拟存储器替换* 算法会涉及磁盘 操作(速度慢)，因此性能损失会大于*cache替换*。



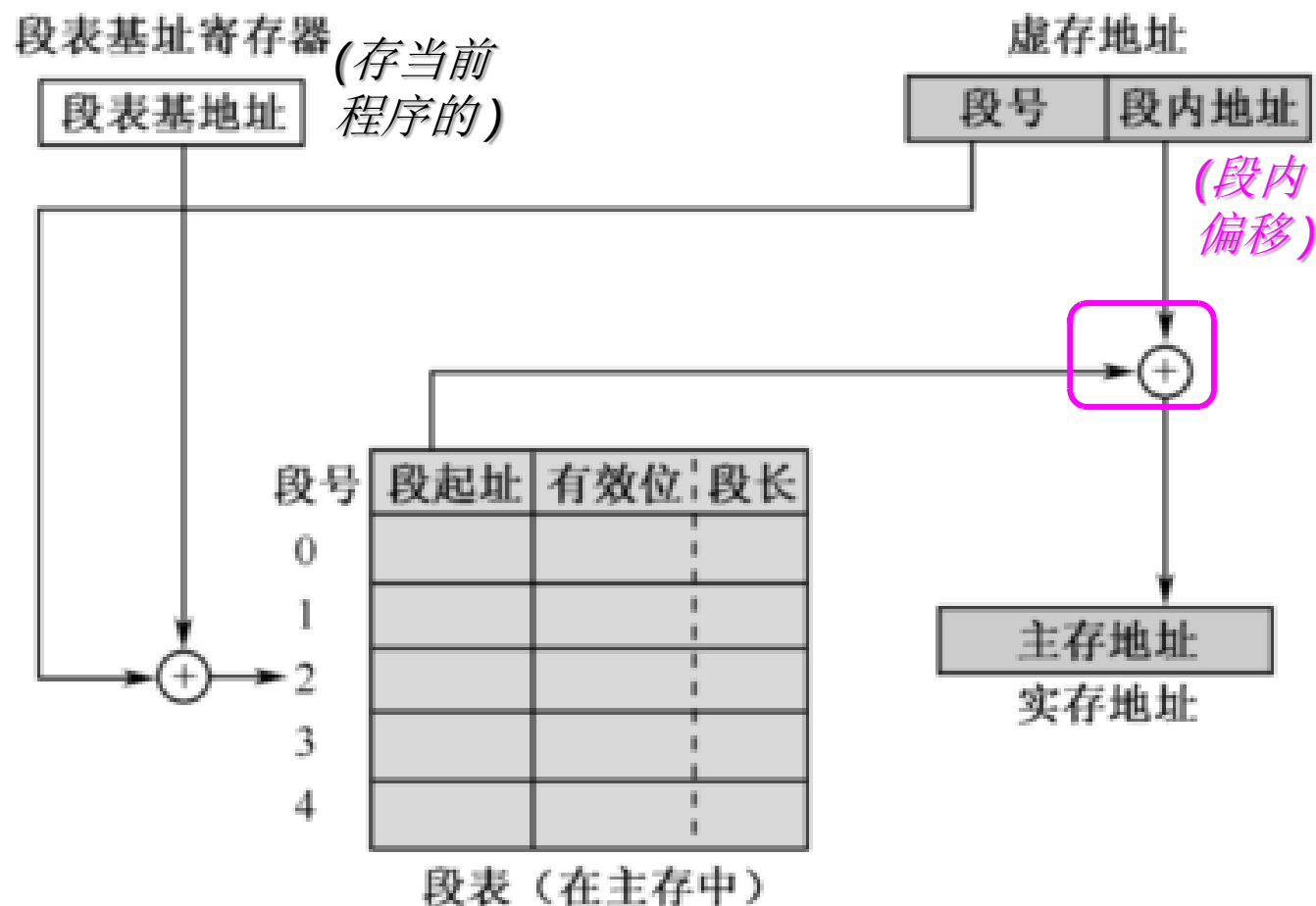
4.6.4 段式虚拟存储器

- 以**段**作为基本信息单位在**主存与辅存之间**进行传送和定位的虚拟存储器称为**段式虚拟存储器**.
- **段**是按照程序的**自然分界**(例如C程序的函数)划分的,是长度可以动态改变的区域。**段长不固定**,可大可小.
- 优点: 段的独立性——易于编译、管理、修改和维护.
- 缺点:**各段的长度不同**,给主存的分配带来麻烦.

段式虚地址向实存地址的变换：



- 每个程序 设置一个 **段表** (一般驻存在主存)，段表的每一个表项对应一个段。每个表项至少包含三个字段：



- 1) **有效位**: 指明该段是否已经调入主存
- 2) **段起址**: 指明在 该段已经调入主存的情况下, 该段 在主存中的 首地址.
- 3) **段长**: 记录该段的实际长度。

作业 4

- 在试题书上完成3.1选择题:
10,11,12,15,16,17,18,19,20,24,25,26,27.
- 本子上完成 :
- 1. 教材习题4.1: LRU算法, LFU算法, 写回法
- 2. 试题书P167: 9 ; 试题书P41: 18
- 3. 教材习题4.12
- 4. 试题书P43: 22(加条件 “设按字节编址” ; 中间结果有印刷错误, 最后结果正确)
- 5. 教材习题 4.15(“**设按字节编址**”), 4.16(**表中: “ 虚页号” 4 对应的 “ 实页号” 改成 1**)。

本章重点

- 存储器的分级结构及相关特点
- SRAM，DRAM工作原理及特点
- 存储器地址译码方式，存储器扩展
- 高速缓冲存储器的组成及工作原理
- 虚拟存储器基本概念及原理。

The End